（三）Gauss-Laguerre (高斯 – 拉盖尔) 求积公式

Laguerre 多项式

$$L_n(x)=\mathrm{e}^x\frac{\mathrm{d}^n}{\mathrm{d}x^n}\left(\mathrm{e}^{-x}x^n\right)\quad(n=0,1,2,\cdots)$$

是区间 $[0,+\infty)$ 上关于权函数 $\rho(x)=\mathrm{e}^{-x}$ 的正交多项式系, 以 n 次 Laguerre 多项式 $L_n(x)$ 的零点 $\{x_k\}$ 为 Gauss 点导出的 Gauss 型求积公式

$$\int_0^{+\infty}\mathrm{e}^{-x}f(x)\mathrm{d}x\approx\sum_{k=1}^{n}A_kf\left(x_k\right)\tag{7.60}$$

称为 Gauss-Laguerre 求积公式. 求积系数的计算公式为

$$A_k=\frac{(n!)^2}{x_k\left[L_n'\left(x_k\right)\right]^2}\quad(k=1,2,\cdots,n)\tag{7.61}$$

求积公式 (7.60) 的截断误差为

$$R(f)=\frac{(n!)^2}{(2n)!}f^{(2n)}(\xi),\quad\xi\in(0,+\infty)\tag{7.62}$$

表 7.5 列出了部分 Gauss-Laguerre 求积公式的节点和求积系数值, 以供查用.

表 7.5　部分 Gauss-Laguerre 求积公式的节点和求积系数值

n	x_k	A_k	$A_k\mathrm{e}^{x_k}$
2	0.585 786 44	0.853 553 39	1.533 326 03
	3.414 213 56	0.146 446 61	4.450 957 34
3	0.415 774 56	0.711 093 01	1.077 692 86
	2.294 280 36	0.278 517 73	2.762 142 96
	6.289 945 08	0.010 389 26	5.601 094 63
4	0.322 547 69	0.603 154 10	0.832 739 12
	1.745 761 10	0.357 418 69	2.048 102 44
	4.536 620 30	0.038 887 91	3.631 146 31
	9.395 070 91	0.000 539 29	6.487 145 08

利用 Gauss-Laguerre 求积公式也可以求积分 $\int_0^{+\infty}g(x)\mathrm{d}x$ 的近似值. 令 $f(x)=\mathrm{e}^xg(x)$, 则

$$\int_0^{+\infty}g(x)\mathrm{d}x=\int_0^{+\infty}\mathrm{e}^{-x}f(x)\mathrm{d}x\approx\sum_{k=1}^{n}A_kf\left(x_k\right)=\sum_{k=1}^{n}A_k\mathrm{e}^{x_k}g\left(x_k\right)\tag{7.63}$$

而对积分 $\int_a^{+\infty}\varphi(t)\mathrm{d}t$, 只要作变换 $x=t-a$, 就有

$$\int_a^{+\infty}\varphi(t)\mathrm{d}t=\int_0^{+\infty}\varphi(x+a)\mathrm{d}x=\int_0^{+\infty}g(x)\mathrm{d}x$$

扫描全能王 创建

按式 (7.63) 即可计算其近似值.

(四) Gauss-Hermite (高斯 – 埃尔米特) 求积公式

求积公式

$$\int_{-\infty}^{+\infty} \mathrm{e}^{-x^2} f(x)\mathrm{d}x \approx \sum_{k=1}^{n} A_k f(x_k) \tag{7.64}$$

称为 Gauss-Hermite 求积公式, 其中节点 x_k $(k=1,2,\cdots,n)$ 为 n 次 Hermite 多项式

$$H_n(x) = (-1)^n \mathrm{e}^{x^2} \frac{\mathrm{d}^n}{\mathrm{d}x^n}\left(\mathrm{e}^{-x^2}\right) \quad (n=0,1,2,\cdots) \tag{7.65}$$

的 n 个零点, 求积系数为

$$A_k = \frac{2^{n+1} n! \sqrt{\pi}}{\left[H_n'(x_k)\right]^2} \quad (k=1,2,\cdots,n) \tag{7.66}$$

因为 Hermite 多项式是 $(-\infty,+\infty)$ 上以 $\rho(x) = \mathrm{e}^{-x^2}$ 为权函数的正交多项式系, 故公式 (7.64) 为 Gauss 型求积公式. 其截断误差为

$$R(f) = \frac{n!\sqrt{\pi}}{2^n (2n)!} f^{(2n)}(\xi), \quad \xi \in (-\infty,+\infty) \tag{7.67}$$

部分 Gauss-Hermite 求积公式的节点与求积系数见表 7.6.

表 7.6 部分 Gauss-Hermite 求积公式的节点与求积系数

n	x_k	A_k	$A_k \mathrm{e}^{x_k^2}$
2	±0.707 106 78	0.886 226 93	1.461 141 18
3	0	1.181 635 90	1.181 635 90
	±1.224 744 87	0.295 408 98	1.323 931 18
4	±0.524 647 62	0.804 914 09	1.059 964 48
	±1.650 680 12	0.081 312 84	1.240 225 82
5	0	0.945 308 72	0.945 308 72
	±0.958 572 46	0.393 619 32	0.986 581 00
	±2.020 182 87	0.019 953 24	1.181 488 63

对积分 $\int_{-\infty}^{+\infty} g(x)\mathrm{d}x$, 令 $f(x) = \mathrm{e}^{x^2} g(x)$, 则

$$\int_{-\infty}^{+\infty} g(x)\mathrm{d}x = \int_{-\infty}^{+\infty} \mathrm{e}^{-x^2} f(x)\mathrm{d}x$$

故只要对 $\mathrm{e}^{x^2} g(x)$ 用 Gauss-Hermite 求积公式, 即可计算 $\int_{-\infty}^{+\infty} g(x)\mathrm{d}x$ 的近似值.

例 7.11 利用 $n=3$ 的 Gauss-Hermite 求积公式计算积分

$$I = \int_0^{+\infty} \mathrm{e}^{-x^3} \sqrt{1+x^2}\, \mathrm{d}x$$

扫描全能王 创建

解　因积分 $\int_0^{+\infty} \mathrm{e}^{-x^2}\sqrt{1+x^2}\,\mathrm{d}x$ 收敛, $\mathrm{e}^{-x^2}\sqrt{1+x^2}$ 是偶函数, 故积分 $\int_{-\infty}^{+\infty} \mathrm{e}^{-x^2}\sqrt{1+x^2}\mathrm{d}x$ 收敛, 且

$$\int_0^{+\infty} \mathrm{e}^{-x^2}\sqrt{1+x^2}\,\mathrm{d}x = \frac{1}{2}\int_{-\infty}^{+\infty} \mathrm{e}^{-x^2}\sqrt{1+x^2}\,\mathrm{d}x$$

查表 7.6 得

$$\begin{aligned}\int_{-\infty}^{+\infty} \mathrm{e}^{-x^2}\sqrt{1+x^2}\,\mathrm{d}x \approx & 1.181\ 635\ 90 \times \sqrt{1+0} + 0.295\ 408\ 98\times \\ & \left[\sqrt{1+(1.224\ 744\ 87)^2} + \sqrt{1+(-1.224\ 744\ 87)^2}\right] \\ = & 1.181\ 635\ 9 + 0.295\ 409\ 0 \times 3.162\ 277\ 7 \\ = & 2.115\ 801\ 2\end{aligned}$$

所以

$$I = \int_0^{+\infty} \mathrm{e}^{-x^2}\sqrt{1+x^2}\,\mathrm{d}x \approx 1.057\ 900\ 6$$

综上所述, 用 Gauss 型求积公式计算积分近似值的步骤为

(1) 把所求积分化成标准形式, 即通过变换将积分区间化成 $[-1,1]$, $[0,+\infty)$ 或 $(-\infty,+\infty)$, 根据被积函数形式, 确定用哪种求积公式;

(2) 选定 n, 查表得节点 $\{x_k\}$ 及求积系数 $\{A_k\}$;

(3) 按求积公式计算积分近似值.

Gauss 型求积公式不仅精度高, 而且可以计算反常积分, 瑕积分亦可化成无穷积分求近似值. 这是前几节所介绍的 N-C 公式, Romberg 方法等无法解决的. 但用 Gauss 型求积公式很难根据事先给定的精度及误差公式确定节点数 n, 并且当节点数 n 增加时, 节点 $\{x_k\}$ 与求积系数 $\{A_k\}$ 都会改变, 需重新查表按新公式计算, 前面计算过的结果不能继续使用, 因而增加了计算量.

7.6　振荡函数的积分

在工程实际中, 常常需要计算振荡函数的积分. 如果被积函数是一个变化激烈的振荡函数, 如

$$\int_a^b f(x)\cos\omega x\,\mathrm{d}x, \quad \int_a^b f(x)\sin\omega x\,\mathrm{d}x \tag{7.68}$$

其中 $f(x)$ 为非振荡函数, ω 为较大的正数. 利用前面给出的求积公式虽然也能计算这两个积分的近似值, 但为得到比较准确的近似值, 必须在每半个周期内都要取到几个节点. 否则, 计算效果可能很不理想. 例如, 用复化梯形公式和复化 Simpson 公式分别计算积分

扫描全能王 创建

$$I=\int_0^{\frac{5}{8}\pi}\sin 20x\ \mathrm{d}x$$

的近似值. 如果取 $n=10, h=\dfrac{\pi}{16}$, 用复化梯形公式和复化 Simpson 公式分别求得近似值为

$$T_{10}=-0.040\ 7,\quad S_{10}=-0.119\ 7$$

这与积分的准确值

$$I=\int_0^{\frac{5}{8}\pi}\sin 20x\ \mathrm{d}x=\frac{1}{20}=0.05$$

相差很大. 事实上, 如果用复化梯形公式计算此积分的近似值, 要求其误差不超过 0.001, 由误差公式

$$|R_n|=\frac{1}{12}\times\frac{5}{8}\pi h^2\times 20^2\sin\xi\leqslant\frac{125\pi}{6}h^2\leqslant 0.001$$

可以估计出 $h\leqslant 0.003\ 908\ 82$, 即至少应取 $n=503$ 个节点. 这样, 节点数比较多, 求积的工作量十分可观.

仔细分析误差公式可以发现, 用复化求积公式计算振荡积分, 之所以要取较多的节点, 原因在于被积函数求导时将出现 ω^2 或 ω^4, 当 ω 较大时, 误差就大. 解决此问题的一个方法是用 $f(x)$ 的插值函数 $\varphi(x)$ 代入式 (7.68), 以积分

$$\int_a^b\varphi(x)\cos\omega x\ \mathrm{d}x,\quad\int_a^b\varphi(x)\sin\omega x\ \mathrm{d}x \tag{7.69}$$

作为积分式 (7.68) 的近似. 特别地, 若用分段低次插值多项式 $\varphi(x)$ 近似 $f(x)$, 积分式 (7.69) 可以精确地求出.

例如, 将区间 $[a,b]$ 分为 n 等份, $h=\dfrac{b-a}{n}, x_i=a+ih\ (i=0,1,\cdots,n)$. 取 $\varphi(x)$ 为 $f(x)$ 的分段线性插值函数, 则有

$$\begin{aligned}
&\int_a^b\varphi(x)\cos\omega x\ \mathrm{d}x\\
=&\sum_{i=0}^{n-1}\int_{x_i}^{x_{i+1}}\varphi(x)\cos\omega x\ \mathrm{d}x\\
=&\sum_{i=0}^{n-1}\int_{x_i}^{x_{i+1}}\left\{f(x_i)+\frac{1}{h}\left[f(x_{i+1})-f(x_i)\right](x-x_i)\right\}\cos\omega x\ \mathrm{d}x\\
=&\sum_{i=0}^{n-1}\left\{\frac{1}{\omega}\left[f(x_i)-\frac{x_i}{h}\left(f(x_{i+1})-f(x_i)\right)\right](\sin\omega x_{i+1}-\sin\omega x_i)+\right.\\
&\left.\frac{1}{\omega h}\left[f(x_{i+1})-f(x_i)\right]\left(x\sin\omega x+\frac{1}{\omega}\cos\omega x\right)\Big|_{x_i}^{x_{i+1}}\right\}
\end{aligned}$$

扫描全能王 创建

$$=\sum_{i=0}^{n-1}\left\{\frac{1}{\omega}\left[f\left(x_{i+1}\right)\sin\omega x_{i+1}-f\left(x_i\right)\sin\omega x_i\right]-\frac{\sin\omega h}{\omega^2 h}\left[f\left(x_{i+1}\right)\sin\omega x_{i+1}-f\left(x_i\right)\sin\omega x_i\right]+\right.$$
$$\left.\frac{2\sin^2\frac{\omega h}{2}}{\omega^2 h}\left[f\left(x_{i+1}\right)\cos\omega x_{i+1}+f\left(x_i\right)\cos\omega x_i\right]\right\}$$
$$=\frac{1}{\omega}\left[f\left(x_n\right)\sin\omega x_n-f\left(x_0\right)\sin\omega x_0\right]-\frac{\sin\omega h}{\omega^2 h}\left[f\left(x_n\right)\sin\omega x_n-f\left(x_0\right)\sin\omega x_0\right]+$$
$$\frac{2}{\omega^2 h}\sin^2\frac{\omega h}{2}\left[f\left(x_n\right)\cos\omega x_n+f\left(x_0\right)\cos\omega x_0\right]+\frac{4}{\omega^2 h}\sin^2\frac{\omega h}{2}\sum_{i=1}^{n-1}f\left(x_i\right)\cos\omega x_i$$

整理上式并令

$$\alpha=\frac{4}{\omega^2 h}\sin^2\frac{\omega h}{2},\quad \beta=\frac{2}{\omega^2 h}\sin\frac{\omega h}{2} \tag{7.70}$$

得

$$\begin{aligned}\int_a^b\varphi(x)\cos\omega x\,\mathrm{d}x=&f\left(x_0\right)\left[-\frac{1}{\omega}\sin\omega x_0+\beta\sin\omega\left(x_0+\frac{h}{2}\right)\right]+\\&f\left(x_n\right)\left[\frac{1}{\omega}\sin\omega x_n-\beta\sin\omega\left(x_n-\frac{h}{2}\right)\right]+\alpha\sum_{i=1}^{n-1}f\left(x_i\right)\cos\omega x_i\end{aligned} \tag{7.71}$$

于是得到求积公式

$$\begin{aligned}\int_a^b f(x)\cos\omega x\,\mathrm{d}x\approx&\int_a^b\varphi(x)\cos\omega x\,\mathrm{d}x\\=&f\left(x_0\right)\left[-\frac{1}{\omega}\sin\omega x_0+\beta\sin\omega\left(x_0+\frac{h}{2}\right)\right]+\\&f\left(x_n\right)\left[\frac{1}{\omega}\sin\omega x_n-\beta\sin\omega\left(x_n-\frac{h}{2}\right)\right]+\alpha\sum_{i=1}^{n-1}f\left(x_i\right)\cos\omega x_i\end{aligned} \tag{7.72}$$

完全类似地有

$$\begin{aligned}\int_a^b f(x)\sin\omega x\,\mathrm{d}x\approx&f\left(x_0\right)\left[\frac{1}{\omega}\cos\omega x_0-\beta\cos\omega\left(x_0+\frac{h}{2}\right)\right]+\\&f\left(x_n\right)\left[-\frac{1}{\omega}\cos\omega x_n+\beta\cos\omega\left(x_n-\frac{h}{2}\right)\right]+\alpha\sum_{i=1}^{n-1}f\left(x_i\right)\sin\omega x_i\end{aligned} \tag{7.73}$$

利用求积公式 (7.73) 计算积分

$$I=\int_0^{\frac{5}{8}\pi}\sin 20x\,\mathrm{d}x$$

取 $n=10,h=\dfrac{\pi}{16}$, 则有

$$\alpha=\frac{64}{400\pi}\sin^2\frac{5\pi}{8}\approx 0.043\ 471\ 1$$

扫描全能王 创建

$$\beta=\frac{32}{400\pi}\sin\frac{5\pi}{8}\approx 0.023\ 526\ 4$$

$$I\approx\frac{1}{20}\cos 0-\beta\cos\left(20\times\frac{\pi}{32}\right)-\frac{1}{20}\cos\left(20\times\frac{5}{8}\pi\right)+\beta\cos\left(20\times\frac{19}{32}\pi\right)+$$

$$\alpha\sum_{i=1}^{9}\sin\left(20\times\frac{i\pi}{16}\right)$$

$$=0.05-0.023\ 526\ 4\times(-0.382\ 683)+0.023\ 526\ 4\times 0.923\ 880+$$

$$0.043\ 471\ 1\times(-0.707\ 107)$$

$$=0.049\ 999\ 99$$

计算结果与准确值十分接近.

由分段线性插值的误差公式容易得到求积公式 (7.72) 和式 (7.73) 的误差为

$$\begin{aligned}\left|\int_a^b f(x)\cos\omega x\,\mathrm{d}x-\int_a^b\varphi(x)\cos\omega x\,\mathrm{d}x\right|&\leqslant(b-a)\max_{a\leqslant x\leqslant b}|f(x)-\varphi(x)|\\&\leqslant\frac{b-a}{8}h^2\max_{a\leqslant x\leqslant b}|f''(x)|\end{aligned}\tag{7.74}$$

$$\left|\int_a^b f(x)\sin\omega x\,\mathrm{d}x-\int_a^b\varphi(x)\sin\omega x\,\mathrm{d}x\right|\leqslant\frac{b-a}{8}h^2\max_{a\leqslant x\leqslant b}|f''(x)|\tag{7.75}$$

需要指出的是: 当 $\omega h=2k\pi$ 或 $\omega h=k\pi,\omega x_0=\left(j+\frac{1}{2}\right)\pi$ (k,j 为正整数) 时, 求积公式 (7.72) 和式 (7.73) 的误差较大. 改进方法可参看文献 [10].

7.7 重积分的数值计算

计算重积分的一个最直接的思路是像计算定积分那样, 用多维插值多项式近似被积函数, 从而求得重积分的近似值. 但是由于重积分积分域和多维插值多项式的复杂性, 得到的近似求积公式将会繁杂得多, 甚至很难得到近似求积公式.

为简便起见, 我们考虑矩形区域 $D=\{(x,y)\mid a\leqslant x\leqslant b,c\leqslant y\leqslant d\}$ (这里 a,b,c,d 是常数) 上的二重积分

$$I=\iint_D f(x,y)\mathrm{d}x\,\mathrm{d}y\tag{7.76}$$

其中 $f(x,y)$ 为 D 上的连续函数.

7.7.1 插值型求积公式

设矩形区域 $D=\{(x,y)\mid a\leqslant x\leqslant b,c\leqslant y\leqslant d\}$ 的剖分为

扫描全能王 创建

$$\Delta_x : a = x_0 < x_1 < \cdots < x_n = b$$

$$\Delta_y : c = y_0 < y_1 < \cdots < y_m = d$$

由第五章公式 (5.49) 得插值多项式为

$$P_{n,m}(x,y) = \sum_{i=0}^{n}\sum_{j=0}^{m} f(x_i,y_j)\, l_i(x)\tilde{l}_j(y)$$

从而有

$$\begin{aligned} I &= \iint_D f(x,y)\mathrm{d}x\,\mathrm{d}y = \int_a^b \mathrm{d}x \int_c^d f(x,y)\mathrm{d}y \\ &\approx \int_a^b \mathrm{d}x \int_c^d \left[\sum_{i=0}^{n}\sum_{j=0}^{m} f(x_i,y_j)\, l_i(x)\tilde{l}_j(y)\right]\mathrm{d}y \\ &= \sum_{i=0}^{n}\sum_{j=0}^{m} f(x_i,y_j)\int_a^b l_i(x)\mathrm{d}x \int_c^d \tilde{l}_j(y)\mathrm{d}y = \sum_{i=0}^{n}\sum_{j=0}^{m} A_{ij} f(x_i,y_j) \end{aligned} \tag{7.77}$$

公式 (7.77) 就是计算二重积分 (7.76) 的 N-C 求积公式, $A_{ij} = \int_a^b l_i(x)\mathrm{d}x \int_c^d \tilde{l}_j(y)\mathrm{d}y$ 为求积系数.

当 $n = m = 1$ 时, 则 $x_0 = a, x_1 = b, y_0 = c, y_1 = d, l_i(x), \tilde{l}_j(y)$ 皆为一次插值基函数, 计算可得 $A_{ij} = \dfrac{1}{4}(b-a)(d-c)$, 求积公式 (7.77) 为

$$\begin{aligned} I &= \int_a^b \mathrm{d}x \int_c^d f(x,y)\mathrm{d}y \\ &\approx \frac{1}{4}(b-a)(d-c)[f(a,c)+f(a,d)+f(b,c)+f(b,d)] \end{aligned}$$

称

$$T(f) = \frac{1}{4}(b-a)(d-c)[f(a,c)+f(a,d)+f(b,c)+f(b,d)] \tag{7.78}$$

为计算二重积分 (7.76) 的梯形公式.

当 $n = m = 2$ 时, 区间 $[a,b]$ 和 $[c,d]$ 均为二等分, 类似可得计算二重积分(7.76) 的 Simpson 公式

$$\begin{aligned} S(f) = &\frac{(b-a)(d-c)}{36}\left[f(a,c) + 4f\left(a,\frac{c+d}{2}\right) + f(a,d) + 4f\left(\frac{a+b}{2},c\right) + \right. \\ &\left. 16f\left(\frac{a+b}{2},\frac{c+d}{2}\right) + 4f\left(\frac{a+b}{2},d\right) + f(b,c) + 4f\left(b,\frac{c+d}{2}\right) + f(b,d)\right] \end{aligned} \tag{7.79}$$

由于公式 (7.78) 和 (7.79) 实际上是关于 x 方向和 y 方向同时使用了定积分的梯形公式或 Simpson 公式, 那么由定积分梯形公式和 Simpson 公式的截断误差(7.26) 和(7.27) , 可以得到式 (7.78) 和(7.79) 的截断误差分别为

扫描全能王 创建

$$\iint_D f(x,y)\mathrm{d}x\ \mathrm{d}y - T(f) = -\frac{(b-a)(d-c)}{12}\left[(b-a)^2\frac{\partial^2 f(\xi_1,\eta_1)}{\partial x^2} + (d-c)^2\frac{\partial^2 f(\xi_2,\eta_2)}{\partial y^2}\right] \tag{7.80}$$

$$\iint_D f(x,y)\mathrm{d}x\ \mathrm{d}y - S(f) = -\frac{(b-a)(d-c)}{180}\left[\frac{(b-a)^4}{16}\frac{\partial^4 f(\xi_1,\eta_1)}{\partial x^4} + \frac{(d-c)^4}{16}\frac{\partial^4 f(\xi_2,\eta_2)}{\partial y^4}\right] \tag{7.81}$$

其中 $(\xi_1,\eta_1),(\xi_2,\eta_2)\in D$.

由误差估计式 (7.80) 和(7.81) 知: 求积公式 (7.78) 和 (7.79) 的截断误差与使用公式 (7.78) 和 (7.79) 的矩形的长和宽有关. 长和宽越小, 截断误差也越小. 因此, 为了提高精确度, 考虑二重积分的复化求积公式.

7.7.2 复化求积公式

设矩形区域 $D=\{(x,y)\mid a\leqslant x\leqslant b, c\leqslant y\leqslant d\}$ 在 x 方向被 n 等分, 在 y 方向被 m 等分, 即 x,y 方向的步长分别为 $h=\dfrac{b-a}{n}, k=\dfrac{d-c}{m}$, 节点

$$x_i = a+ih,\quad i=0,1,\cdots,n$$
$$y_j = c+jk,\quad j=0,1,\cdots,m$$

那么积分

$$I=\iint_D f(x,y)\mathrm{d}x\ \mathrm{d}y = \sum_{i=0}^{n-1}\sum_{j=0}^{m-1}\int_{x_i}^{x_{i+1}}\mathrm{d}x\int_{y_j}^{y_{j+1}} f(x,y)\mathrm{d}y$$

使用梯形公式 (7.78) 得

$$\begin{aligned} T_{n,m}(f) &= \frac{hk}{4}\sum_{i=0}^{n-1}\sum_{j=0}^{m-1}\left[f(x_i,y_j)+f(x_i,y_{j+1})+f(x_{i+1},y_j)+f(x_{i+1},y_{j+1})\right] \\ &= \frac{hk}{4}\sum_{i=0}^{n}\sum_{j=0}^{m} t_{ij} f(x_i,y_j) \end{aligned} \tag{7.82}$$

式 (7.82) 称为二重积分(7.76) 的复化梯形公式, 其中系数 t_{ij} 由表 7.7 给出.

若矩形的等分数 n 和 m 皆为偶数, 类似可得计算二重积分(7.76) 的复化 Simpson 公式

$$\begin{aligned} S_{n,m}(f) =& \frac{hk}{9}\sum_{i=0}^{n/2-1}\sum_{j=0}^{m/2-1}\left[f(x_{2i},y_{2j})+4f(x_{2i},y_{2j+1})+f(x_{2i},y_{2j+2})+\right. \\ & 4f(x_{2i+1},y_{2j})+16f(x_{2i+1},y_{2j+1})+4f(x_{2i+1},y_{2j+2})+ \\ & \left. f(x_{2i+2},y_{2j})+4f(x_{2i+2},y_{2j+1})+f(x_{2i+2},y_{2j+2})\right] \\ =& \frac{hk}{9}\sum_{i=0}^{n}\sum_{j=0}^{m} s_{ij} f(x_i,y_j) \end{aligned} \tag{7.83}$$

扫描全能王 创建

表 7.7　二重积分的复化梯形公式系数

i	j						
	0	1	2	$\cdots$	$m-2$	$m-1$	m
0	1	2	2	$\cdots$	2	2	1
1	2	4	4	$\cdots$	4	4	2
2	2	4	4	$\cdots$	4	4	2
$\vdots$	$\vdots$	$\vdots$	$\vdots$		$\vdots$	$\vdots$	$\vdots$
$n-2$	2	4	4	$\cdots$	4	4	2
$n-1$	2	4	4	$\cdots$	4	4	2
n	1	2	2	$\cdots$	2	2	1

其中系数 s_{ij} 由表 7.8 给出.

表 7.8　二重积分的复化 Simpson 公式系数

i	j									
	0	1	2	3	4	$\cdots$	$m-3$	$m-2$	$m-1$	m
0	1	4	2	4	2	$\cdots$	4	2	4	1
1	4	16	8	16	8	$\cdots$	16	8	16	4
2	2	8	4	8	4	$\cdots$	8	4	8	2
3	4	16	8	16	8	$\cdots$	16	8	16	4
4	2	8	4	8	4	$\cdots$	8	4	8	2
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$		$\vdots$	$\vdots$	$\vdots$	$\vdots$
$n-3$	4	16	8	16	8	$\cdots$	16	8	16	4
$n-2$	2	8	4	8	4	$\cdots$	8	4	8	2
$n-1$	4	16	8	16	8	$\cdots$	16	8	16	4
n	1	4	2	4	2	$\cdots$	4	2	4	1

例 7.12　取 $n=m=4$, 用复化 Simpson 公式计算二重积分 $I=\displaystyle\int_0^{0.2}\mathrm{d}x\int_0^{0.5}\mathrm{e}^{x-y}\,\mathrm{d}y$.

解　将积分域分别关于 x,y 方向 4 等分, 则 $h=0.05$, $k=0.125$, 节点 $x_i=0.05i, y_j=0.125j$ $(i,j=0,1,\cdots,4)$. 各网格点上的系数 s_{ij} 如图 7.4 所示, 那么

$$I=\int_0^{0.2}\mathrm{d}x\int_0^{0.5}\mathrm{e}^{x-y}\,\mathrm{d}y\approx S_{4,4}=\frac{0.125\times0.05}{9}\sum_{i=0}^{4}\sum_{j=0}^{4}s_{ij}\mathrm{e}^{x_i-y_j}$$

$$=\frac{0.006\ 25}{9}\sum_{i=0}^{4}\sum_{j=0}^{4}s_{ij}\mathrm{e}^{0.05i-0.12s_j}$$

$$\approx 0.087\ 115\ 277\ 777\ 78$$

事实上准确值

扫描全能王 创建

$$I=\int_0^{0.2}\mathrm{d}x\int_0^{0.5}\mathrm{e}^{x-y}\,\mathrm{d}y=\left(\mathrm{e}^{0.2}-1\right)\left(1-\mathrm{e}^{-0.5}\right)$$

$$=0.087\ 115\ 197\ 191\ 09\cdots$$

可见近似值 $S_{4,4}$ 已有 5 位有效数字.

实际计算中, 将二重积分化为

$$I=\int_a^b\mathrm{d}x\int_c^d f(x,y)\mathrm{d}y=\int_a^b F(x)\mathrm{d}x$$

其中 $F(x)=\int_c^d f(x,y)\mathrm{d}y$, 再分别对积分变量 x 在区间 $[a,b]$ 上、y 在区间 $[c,d]$ 上使用 Gauss 型求积公式, 也可得到高精度的二重积分求积公式. 详细可参阅文献 [24].

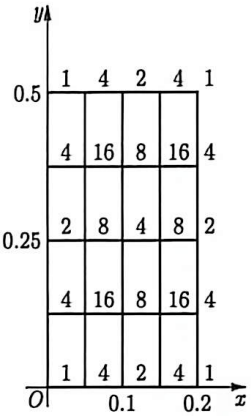


图 7.4

7.8 算法实现与应用实例

7.8.1 逐次分半 Simpson 算法的实现

根据算法 7.2 , 编写逐次分半 Simpson 算法计算例 7.6 中的积分 $I=\int_0^1\frac{\sin x}{x}\,\mathrm{d}x$, 要求误差不超过 10^{-14}. 利用数值软件编写脚本函数如下:

```
function y=f(x)
 if  x==0
      y=1 ;
 else
      y=sin(x) · /x ;
 end
```

```
function I = simpsonstep ( a,b,eps)
h = b − a; c = (a + b)/2; m = 1; h = h/2;
f1=f(a)+f(b) ; f2=f(c) ; f3=0 ;
s0 = 0; s = h/6 ∗ (f1 + 4 ∗ f2);
while (abs(s − s0) >= 15 ∗ eps)
m = m + 1; h = h/2; n = 2^m, f2 = f2 + f3; s0 = s;
x = a + h : 2 ∗ h : b − h; f3 = sum(f(x));
s = h/3 ∗ (f1 + 2 ∗ f2 + 4 ∗ f3);
end
I = s;
```


扫描全能王 创建

然后调用函数 simpsonstep( ):

$\gg I = \text{simpsonstep}\ (0, 1, 1\text{e}-14);$

计算结果列于表 7.9 中.

表 7.9　计 算 结 果

m	n	h	I
2	4	0.250 000 00	0.946 086 933 951 79
3	8	0.125 000 00	0.946 083 310 888 47
4	16	0.062 500 00	0.946 083 085 384 95
5	32	0.031 250 00	0.946 083 071 305 56
6	64	0.015 625 00	0.946 083 070 425 83
7	128	0.007 812 50	0.946 083 070 370 85
8	256	0.003 906 25	0.946 083 070 367 41
9	512	$1.953\ 125\ 00 \times 10^{-3}$	0.946 083 070 367 20
10	1024	$9.765\ 625\ 0 \times 10^{-4}$	0.946 083 070 367 18
11	512	$4.882\ 812\ 5 \times 10^{-4}$	0.946 083 070 367 18

7.8.2　RC 电路的电压

如图 7.5, 考虑由电阻器、电容器、电源和电压器组成的简单 RC 电路. 设导线内的电流为 $u(t)$, 电压器两端的电压为 $y(t)$. 设电压的总量经过整个环路下降到零, 由 Kirchhoff (基尔霍夫) 电压原理, 知电压可表示为

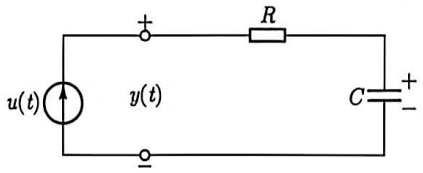


图 7.5

$$y(t) = Ru(t) + x(t)$$

其中第一项表示电流经过电阻时电压的损耗, 第二项 $x(t)$ 表示电流经过电容器时电压的损耗. 由于电容器两端电压的变化率正比于流经电容器的电流, 当 C 表示为电容时, 有

$$C\frac{\mathrm{d}x(t)}{\mathrm{d}t} = u(t)$$

设电容器是在时刻 $t = 0$ 时启动, 两端电压具有初值 $x(0)$. 对上述微分方程两端积分, 得电容器两端电压

$$x(t) = x(0) + \frac{1}{C}\int_0^t u(\tau)\mathrm{d}\tau$$

从而电压器两端电压为

$$y(t) = Ru(t) + x(0) + \frac{1}{C}\int_0^t u(\tau)\mathrm{d}\tau$$

扫描全能王 创建

假设电流函数为 $u(t)=\dfrac{10te^{-t}}{1+\cos^2 t}$ mA, 电阻 $R=33\ \Omega$, 电容 $C=470\ \mu$F, 电容器两端电压初值 $x(0)=2$ V, 计算各时刻的电压值 $y(t)$. 将时间区间 $[0,10]$ 分成 100 等份, 然后用复化梯形公式计算电压 $y(t)$ 在各个时间节点 t_i $(i=0,1,2,\cdots,100)$ 上的近似值. 用数值软件编程计算如下:

```
function y=f(t)
   y = t. * exp(-t)./(100 * (1 + cos(t). * cos(t)))   % 被积函数
end
```

```
function rcint =rcint(n)
T = 10; R = 33;   C = 470.e - 6;   m = 10; h1 = T/n;
t = 0 : h1 : T;   x =  zeros (n + 1, 1); y = zeros(n + 1, 1);
h2 =  h1/m;
ft = f(t);
x(1) = 2.0; y(1) = R * ft(1) + x(1);
 for i = 2 : n + 1
   dx = (ft(i - 1) + ft(i))/2.0;
   for j = 1 : m - 1   dx = dx + f(t(i - 1) + j *  h2);    end
   dx = dx *  h2/C;
x(i) = x(i - 1) + dx;
   y(i) = R * ft(i) + x(i);
end
plot(t,y,'k-');
xlabel('t/s');ylabel('y(t)/V');
```

计算得电压 $y(t)$ 变化如图 7.6 所示.

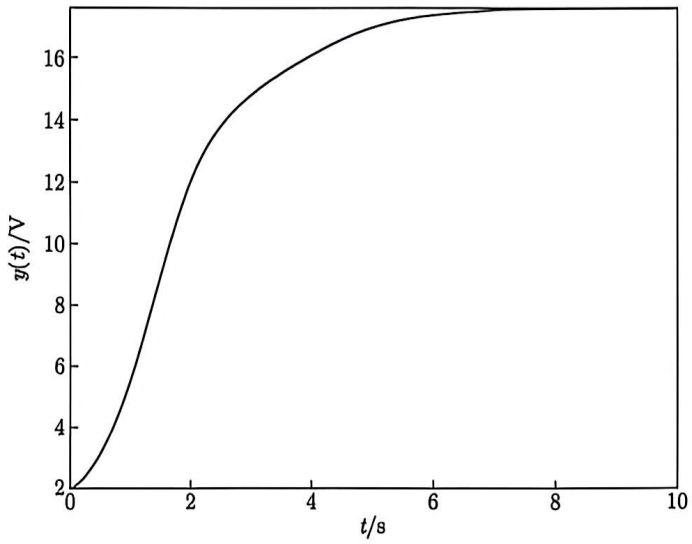


图 7.6 RC 电路电压图

扫描全能王 创建

由于当 $t \to \infty$ 时, $u(t) \to 0$, 故 $y(t)$ 的稳态值就是电容器两端电压 $x(t)$ 的稳态值, 此问题中稳态值 $x = 17.61$ V. 电压 $y(t)$ 在各个时间节点 t_i $(i = 0, 1, 2, \cdots, 100)$ 处的近似值如下:

y = 2.000 000 000 000 00　2.064 887 101 212 77　2.215 773 449 959 87　...
17.610 558 304 190 03　17.611 279 128 185 24　⋯　17.613 756 833 928 92

根据计算结果, $t = 10$ s 时, 电压 $y(10) = 17.613\,756\,833\,9$ 已趋于稳定值.

7.8.3　利用数值积分求解积分方程

在工程具体问题中, 常会遇到如下形式的方程

$$y(x) = \int_a^b K(x,t)y(t)\mathrm{d}t + f(x) \tag{7.84}$$

其中 $K(x,t)$ 和 $f(x)$ 是已知函数, $y(x)$ 是未知函数. 由于积分号下含有未知函数, 故称式 (7.84) 为积分方程. 积分方程的求解往往很困难, 利用数值积分法, 可求得近似解.

若对于区间 $[a,b]$ 上的积分采用数值求积公式 $I(g) = \sum_{k=0}^{n} A_k g(x_k)$, 其中 x_k 和 A_k $(k = 0, 1, 2, \cdots, n)$ 分别为求积节点和求积系数. 那么上述积分方程中的积分项近似为

$$\int_a^b K(x,t)y(t)\mathrm{d}t \approx \sum_{k=0}^{n} A_k K(x, x_k)\, y(x_k), \quad x \in [a,b]$$

从而积分方程可近似为

$$y(x) \approx \sum_{k=0}^{n} A_k K(x, x_k)\, y(x_k) + f(x), \quad x \in [a,b]$$

设 $\tilde{y}(x)$ 为 $y(x)$ 的近似函数, $\tilde{y}_k$ 表示 $y(x_k)$ 的近似值. 由上述近似方程, 得

$$\tilde{y}(x) = \sum_{k=0}^{n} A_k K(x, x_k)\, \tilde{y}(x_k) + f(x), \quad x \in [a,b]$$

记 $K_{ik} = K(x_i, x_k)$, $f_i = f(x_i)$ $(i, k = 0, 1, 2, \cdots, n)$, 得线性代数方程组

$$\tilde{y}_i = \sum_{k=0}^{n} A_k K_{ik} \tilde{y}_k + f_i, \quad i = 0, 1, 2, \cdots, n$$

若记

$$\boldsymbol{K} = \begin{bmatrix} A_0K_{00} & A_1K_{01} & \cdots & A_nK_{0n} \\ A_0K_{10} & A_1K_{11} & \cdots & A_nK_{1n} \\ \vdots & \vdots & & \vdots \\ A_0K_{n0} & A_1K_{n1} & \cdots & A_nK_{nn} \end{bmatrix}$$

$\boldsymbol{f} = (f_0, f_1, f_2, \cdots, f_n)^{\mathrm{T}}$, $\tilde{\boldsymbol{y}} = (\tilde{y}_0, \tilde{y}_1, \cdots, \tilde{y}_n)^{\mathrm{T}}$, $\boldsymbol{M} = \boldsymbol{I} - \boldsymbol{K}$, 线性代数方程组可表示

扫描全能王 创建

为矩阵形式 $M\tilde{y}=f$. 若系数矩阵 M 非奇异, 线性代数方程组存在唯一解 $\tilde{y}=(\tilde{y}_0,\tilde{y}_1,\cdots,\tilde{y}_n)^{\mathrm{T}}$, 从而得积分方程的近似表达式.

例如, 分别用梯形求积公式和 Simpson 求积公式及其复化求积公式求解积分方程

$$y(x)=\int_0^1\left(\frac{1}{1+t}-x\right)y(t)\mathrm{d}t+\frac{4x^3+5x^2-2x+5}{8(x+1)^2}$$

1. 用梯形公式离散方程中的积分项, 即

$$\int_0^1\left(\frac{1}{1+t}-x\right)y(t)\mathrm{d}t\approx\frac{1}{2}\left[(1-x)\tilde{y}_0+\left(\frac{1}{2}-x\right)\tilde{y}_1\right]$$

从而有

$$\tilde{y}_t(x)=\frac{1}{2}\left[(1-x)\tilde{y}_0+\left(\frac{1}{2}-x\right)\tilde{y}_1\right]+\frac{4x^3+5x^2-2x+5}{8(x+1)^2}$$

分别令 $x=0,1$, 得方程组

$$\begin{cases}\tilde{y}_0=\dfrac{1}{2}\left(\tilde{y}_0+\dfrac{1}{2}\tilde{y}_1\right)+\dfrac{5}{8}\\ \tilde{y}_1=\dfrac{1}{2}\left(-\dfrac{1}{2}\tilde{y}_1\right)+\dfrac{3}{8}\end{cases}$$

求得解 $\tilde{y}_0=0.3,\tilde{y}_1=1.4$, 代入 $\tilde{y}_t(x)$ 的表达式, 得

$$\tilde{y}_t(x)=-\frac{7x^3+6x^2-9x-28}{20(x+1)^2}$$

2. 若用 Simpson 公式求解积分方程, 那么

$$\int_0^1\left(\frac{1}{1+t}-x\right)y(t)\mathrm{d}t\approx\frac{1}{6}\left[(1-x)\tilde{y}_0+4\left(\frac{2}{3}-x\right)\tilde{y}_1+\left(\frac{1}{2}-x\right)\tilde{y}_2\right]$$

有

$$\tilde{y}_s(x)=\frac{1}{6}\left[(1-x)\tilde{y}_0+4\left(\frac{2}{3}-x\right)\tilde{y}_1+\left(\frac{1}{2}-x\right)\tilde{y}_2\right]+\frac{4x^3+5x^2-2x+5}{8(x+1)^2}$$

当 $x=0,\dfrac{1}{2},1$ 时, 得线性方程组

$$\begin{cases}\tilde{y}_0=\dfrac{1}{6}\left(\tilde{y}_0+\dfrac{8}{3}\tilde{y}_1+\dfrac{1}{2}\tilde{y}_2\right)+\dfrac{5}{8}\\ \tilde{y}_1=\dfrac{1}{6}\left(\dfrac{1}{2}\tilde{y}_0+\dfrac{2}{3}\tilde{y}_1\right)+\dfrac{23}{72}\\ \tilde{y}_2=\dfrac{1}{6}\left(-\dfrac{4}{3}\tilde{y}_1-\dfrac{1}{2}\tilde{y}_2\right)+\dfrac{3}{8}\end{cases}$$

经求解得 $\tilde{y}_0=1.017\ 845\ 117\ 8,\tilde{y}_1=0.454\ 797\ 979\ 8,\tilde{y}_2=0.252\ 861\ 952\ 9$, 代入 $\tilde{y}_s(x)$ 的表达式, 得

扫描全能王 创建

$$\tilde{y}_9(x) = -\frac{4\ 972x^3 + 5\ 587x^2 - 3\ 742x - 39\ 997}{35\ 640(x+1)^2}$$

此积分方程的精确解为 $y(x) = \dfrac{1}{(x+1)^2}$, 两个近似函数与精确解形式上有很大差异, 近似函数与精确解的存在区间 $[0,1]$ 上的曲线如图 7.7 所示, 两个近似函数的效果并不理想.

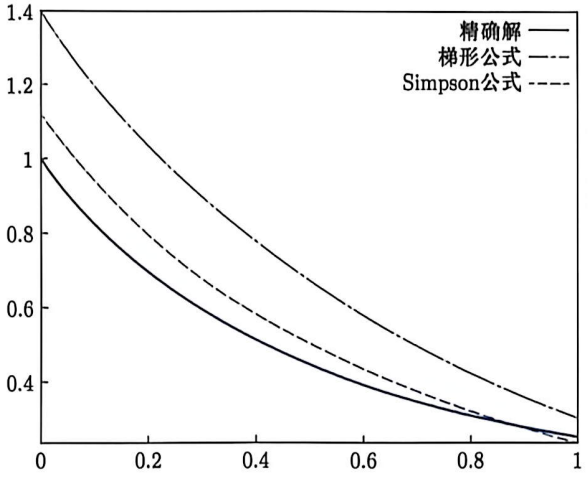


图 7.7

3. 如果取 $n=4$, 采用复化梯形求积公式计算, 则有

$$\tilde{y}_{t4}(x) = \frac{1}{8}\left[(1-x)\tilde{y}_0 + 2\left(\frac{4}{5}-x\right)\tilde{y}_1 + 2\left(\frac{2}{3}-x\right)\tilde{y}_2 + 2\left(\frac{4}{7}-x\right)\tilde{y}_3 + \left(\frac{1}{2}-x\right)\tilde{y}_4\right] + \frac{4x^3+5x^2-2x+5}{8(x+1)^2}$$

分别取 $x = 0, \dfrac{1}{4}, \dfrac{1}{2}, \dfrac{3}{4}, 1$, 得线性方程组

$$\begin{bmatrix} \tilde{y}_0 \\ \tilde{y}_1 \\ \tilde{y}_2 \\ \tilde{y}_3 \\ \tilde{y}_4 \end{bmatrix} = \begin{bmatrix} 8 & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{16} \\ \frac{3}{32} & \frac{11}{80} & \frac{5}{48} & \frac{9}{112} & \frac{3}{32} \\ \frac{1}{16} & \frac{3}{40} & \frac{1}{24} & \frac{1}{56} & 0 \\ \frac{3}{32} & \frac{1}{80} & -\frac{1}{48} & -\frac{5}{112} & -\frac{3}{32} \\ 0 & -\frac{1}{20} & -\frac{1}{12} & -\frac{3}{28} & -\frac{1}{16} \end{bmatrix} \begin{bmatrix} \tilde{y}_0 \\ \tilde{y}_1 \\ \tilde{y}_2 \\ \tilde{y}_3 \\ \tilde{y}_4 \end{bmatrix} + \begin{bmatrix} \frac{5}{8} \\ \frac{39}{100} \\ \frac{23}{72} \\ \frac{16}{49} \\ \frac{3}{8} \end{bmatrix}$$

求解线性方程组得

扫描全能王 创建

$$\tilde{y}=\begin{bmatrix}1.038\ 424\ 081\ 591\ 44\\0.683\ 191\ 582\ 099\ 52\\0.461\ 654\ 118\ 689\ 67\\0.382\ 659\ 883\ 735\ 29\\0.245\ 995\ 266\ 899\ 01\end{bmatrix}$$

代入 $\tilde{y}_{t4}(x)$ 的表达式, 得

$$\tilde{y}_{t4}(x)=\frac{-0.042\ 429x^3-0.046\ 434x^2+0.034\ 419x+1.038\ 424}{(x+1)^2}$$

近似函数 $\tilde{y}_{t4}(x)$ 与精确解仍有很大差别, 结果不理想.

4. 如果取 $n=4$, 采用复化 Simpson 求积公式计算, 有

$$\begin{aligned}\tilde{y}_{s4}(x)=&\frac{1}{12}\left[(1-x)\tilde{y}_0+4\left(\frac{4}{5}-x\right)\tilde{y}_1+2\left(\frac{2}{3}-x\right)\tilde{y}_2+\right.\\&\left.4\left(\frac{4}{7}-x\right)\tilde{y}_3+\left(\frac{1}{2}-x\right)\tilde{y}_4\right]+\frac{4x^3+5x^2-2x+5}{8(x+1)^2}\end{aligned}$$

分别取 $x=0,\frac{1}{4},\frac{1}{2},\frac{3}{4},1$, 得如下的线性方程组

$$\begin{bmatrix}\tilde{y}_0\\\tilde{y}_1\\\tilde{y}_2\\\tilde{y}_3\\\tilde{y}_4\end{bmatrix}=\begin{bmatrix}\frac{1}{12}&\frac{4}{15}&\frac{1}{9}&\frac{4}{21}&\frac{1}{24}\\\frac{1}{16}&\frac{11}{60}&\frac{5}{72}&\frac{3}{28}&\frac{1}{48}\\\frac{1}{24}&\frac{1}{10}&\frac{1}{36}&\frac{1}{42}&0\\\frac{1}{48}&\frac{1}{60}&-\frac{1}{72}&-\frac{5}{84}&-\frac{1}{48}\\0&-\frac{1}{15}&-\frac{1}{18}&-\frac{1}{7}&-\frac{1}{24}\end{bmatrix}\begin{bmatrix}\tilde{y}_0\\\tilde{y}_1\\\tilde{y}_2\\\tilde{y}_3\\\tilde{y}_4\end{bmatrix}+\begin{bmatrix}\frac{5}{8}\\\frac{39}{100}\\\frac{23}{72}\\\frac{16}{49}\\\frac{3}{8}\end{bmatrix}$$

求解线性方程组, 得

$$\tilde{y}=\begin{bmatrix}1.001\ 780\ 540\ 130\ 38\\0.641\ 414\ 181\ 527\ 15\\0.445\ 492\ 267\ 368\ 35\\0.327\ 212\ 076\ 565\ 57\\0.250\ 315\ 105\ 717\ 43\end{bmatrix}$$

将其代入 $\tilde{y}_{s4}(x)$ 的表达式, 取 6 位小数位得

$$\tilde{y}_{s4}(x)=\frac{0.001\ 465x^3+0.001\ 150x^2-0.002\ 096x+1.001\ 781}{(x+1)^2}$$

比较 $\tilde{y}_{s4}(x)$ 与精确解, $\tilde{y}_{s4}(x)$ 的分子中各项系数误差已小于 0.002 1. 由图 7.8 可见, 近似

扫描全能王 创建

函数 $\tilde{y}_{s4}(x)$ 与精确解的图形已完全重合, 经计算近似解的误差小于 0.002, 若要求更高的精度, 可再增加节点.

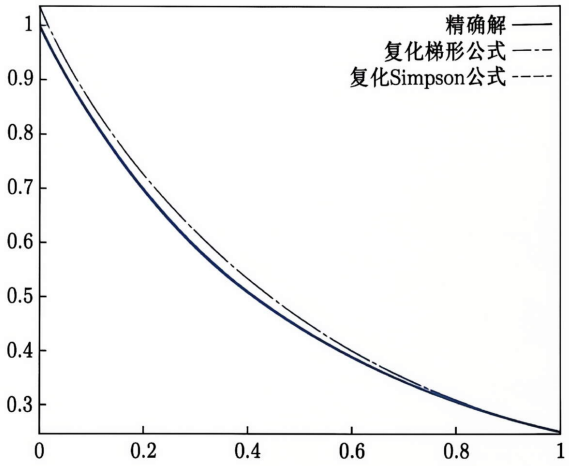


图 7.8

评注七

本章介绍了数值微分的最基本的差商型近似计算公式和插值型近似计算公式 (含三次样条求导公式). 数值微分公式还可由 Taylor 展开法和数值积分得到. 若将 Taylor 展开式展开至待求的一阶或二阶导数, 得到的数值微分公式与差商型公式一致; 若将 Taylor 展开式展开至比待求导数高两阶的导数, 并用待求导数的差商近似更高阶的导数, 得到数值微分的隐式格式, 且精度可提高到 $O(h^4)$. 利用微分与积分运算的互逆性, 可借助数值积分公式计算数值微分. 若采用 Simpson 求积公式, 同样可得到具有精度 $O(h^4)$ 的隐式格式. 数值微分的隐式格式需要求解一个主对角严格占优的三对角线性方程组, 用追赶法求解是数值稳定的. 结合外推技术, 也可大大地提高数值微分的精度. 详细可参阅文献 [24].

本章重点讨论了 Newton-Cotes 求积公式、Gauss 型求积公式和 Romberg 求积公式, 它们都是以插值法为基础的. Newton-Cotes 求积公式中的节点是等距的, 高阶 Newton-Cotes 求积公式 ($n \geqslant 8$) 是数值不稳定的, 通常使用低阶复化求积公式, 实际计算中常采用逐次分半或自适应的梯形算法和 Simpson 算法. Gauss 型求积公式中的节点是正交多项式的零点, 通常不是等距的, 具有 $2n-1$ 次代数精确度, 且是数值稳定的. 实际计算中低阶复化 Gauss 型求积公式也是很有效的. Richardson 外推法是数值计算中提高计算精度的一种重要技术, 由此产生的 Romberg 求积方法计算简单, 精度较高, 是实际计算中的常用方法.

本章对振荡函数的积分作了简单的讨论. 也可对被积函数中的非振荡部分用分段二次插值近似, 得到振荡函数积分的近似求积公式; 或先以被积函数中振荡部分的零点分割积分区间, 再在每两个相邻零点构成的子区间上, 运用数值积分公式. 本章未涉及奇异积分的计算. 奇异积分有时可通过变量置换消去奇点, 或通过区间截断和极限过程计算积分近似值, 无穷区间上的积分可利用 Gauss-Laguerre 求积公式和 Gauss-Hermite 求积公式计算.

扫描全能王 创建

本章知识要点 梯形公式；Simpson 公式；代数精确度；截断误差；复化积分公式；逐次分半算法；Gauss 型积分公式.

算法历史及背后的数学家

习题七

1. 已知函数表如下:

x_i	101	102	103
$f(x_i)$	10.049 88	10.099 50	10.148 89

用三点公式求 $f'(101), f'(102)$.

2. 已知函数 $y = \mathrm{e}^x$ 的函数值如下:

x_i	2.5	2.6	2.7	2.8	2.9
$y_i = \mathrm{e}^{x_i}$	12.182 5	13.463 7	14.879 7	16.444 6	18.174 1

取 $h = 0.1, h = 0.2$, 分别用三点公式计算 $x = 2.7$ 处的一、二阶导数值.

3. 设 $f(x) = \dfrac{1}{(1+x)^2}$, 取步长 $h = 0.1$, 分别用二点和三点公式计算 $f'(0.5)$, 并估计误差.

4. 分别用梯形公式和 Simpson 公式计算下列积分的近似值并估计误差:

(1) $\displaystyle\int_0^1 \mathrm{e}^{-x}\,\mathrm{d}x$;

(2) $\displaystyle\int_{0.1}^{0.5} \mathrm{e}^{\frac{1}{x}}\,\mathrm{d}x$.

5. 求 Newton-Cotes 系数 $C_k^{(3)}$ $(k = 0, 1, 2, 3)$.

6. 试证明 $n+1$ 个节点的 Newton-Cotes 系数满足

$$\sum_{k=0}^{n} C_k^{(n)} = 1$$

7. 设 $f''(x) < 0, x \in [a, b]$, 试证明用梯形公式计算积分 $\displaystyle\int_a^b f(x)\mathrm{d}x$ 所得近似值小于准确值, 并说明其几何意义.

扫描全能王 创建

8. 求近似公式

$$\int_0^1 f(x)\mathrm{d}x \approx \frac{1}{3}\left[2f\left(\frac{1}{4}\right) - f\left(\frac{1}{2}\right) + 2f\left(\frac{3}{4}\right)\right]$$

的代数精确度.

9. 分别用 $n=10$ 的复化梯形公式和复化 Simpson 公式计算积分

$$I = \int_0^1 \mathrm{e}^{-x^2}\,\mathrm{d}x$$

的近似值, 并估计误差.

10. 分别用复化梯形公式和复化 Simpson 公式计算下列积分的近似值:

(1) $\int_0^1 \frac{x}{1+x^2}\,\mathrm{d}x, n=8;$

(2) $\int_0^1 \frac{\ln(1+x)}{(1+x)^2}\,\mathrm{d}x, n=4.$

11. 若用复化梯形公式计算积分 $\int_0^1 \mathrm{e}^{-x^2}\,\mathrm{d}x$, 问积分区间要等分为多少份才能保证计算结果有四位有效数字（假定计算过程无舍入误差）? 若用复化 Simpson 公式计算, 积分区间应等分为多少份?

12. 根据函数表

i	0	1	2	3	4	5	6	7	8
x_i	0	0.125	0.25	0.375	0.5	0.625	0.75	0.875	1
$f(x_i)$	1	0.997 398	0.989 616	0.976 727	0.958 851	0.936 156	0.908 852	0.877 193	0.841 471

计算积分 $I = \int_0^1 f(x)\mathrm{d}x$ 的近似值, 并估计误差.

13. 试用梯形公式的逐次分半算法计算下列积分:

(1) $\int_0^{\pi} \frac{\mathrm{d}x}{x+\cos x};$

(2) $\int_0^1 \frac{\mathrm{d}x}{1+x^3}.$

要求误差不超过 $\varepsilon = 3\times 10^{-4}$.

14. 用 Romberg 方法计算积分

$$I = \int_0^4 \sqrt{1+\cos^2 x}\,\mathrm{d}x$$

扫描全能王 创建

的近似值, 要求误差不超过 10^{-2}.

15. 用 Romberg 方法计算积分

$$I=\int_0^1 \mathrm{e}^{-x^2}\,\mathrm{d}x$$

的近似值, 要求误差不超过 10^{-4}.

16. 确定下列公式中的节点及求积系数, 使其具有尽可能高的代数精确度:

(1) $\int_{-h}^{h} f(x)\mathrm{d}x \approx A_1 f\left(-\frac{h}{2}\right)+A_2 f(0)+A_3 f\left(\frac{h}{2}\right)$;

(2) $\int_0^2 f(x)\mathrm{d}x \approx f(x_1)+f(x_2)$;

(3) $\int_{-1}^{1} f(x)\mathrm{d}x \approx A_1 f(x_1)+A_2 f(0)+f(1)$.

17. 试证明区间 $[a,b]$ 上的两点 Gauss 型求积公式为

$$\int_a^b f(x)\mathrm{d}x \approx \frac{h}{2}\left[f\left(a+\left(\frac{1}{2}-\frac{\sqrt{3}}{6}\right)h\right)+f\left(a+\left(\frac{1}{2}+\frac{\sqrt{3}}{6}\right)h\right)\right]$$

并估计截断误差, 其中 $h=b-a$.

18. 试确定 x_1, x_2, A_1, A_2, 使求积公式

$$\int_0^1 \sqrt{x} f(x)\mathrm{d}x \approx A_1 f(x_1)+A_2 f(x_2)$$

为 Gauss 型求积公式.

19. 判别下列求积公式中, 哪个是 Gauss 型求积公式:

(1) $\int_{-1}^{1} f(x)\mathrm{d}x \approx \frac{2}{3}[f(-1)+f(0)+f(1)]$;

(2) $\int_{-1}^{1} f(x)\mathrm{d}x \approx \frac{1}{3}[f(-1)+4f(0)+f(1)]$;

(3) $\int_0^2 f(x)\mathrm{d}x \approx \frac{1}{9}[5f(1-\sqrt{0.6})+8f(1)+5f(1+\sqrt{0.6})]$.

20. 证明: 若求积公式

$$\int_a^b f(x)\mathrm{d}x \approx \sum_{k=1}^{n} A_k f(x_k)$$

的代数精确度不小于 $n-1$, 则必有

$$A_k=\int_a^b l_k(x)\mathrm{d}x \quad (k=1,2,\cdots,n)$$

扫描全能王 创建

其中 $l_k(x)$ 是以 $x_1, x_2, \cdots, x_n$ 为节点的 Lagrange 插值基函数.

21. 用三点或五点 Gauss-Legendre 求积公式计算积分

$$I = \int_0^1 \frac{2\sin(2x-1)}{3-2x}\,\mathrm{d}x$$

22. 利用 Gauss-Chebyshev 求积公式计算积分

$$I = \int_{-1}^1 \frac{p_n(x)}{\sqrt{1-x^2}}\,\mathrm{d}x$$

其中 $p_n(x)$ 为 n 次多项式.

23. 分别用 $n=4$ 的 Gauss-Laguerre 求积公式和 Gauss-Hermite 求积公式计算积分

$$I = \int_0^{+\infty} \mathrm{e}^{-x^2}\,\mathrm{d}x$$

的近似值, 并与准确值 $I = \dfrac{\sqrt{\pi}}{2}$ 作比较.

24. 用求积公式 (7.73) 计算积分

$$I = \int_0^{\frac{3}{2}\pi} \cos 15x\,\mathrm{d}x$$

取 $n=10, h=\dfrac{3\pi}{20}$.

25. 若等分区间 $[a,b]$ 为 N 等份, 记 $h=\dfrac{b-a}{N}, x_k = a+kh$ $(k=0,1,\cdots,N)$, 用阶梯函数

$$\varphi(x) = f\left(x_k + \frac{h}{2}\right), \quad x_k \leqslant x < x_{k+1} \quad (k=0,1,\cdots,N-1)$$

近似函数 $f(x)$, 试导出振荡积分 $\displaystyle\int_a^b f(x)\cos\omega x\,\mathrm{d}x$ 的求积公式.

26. 设 $n=m=4$, 试分别用复化梯形公式和复化 Simpson 计算二重积分, 并将计算结果与精确值作比较:

(1) $I = \displaystyle\int_{1.0}^{1.5}\mathrm{d}x\int_{1.2}^{1.6}\ln(x+2y)\mathrm{d}y$;

(2) $I = \displaystyle\int_0^{0.5}\mathrm{d}x\int_1^{1.5}\frac{1}{1+xy}\,\mathrm{d}y$.

27. 设 $n=m=4$, 试推导在 x 方向用复化梯形公式, 在 y 方向用复化 Simpson 公式的计算矩形区域上二重积分的近似求积公式, 并计算 26 题中的积分.

扫描全能王 创建

数值实验七

1. Cornu(考纽) 螺线的形状像钟表的发条, 也称回旋曲线, 它在直角坐标系中的参数方程为

$$\begin{cases} x(s)=\displaystyle\int_0^s \cos\frac{1}{2}at^2\,\mathrm{d}t \\ y(s)=\displaystyle\int_0^s \sin\frac{1}{2}at^2\,\mathrm{d}t \end{cases}$$

曲线关于原点对称. 取 $a=1$, 参数 s 的变化范围 $[-5,5]$, 容许误差限分别是 10^{-6} 和 10^{-10}. 选取适当的节点个数, 利用数值积分方法计算曲线上点的坐标, 并画出曲线的图形.

2. 利用数值积分公式求解积分方程:

(1) $y(x)=\displaystyle\int_0^1 (t-x)y(t)\mathrm{d}t+\mathrm{e}^{2x}+\left(\frac{\mathrm{e}^2-1}{2}\right)x-\frac{\mathrm{e}^2+1}{4}$, 精确解是 $y(x)=\mathrm{e}^{2x}$;

(2) $\displaystyle\int_{-\pi}^{\pi} g(t)\ln\left[(\cos x-\cos t)^2+(\sin x-\sin t)^2\right]\mathrm{d}t=-2\cos x$, 精确解是 $g(x)=2\cos x$.

分别用复化求积公式和 Gauss 型求积公式离散积分项, 求解上述积分方程, 得到解函数的近似表达式, 分析计算结果, 要求误差小于 10^{-6}. 绘出数值解和精确解的图形及误差图形.

3. 设 n,m 分别取 10,20 , 编写计算二重积分(7.76) 的程序, 计算积分:

(1) $I=\displaystyle\int_{-1}^{1}\mathrm{d}x\int_{-2}^{2}\mathrm{e}^{-\frac{x^2}{2}}\sin\left(x^2+y^2\right)\mathrm{d}y$;

(2) $I=\displaystyle\int_0^{\frac{\pi}{2}}\mathrm{d}x\int_0^{\frac{\pi}{3}}\frac{\tan\left(x^2+y^2\right)}{(1+x+y)}\mathrm{d}y$.

扫描全能王 创建

/第八章/

非线性方程及方程组的解法

科学应用与工程实际中出现的问题常常可以归结为求解非线性方程或非线性方程组的问题, 例如多项式求根问题. 一般地, 求方程

$$f(x)=0 \tag{8.1}$$

的根, 其中 $f(x)$ 为非线性函数, 即为非线性方程求解问题.

非线性方程的根可以是实数, 也可以是复数, 如果在 $x=a$ 处, $f(x)$ 满足

$$\begin{aligned} &f(a)=0, \quad f^{(l)}(a)=0 \ (l=1,\cdots,k-1) \\ &f^{(k)}(a)\neq 0 \ (k=1,2,\cdots) \end{aligned} \tag{8.2}$$

则称 $x=a$ 是方程 (8.1) 的 k 重根或 $f(x)$ 的 k 重零点, 特别地, 当 $k=1$ 时, $x=a$ 称为单根或单零点. 方程 (8.1) 有 k 重根 $x=a$ 等价于函数 $f(x)$ 可以分解成以下形式

$$f(x)=(x-a)^k\varphi(x)$$

其中 $\varphi(a)\neq 0$.

非线性方程 (组) 的根能用解析式表示的极少, 绝大多数需借助于数值方法求近似解. 本章主要介绍几种求解非线性方程的常用数值方法, 最后一节简略地介绍求解非线性方程组的方法.

8.1 二 分 法

设 $f(x)$ 在区间 $[a,b]$ 上连续, 且有

$$f(a)f(b)<0$$

则由连续函数介值定理, $f(x)$ 在 (a,b) 内必有零点, 称 (a,b) 为方程 (8.1) 的有根区间. 不妨设 $f(a)<0, f(b)>0$. 取 $x_0=\dfrac{a+b}{2}$, 若 $f(x_0)=0$, 则 $x=x_0$ 就是方程 (8.1) 的解. 否则, 若 $f(x_0)<0$, 取 $a_1=x_0, b_1=b$; 若 $f(x_0)>0$, 取 $a_1=a, b_1=x_0$,

$$[a_1,b_1]\subset[a,b], \quad b_1-a_1=\frac{b-a}{2}$$

且 $f(x)$ 在 $[a_1,b_1]$ 上连续, 满足 $f(a_1)f(b_1)<0$. 重复上述过程又可得到区间 $[a_2, b_2]$, 满足 $[a_2,b_2]\subset[a_1,b_1], b_2-a_2=\dfrac{b_1-a_1}{2}$, 且 $f(a_2)f(b_2)<0$. 如此继续下去, 得到一个区间

扫描全能王 创建

序列

$$[a,b]\supset[a_1,b_1]\supset[a_2,b_2]\supset\cdots\supset[a_n,b_n]\supset\cdots$$

满足

$$f(a_n)f(b_n)<0,\quad b_n-a_n=\frac{b-a}{2^n}$$

因而 $[a_n,b_n](n=1,2,\cdots)$ 均为方程 (8.1) 的有根区间. 由区间套定理, 存在 $x^*\in[a,b]$, 使得

$$\lim_{n\to\infty}a_n=\lim_{n\to\infty}b_n=x^*$$

且 $x=x^*$ 是方程 (8.1) 的根. 若取 $x_n=\dfrac{a_n+b_n}{2}$ 作为根 x^* 的近似值, 则误差为

$$|x^*-x_n|\leqslant\frac{b_n-a_n}{2}=\frac{b-a}{2^{n+1}}\tag{8.3}$$

按上述方法求非线性方程 (8.1) 的近似解称为二分法 (bisection). 式 (8.3) 表明, 只要 $f(x)$ 连续, 对分区间法总是收敛的.

算法 8.1 二分法

1. 输入 $f(x),a,b$, 误差限 ε, 最大容许迭代次数 N.
2. 置 $n=0$.
3. $x_n=\dfrac{a+b}{2}$.
4. 若 $f(x_n)=0$ 或 $\dfrac{b-a}{2}<\varepsilon$, 则输出 $x^*=x_n$, 停机; 否则转 5 .
5. 若 $n<N$, 则转 6 ; 否则输出失败信息, 停机.
6. 若 $f(a)f(x_n)<0$, 则 $x_n\Rightarrow b$; 否则, $x_n\Rightarrow a$.
7. 置 $n+1\Rightarrow n$, 转 3 .

式 (8.3) 不仅可以估计对分区间法的误差, 而且可以由给定的误差限估计出对分区间的次数. 设要求近似解的误差不超过 ε, 则由式 (8.3)

$$|x^*-x_n|\leqslant\frac{b-a}{2^{n+1}}\leqslant\varepsilon$$

可得

$$2^{n+1}\geqslant\frac{b-a}{\varepsilon}$$

从而有

$$n\geqslant\frac{\ln(b-a)-\ln\varepsilon}{\ln 2}-1\tag{8.4}$$

例 8.1 试用对分区间法求方程

扫描全能王 创建

$$f(x)=x^3+10x-20=0 \tag{8.5}$$

的唯一实根, 要求误差不超过 $\frac{1}{2}\times 10^{-4}$.

解　容易验证

$$f(1)=1+10-20<0,\quad f(2)=8+20-20>0$$

且 $f'(x)=3x^2+10>0, \forall x\in(-\infty,+\infty)$, 故 $f(x)$ 在 $(-\infty,+\infty)$ 上单调增加, 于是方程 (8.5) 有唯一实根, 且 $[1,2]$ 为有根区间. 为达到要求精度, 按式 (8.4), 对分区间次数为

$$n\geqslant \frac{\ln(2-1)-\ln\left(\frac{1}{2}\times 10^{-4}\right)}{\ln 2}-1=13.28$$

故 $n=14$, 计算结果见表 8.1.

表 8.1　二分法计算结果

n	a_n	b_n	x_n	$f(x_n)$
0	1	2	1.5	−1.625
1	1.5	2	1.75	2.859 375
2	1.5	1.75	1.625	0.541 015 6
3	1.5	1.625	1.562 5	−0.560 302 7
4	1.562 5	1.625	1.593 75	−0.014 312 7
5	1.593 75	1.625	1.609 375	0.262 172 6
6	1.593 75	1.609 375	1.601 562 5	0.123 636 7
7	1.593 75	1.601 562 5	1.597 656 3	0.054 589 4
8	1.593 75	1.597 656 3	1.595 703 2	0.020 120 8
9	1.593 75	1.595 703 2	1.594 726 6	0.002 899 6
10	1.593 75	1.594 726 6	1.594 238 3	−0.005 707 7
11	1.594 238 3	1.594 726 6	1.594 482 5	−0.001 403 7
12	1.594 482 5	1.594 726 6	1.594 604 6	0.000 748 64
13	1.594 482 5	1.594 604 6	1.594 543 6	−0.000 326 42
14	1.594 543 6	1.594 604 6	1.594 574 1	

所以 $x_{14}=1.594\,574\,1$ 即为符合精度要求的解.

注 8.1　二分法的显著优点是计算简便, 容易估计误差, 但它收敛较慢, 且只能用于求奇数重实根.

8.2　简单迭代法

定义 8.1　如果已知方程 $f(x)=0$ 的根 x^* 的一个近似值 x_0, 通过构造一个递推公式

$$x_{n+1}=g(x_n)\quad (n=0,1,2,\cdots) \tag{8.6}$$

扫描全能王 创建

由此产生序列 $\{x_n\}$, 以 x_n 逐次逼近 x^*. 上述求非线性方程近似根的方法称为迭代法 (单步), 式 (8.6) 称为迭代格式, $g(x)$ 称为迭代函数, $\{x_n\}$ 称为迭代序列.

若 $\{x_n\}$ 收敛于 x^*, 则称迭代法收敛, 否则称迭代法是发散的. 当 $g(x)$ 连续时, 若 $\lim\limits_{n\to\infty} x_n = x^*$, 则有 $x^* = g(x^*)$, 称 x^* 为函数 $g(x)$ 的不动点, 上述方法又称为**不动点迭代法**. 迭代格式的构造不同即得出不同的迭代法, 本节介绍一种最直接、简单的迭代法.

8.2.1 简单迭代法的一般形式及其几何意义

如果将方程 (8.1) 改写成以下等价形式

$$x = \varphi(x) \tag{8.7}$$

则方程 (8.7) 和 (8.1) 同解. 取定初始值 x_0, 由迭代公式

$$x_{n+1} = \varphi(x_n) \quad (n = 0, 1, 2, \cdots) \tag{8.8}$$

产生迭代序列 $\{x_n\}$. 显然, 若 $\{x_n\}$ 收敛于 x^*, $\varphi(x)$ 在 x^* 处连续, 就有

$$x^* = \lim_{n\to\infty} x_{n+1} = \lim_{n\to\infty} \varphi(x_n) = \varphi(x^*)$$

即 x^* 是方程 (8.7) 的解, 从而也是 (8.1) 的解. 故当 n 充分大时, 可取 x_n 作为方程 (8.1) 的根的近似值. 按上述方法构造迭代格式 (8.8) 来求解方程 (8.1) 称为**简单迭代法**. 简单迭代法是一种不动点迭代.

例 8.2 用简单迭代法求方程

$$f(x) = x^3 + 10x - 20 = 0$$

的根, 要求精确到六位小数.

解 由例 8.1, 所给方程在 $[1,2]$ 内有根, 取 $x_0 = 1.5$.

(1) 把原方程化成等价形式

$$x = x^3 + 11x - 20$$

按迭代格式

$$x_{n+1} = x_n^3 + 11x_n - 20 \tag{8.9}$$

$$x_1 = -0.125, \quad x_2 = -21.376\,953, \quad x_3 = -10\,023.861, \cdots$$

显然, 按迭代格式 (8.9) 计算, 所得迭代序列发散.

(2) 若化原方程为等价方程

$$x = \frac{20}{x^2 + 10}$$

按迭代格式

$$x_{n+1} = \frac{20}{x_n^2 + 10}$$

计算结果见表 8.2.

扫描全能王 创建

表 8.2　计 算 结 果

n	x_n	n	x_n
0	1.5	8	1.594 493 4
1	1.632 653 1	9	1.594 590 0
2	1.579 085 8	10	1.594 550 8
3	1.600 830 9	11	1.594 566 7
4	1.592 019 6	12	1.594 560 3
5	1.595 592 8	13	1.594 562 9
6	1.594 144 2	14	1.594 561 8
7	1.594 731 5	15	1.594 562 2

因为

$$|x_{15}-x_{14}| = 1.594\,562\,2 - 1.594\,561\,8 = 0.000\,000\,4 < \frac{1}{2}\times 10^{-6}$$

所以 $x_{15} = 1.594\,562$ 即为方程满足精度要求的近似根.

(3) 若化原方程为等价方程

$$x = 2 - \frac{x^3}{10}$$

按迭代格式

$$x_{n+1} = 2 - \frac{x_n^3}{10} \tag{8.10}$$

计算结果见表 8.3.

表 8.3　计 算 结 果

n	x_n	n	x_n
0	1.5	8	1.584 093 0
1	1.662 5	9	1.602 495 5
2	1.540 500 6	10	1.588 480 5
3	1.634 417 3	11	1.599 183 5
4	1.563 394 8	12	1.591 026 8
5	1.617 874 5	13	1.597 252 9
6	1.576 518 5	14	1.592 506 2
7	1.608 170 4	15	1.596 128 3

$$|x_{15}-x_{14}| = 0.362\,21\times 10^{-2}$$

容易看出, 按迭代格式 (8.10) 计算, 所得迭代序列也收敛, 但收敛较慢, 迭代 15 次尚未达到精度要求.

例 8.2 的计算结果表明, 将一方程化为等价方程的方法很多, 由此可以构造许多不同的迭代函数, 得到多种迭代格式. 而它们所产生的迭代序列则可能收敛, 也可能发散, 可能收敛很快, 也可能收敛很慢. 迭代法的收敛性取决于迭代函数在方程的根的邻近的性态.

扫描全能王 创建

从几何上看, 简单迭代法是将求曲线 $y=f(x)$ 的零点问题化为求曲线 $y=\varphi(x)$ 与直线 $y=x$ 的交点, 迭代过程如图 8.1 所示, 从初始点 x_0 出发, 沿直线 $x=x_0$ 走到曲线 $y=\varphi(x)$, 得点 $(x_0,\varphi(x_0))$, 再沿直线 $y=\varphi(x_0)$ 走到直线 $y=x$, 交点为 (x_1,x_1), 如此继续下去, 越来越接近点 (x^*,x^*).

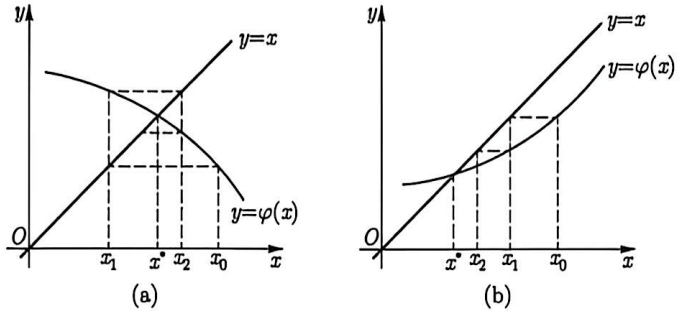


图 8.1 迭代收敛示意图

当然, 迭代过程也可能出现图 8.2 所示的情况, 此时点 (x_n,x_n) 越来越远离交点 (x^*,x^*), 迭代序列发散.

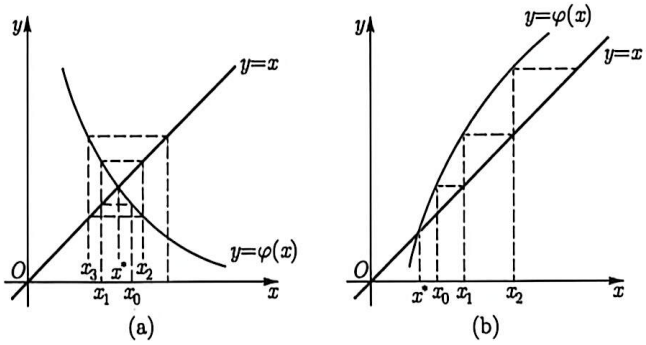


图 8.2 迭代发散示意图

8.2.2 迭代法的收敛条件

定理 8.1 压缩映射原理 设函数 $\varphi(x)$ 在区间 $[a,b]$ 上满足条件:

(1) 对任意 $x\in[a,b]$, 都有 $a\leqslant\varphi(x)\leqslant b$;

(2) 存在常数 $0<L<1$, 使得对一切 $x,y\in[a,b]$, 都有

$$|\varphi(x)-\varphi(y)|\leqslant L|x-y|$$

则方程 $x=\varphi(x)$ 在 $[a,b]$ 内有唯一的根 x^*, 且对任何初值 $x_0\in[a,b]$, 迭代序列

$$x_{n+1}=\varphi(x_n)\quad(n=0,1,\cdots)$$

均收敛于 x^*, 并有

$$|x^*-x_n|\leqslant\frac{L^n}{1-L}|x_1-x_0| \tag{8.11}$$

扫描全能王 创建

$$|x^* - x_n| \leqslant \frac{L}{1-L}|x_n - x_{n-1}| \tag{8.12}$$

证明　由条件 (2) 易得 $\varphi(x)$ 在 $[a,b]$ 上连续. 令 $\psi(x) = x - \varphi(x)$, 则 $\psi(x)$ 在 $[a,b]$ 上连续, 且

$$\psi(a) = a - \varphi(a) \leqslant 0, \quad \psi(b) = b - \varphi(b) \geqslant 0$$

由连续函数介值定理, 存在 $\xi \in [a,b]$, 使得 $\psi(\xi) = 0$, 即 $\xi = \varphi(\xi)$, 所以方程 $x = \varphi(x)$ 在 $[a,b]$ 内有根.

假设方程 $x = \varphi(x)$ 在 $[a,b]$ 内有两个根 $x_1^* \neq x_2^*$, 由条件 (2), 有

$$|x_1^* - x_2^*| = |\varphi(x_1^*) - \varphi(x_2^*)| \leqslant L|x_1^* - x_2^*| < |x_1^* - x_2^*|$$

导出矛盾, 唯一性得证.

对任意 $x_0 \in [a,b]$, 由迭代公式有

$$|x_n - x^*| = |\varphi(x_{n-1}) - \varphi(x^*)| \leqslant L|x_{n-1} - x^*|$$

依此类推, 即得

$$|x_n - x^*| \leqslant L^n |x_0 - x^*|$$

因为 $0 < L < 1$, 所以

$$\lim_{n\to\infty} x_n = x^*$$

即对任意初值 $x_0 \in [a,b]$, 迭代序列 $\{x_n\}$ 均收敛到方程的根 x^*.

类似地, 对任意正整数 n, 有

$$\begin{aligned} |x^* - x_n| &= |\varphi(x^*) - \varphi(x_{n-1})| \\ &\leqslant L|x^* - x_{n-1}| \leqslant \cdots \\ &\leqslant L^n |x^* - x_0| \end{aligned}$$

显然有 $|x^* - x_1| \leqslant L|x^* - x_0|$, $|x^* - x_n| \leqslant L|x^* - x_{n-1}|$, 再利用三角不等式

$$|x^* - x_0| \leqslant |x^* - x_1| + |x_1 - x_0|$$

$$|x^* - x_{n-1}| \leqslant |x^* - x_n| + |x_n - x_{n-1}|$$

可得

$$|x^* - x_0| \leqslant \frac{1}{1-L}|x_1 - x_0|$$

$$|x^* - x_{n-1}| \leqslant \frac{1}{1-L}|x_n - x_{n-1}|$$

所以

$$|x^* - x_n| \leqslant \frac{L^n}{1-L}|x_1 - x_0| \tag{8.13}$$

$$|x^* - x_n| \leqslant \frac{L}{1-L}|x_n - x_{n-1}|. \tag{8.14}$$

扫描全能王 创建

注 8.2 压缩映射条件 (1)(2) 保证了不动点问题的存在唯一性及迭代序列的收敛性; 式 (8.11) 说明迭代序列收敛速度由压缩率 L 决定; 式 (8.12) 说明可以由迭代结果分析计算误差, 这也是简单迭代法常以 $|x_n - x_{n-1}| < \varepsilon$ 作为停步准则的依据.

定理 8.1 给出了判别不动点迭代法收敛的充分条件. 由于实际计算时, 常数 L 十分难求, 而我们所讨论的函数通常都是可导函数, 故实用的收敛条件是用导数的界得到的.

定理 8.2 设函数 $\varphi(x)$ 在区间 $[a,b]$ 上满足条件:

(1) 对任意 $x \in [a,b]$, 有 $\varphi(x) \in [a,b]$;

(2) 存在常数 $0 < L < 1$, 使得对任意 $x \in [a,b]$, 都有 $|\varphi'(x)| \leqslant L$,

则方程 (8.7) 在 $[a,b]$ 上有唯一的根 x^*, 且对任意初值 $x_0 \in [a,b]$, 迭代序列

$$x_{n+1} = \varphi(x_n) \quad (n = 0, 1, 2 \cdots)$$

收敛于 x^*, 并有

$$|x^* - x_n| \leqslant \frac{L^n}{1-L} |x_1 - x_0|$$

$$|x^* - x_n| \leqslant \frac{L}{1-L} |x_n - x_{n-1}|$$

证明 设 x, y 为 $[a,b]$ 上任意两点, 由微分中值定理, 在 x, y 之间至少存在一点 ξ, 使得

$$\varphi(x) - \varphi(y) = \varphi'(\xi)(x - y)$$

于是

$$|\varphi(x) - \varphi(y)| = |\varphi'(\xi)(x - y)| \leqslant L|x - y|$$

用定理 8.2判别不动点迭代法的收敛性比定理 8.1方便. 如例 8.2 中, 取 $\varphi_1(x) = \dfrac{20}{x^2 + 10}$, 则有

$$\varphi_1'(x) = \frac{-40x}{(x^2 + 10)^2}$$

对 $\forall x \in [1,2]$, 有

$$|\varphi_1'(x)| = \frac{40x}{(x^2 + 10)^2} \leqslant \frac{80}{11^2} < 1$$

所以迭代收敛.

对 $\varphi_2(x) = 2 - \dfrac{x^3}{10}$, 因为

$$\varphi_2'(x) = -\frac{3}{10}x^2$$

其绝对值在区间 $[1,2]$ 上的最大值超过 1 , 故 $\varphi_2(x)$ 在 $[1,2]$ 上不满足定理 8.2的条件, 不能用定理 8.2 判定收敛性. 容易看出, 在小区间 $[1.5, 1.7]$ 上, 有 $1.5 < \varphi_2(x) < 1.7$, 且

$$|\varphi_2'(x)| = \frac{3}{10}x_2 \leqslant 0.867$$

因此 $\varphi_2(x)$ 在小区间 $[1.5, 1.7]$ 上满足定理 8.2 的条件, 故取 $x_0 = 1.5$, 按迭代函数

扫描全能王 创建

$\varphi_2(x)$ 所导出的迭代序列是收敛的.

误差估计式 (8.11) 表明, 常数 L 越小, 不动点迭代法收敛越快, 因而用简单迭代法求方程 (8.1) 的根时, 构造迭代函数 $\varphi(x)$ 的原则是使 $|\varphi'(x)|$ 在有根区间 $[a,b]$ 上有尽可能小的上界. 如例 8.2 中迭代函数 $\varphi_1(x)$ 所产生的迭代序列收敛比较快.

按式 (8.12), 当 $0 < L \leqslant 0.5$ 时, 有

$$|x^* - x_n| \leqslant \frac{L}{1-L}|x_n - x_{n-1}| \leqslant |x_n - x_{n-1}|$$

故简单迭代法常以 $|x_n - x_{n-1}| < \varepsilon$ 作为停步准则.

一般说来, 构造迭代函数使其在预先指定的较大区间上满足定理 8.2的条件是比较困难的, 但若取 x_0 充分接近根 x^*, 则收敛性的讨论可在根的邻近进行.

定理 8.3　如果函数 $\varphi(x)$ 在 x^* 的一邻域 $O(x^*,\delta^*)$ 内连续可微, x^* 为方程 (8.7) 的根, 且 $|\varphi'(x^*)| < 1$, 则存在正数 $\delta, \delta \leqslant \delta^*$, 使得对任意 $x_0 \in [x^* - \delta,\ x^* + \delta]$, 迭代序列

$$x_{n+1} = \varphi(x_n) \quad (n = 0, 1, 2, \cdots)$$

收敛于 x^*.

证明　因为 $\varphi'(x)$ 在 $O(x^*,\delta^*)$ 内连续, 且 $|\varphi'(x^*)| < 1$, 故存在正数 $L < 1, \delta < \delta^*$, 使得对 $\forall x \in [x^* - \delta, x^* + \delta]$, 有

$$|\varphi'(x)| \leqslant L < 1$$

另一方面, 由 $\varphi(x^*) = x^*$, 又有

$$|\varphi(x) - x^*| = |\varphi(x) - \varphi(x^*)| \leqslant L|x - x^*| < \delta$$

即 $\varphi(x) \in [x^* - \delta, x^* + \delta]$. 根据定理 8.2, 迭代序列 $x_{n+1} = \varphi(x_n)$ 收敛于 x^*.

注 8.3　定理 8.3 给出了初值在根邻近时, 不动点迭代法收敛的充分条件, 称为局部收敛性定理. 这一定理表明, 只要构造迭代函数, 使其在根的邻近满足导数的绝对值有小于 1 的上界, 即可保证简单迭代法收敛, 如果对解的信息了解很少, 局部收敛性定理并不实用. 事实上, 在用简单迭代法求解方程 (8.1) 时, 常常先用对分区间法求得较好的初值, 然后再进行迭代.

注 8.4　如果 $\varphi(x)$ 在 x^* 的邻域 $O(x^*,\delta^*)$ 内 1 阶连续可微, 且

$$x^* = \varphi(x^*), \quad |\varphi'(x^*)| > 1$$

迭代序列 $x_{n+1} = \varphi(x_n)(n = 0, 1, 2, \cdots)$ 发散.

事实上, $\exists\delta$, 使得 $\forall x \in [x^* - \delta, x^* + \delta], |\varphi'(x)| > L > 1$

$$|x_{n+1} - x^*| = |\varphi(x_n) - \varphi(x^*)| = |\varphi'(\xi)|\,|x_n - x^*| > |x_n - x^*|$$

8.2.3　Steffensen 方法——简单迭代法的加速

(一) 收敛速度

定义 8.2　设序列 $\{x_n\}$ 收敛于 x^*. 如果存在正数 r 和 a, 使得

扫描全能王 创建

$$\lim_{n\to\infty}\frac{|x_{n+1}-x^*|}{|x_n-x^*|^r}=a$$

成立, 则称序列 $\{x_n\}$ 是 r 阶收敛的, 或称 $\{x_n\}$ 的收敛阶为 r. (如果 $r=1$, 要求 $a<1$)

显然, 收敛阶 r 的大小刻画了序列 $\{x_n\}$ 的收敛速度, r 越大, 收敛越快. 特别地, 若 $r=1$, 称序列是线性收敛的; 若 $r>1$, 称为超线性收敛; $r=2$ 为平方收敛.

关于迭代法的收敛速度有以下结果.

定理 8.4　设迭代函数 $\varphi(x)$ 在 x^* 邻近有 r 阶连续导数 $(r\geqslant 2)$, 且 $x^*=\varphi(x^*)$, $\varphi^{(k)}(x^*)=0$ $(k=1,2,\cdots,r-1)$, $\varphi^{(r)}(x^*)\neq 0$, 则迭代公式 (8.8) 所产生的迭代序列 $\{x_n\}$ 是 r 阶收敛的. 若 $x^*=\varphi(x^*)$, $0<|\varphi'(x^*)|<1$, 则迭代序列 $\{x_n\}$ 是线性收敛的.

证明　显然, $\varphi(x)$ 满足定理(8.3) 的条件, 故存在 $\delta>0$, 使得对任意 $x_0\in[x^*-\delta,x^*+\delta]$, 迭代序列 $x_{n+1}=\varphi(x_n)$ 收敛于 x^*, 且 $x_n\in[x^*-\delta,x^*+\delta]$ $(n=0,1,2,\cdots)$.

由 $\varphi(x)$ 在 x^* 处的 Taylor 公式得

$$\begin{aligned}x_{n+1}=\varphi(x_n)&=\varphi(x^*)+\varphi'(x^*)(x_n-x^*)+\cdots+\frac{\varphi^{(r-1)}(x^*)}{(r-1)!}(x_n-x^*)^{r-1}+\frac{\varphi^{(r)}(\xi)}{r!}(x_n-x^*)^r\\&=x^*+\frac{\varphi^{(r)}(\xi)}{r!}(x_n-x^*)^r,\quad \xi\text{ 在 }x^*\text{ 与 }x_n\text{ 之间}\end{aligned}$$

于是

$$\frac{x_{n+1}-x^*}{(x_n-x^*)^r}=\frac{\varphi^{(r)}(\xi)}{r!}$$

令 $n\to\infty$, 因为 $x_n\to x^*$, 故 $\xi\to x^*$, 从而有

$$\lim_{n\to\infty}\frac{|x_{n+1}-x^*|}{|x_n-x^*|^r}=\frac{|\varphi^{(r)}(x^*)|}{r!}\neq 0$$

所以, 迭代序列 $\{x_n\}$ 是 r 阶收敛的.

类似地, 当 $0<|\varphi'(x^*)|<1$ 时, 由定理 8.3, 迭代序列 $\{x_n\}$ 收敛于 x^*, 且由微分中值定理

$$\lim_{n\to\infty}\frac{|x_{n+1}-x^*|}{|x_n-x^*|}=\lim_{n\to\infty}\frac{|\varphi(x_n)-\varphi(x^*)|}{|x_n-x^*|}=|\varphi'(x^*)|\neq 0$$

故 $\{x_n\}$ 线性收敛于 x^*.

按定理 8.4, 若 $\varphi'(x^*)\neq 0$, 简单迭代法是线性收敛的. 为加快收敛速度, 可用 Aitken 加速, 由此导出 Steffensen 方法.

(二) Steffensen 方法

第四章中已提到, 如果序列 $\{x_n\}$ 线性收敛于 x^*, 则按下式产生的新序列

$$\bar{x}_n=x_n-\frac{(x_{n+1}-x_n)^2}{x_{n+2}-2x_{n+1}+x_n}$$

(称为序列 $\{x_n\}$ 的 Aitken 序列) 比序列 $\{x_n\}$ 更快地收敛于 x^*. 将 Aitken 加速法与简单迭代格式 (8.8) 相结合, 得到 Steffensen 方法, 其迭代公式为

扫描全能王 创建

$$\begin{cases} y_n = \varphi(x_n) \\ z_n = \varphi(y_n) \\ x_{n+1} = x_n - \dfrac{(y_n - x_n)^2}{z_n - 2y_n + x_n} \end{cases} \quad (n = 0, 1, 2, \cdots) \tag{8.15}$$

Steffensen 方程也是一种不动点迭代, 其迭代公式为

$$x_{n+1} = x_n - \frac{[\varphi(x_n) - x_n]^2}{\varphi(\varphi(x_n)) - 2\varphi(x_n) + x_n}$$

可以证明, 当 $\varphi'(x^*) \neq 1$ 时, Steffensen 方法是至少二阶局部收敛的.

例 8.3　用 Steffensen 方法求方程 $f(x) = x^3 + 10x - 20 = 0$ 的根, 取 $x_0 = 1.5$, $\varepsilon = 10^{-6}$.

解　取迭代函数为 $\varphi(x) = \dfrac{20}{x^2 + 10}$, 按式 (8.15) 计算, 结果见表 8.4.

表 8.4　计 算 结 果

n	x_n	y_n	z_n
0	1.5	1.632 653 1	1.579 085 8
1	1.594 494 7	1.594 589 5	1.594 551 0
2	1.594 562 1	1.594 562 1	1.594 562 1

迭代 2 次, 就得到满足精度要求的解

$$x^* \approx x_2 = 1.594\ 562\ 1$$

可见 Steffensen 方法收敛很快.

8.3　Newton 法与弦截法

8.3.1　Newton 法

将非线性方程线性化, 以线性方程的解逐步逼近非线性方程的解, 这就是 Newton 法的基本思想.

设 $f(x)$ 在其零点 x^* 邻近一阶连续可微, 且 $f'(x) \neq 0$, 当 x_0 充分接近 x^* 时, 由 Taylor 公式有

$$f(x^*) \approx f(x_0) + f'(x_0)(x^* - x_0)$$

以方程

$$f(x_0) + f'(x_0)(x - x_0) = 0$$

近似方程 (8.1), 其解

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

扫描全能王 创建

可作为方程 (8.1) 的近似解. 重复以上过程, 得迭代公式

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (n = 0, 1, 2, \cdots) \tag{8.16}$$

按式 (8.16) 求方程 (8.1) 的近似解称为 Newton 法. Newton 法也是一种不动点迭代, 其迭代函数为

$$\varphi(x) = x - \frac{f(x)}{f'(x)}$$

如图 8.3 所示, 从几何上看, $y = f(x_0) + f'(x_0)(x - x_0)$ 为曲线 $y = f(x)$ 过点 $(x_0, f(x_0))$ 的切线, x_1 为切线与 x 轴的交点, x_2 则是曲线上点 $(x_1, f(x_1))$ 处的切线与 x 轴的交点. 如此继续下去, x_{n+1} 为曲线上点 $(x_n, f(x_n))$ 处的切线与 x 轴的交点. 因此 Newton 法是以曲线的切线与 x 轴的交点作为曲线与 x 轴的交点的近似, 故 Newton 法又称为切线法.

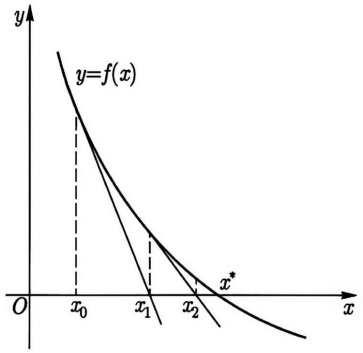


图 8.3 Newton 法 (切线法)

算法 8.2 Newton 法

1. 输入 $f(x)$, 初始值 x_0, 误差限 ε, 最大允许迭代次数 N.
2. 置 $n = 1$.
3. 计算

$$x = x_0 - \frac{f(x_0)}{f'(x_0)}$$

4. 若 $|x - x_0| < \varepsilon$ 或 $\dfrac{|x - x_0|}{|x|} < \varepsilon$, 则输出 $x^* = x$, 停机; 否则转 5 .
5. 若 $n < N$, 则置 $n + 1 \Rightarrow n, x \Rightarrow x_0$, 转 3 ; 否则输出失败信息, 停机.

说明: 在第 4 步中, $\dfrac{|x - x_0|}{|x|} < \varepsilon$ 为相对误差检验, 即相对误差达到精度也可作为停机准则. 有时, Newton 法还以 $|f(x)| < \varepsilon$ 作为停机准则.

例 8.4 用 Newton 法求方程 $f(x) = x^3 + 10x - 20 = 0$ 的根, 取 $x_0 = 1.5$.

解 因为 $f'(x) = 3x^2 + 10$, 故 Newton 迭代公式为

扫描全能王 创建

$$x_{n+1} = x_n - \frac{x_n^3 + 10x_n - 20}{3x_n^2 + 10}$$

代入初值得

$$x_1 = 1.597\,014\,9, \quad x_2 = 1.594\,563\,7$$
$$x_3 = 1.594\,562\,1, \quad x_4 = 1.594\,562\,1$$

迭代 3 次所得近似解就准确到 8 位有效数字, 可见 Newton 法收敛很快.

定理 8.5　设函数 $f(x)$ 在其零点 x^* 邻近二阶连续可微, 且 $f'(x^*) \neq 0$, 则存在 $\delta > 0$, 使得对任意 $x_0 \in [x^* - \delta, x^* + \delta]$, Newton 法所产生的序列 $\{x_n\}$ 至少二阶收敛于 x^*.

证明　按式 (8.16), Newton 法的迭代函数为

$$\varphi(x) = x - \frac{f(x)}{f'(x)}$$

于是有

$$\varphi'(x) = 1 - \frac{[f'(x)]^2 - f(x)f''(x)}{[f'(x)]^2} = \frac{f(x)f''(x)}{[f'(x)]^2}$$

$$\varphi'(x^*) = \frac{f(x^*)f''(x^*)}{[f'(x^*)]^2} = 0$$

根据定理 8.4, Newton 法所产生的序列 $\{x_n\}$ 至少二阶收敛于 x^*.

定理 8.5 表明, 当初始值 x_0 充分接近方程 (8.1) 的单根 x^* 时, Newton 法的收敛速度较快, 但计算实践表明, 当初始值不够好时, Newton 法可能发散. 一般可由问题的实际背景来预测或由对分区间法求得较好的初始值. 还要指出的是, 当 x^* 为方程 (8.1) 的 r 重根时, Newton 法是线性收敛的, 且

$$\lim_{n\to\infty} \frac{|x_{n+1} - x^*|}{|x_n - x^*|} = \frac{r-1}{r}$$

显然, r 越大收敛越慢. 为了改善其收敛性, 通常用以下两种方法:

(1) 如果重根 x^* 的重数 r 已知 ($r \geqslant 2$), 则用迭代公式

$$x_{n+1} = x_n - r\frac{f(x_n)}{f'(x_n)} \tag{8.17}$$

(2) 如果 x^* 的重数未知, 则令 $\mu(x) = \dfrac{f(x)}{f'(x)}$, 对 $\mu(x)$ 用 Newton 法, 得迭代公式

$$x_{n+1} = x_n - \frac{f(x_n)f'(x_n)}{[f'(x_n)]^2 - f(x_n)f''(x_n)} \tag{8.18}$$

容易证明, 这两种方法都是至少二阶收敛的, 作为习题, 请读者自行证明.

例 8.5　取 $x_0 = 0.5$, 求方程 $\sin x - x = 0$ 的重根, 要求误差小于 10^{-8}.

解　设 $f(x) = \sin x - x$, 显然 $f(0) = 0$, 且 $f'(0) = f''(0) = 0$, $f^{(3)}(0) = -1 \neq 0$, 故是方程的 3 重根. 分别用 Newton 法、公式 (8.17) 和公式 (8.18) 计算, 结果列于表 8.5:

扫描全能王 创建

表 8.5 计 算 结 果

n	Newton 法 x_n	公式 (8.17)x_n	公式 (8.18) x_n
0	0.5	0.5	0.5
1	0.331 931 939 489 11	−0.004 204 181 532 67	0.008 274 056 887 75
2	0.220 880 000 703 62	$2.476\,920\,685\,917\,650 \times 10^{-9}$	$3.776\,252\,810\,970\,548 \times 10^{-8}$
3	0.147 133 388 298 26		$1.502\,191\,775\,200\,419 \times 10^{-8}$
4	0.098 053 507 283 12		$1.502\,191\,775\,200\,419 \times 10^{-8}$
⋮	⋮		
43	0.000 000 019 811 53		
44	0.000 000 019 811 53		

由计算结果可见, Newton 法对于重根的求解确实收敛缓慢, 而公式 (8.17) 和公式 (8.18) 的收敛速度要快得多.

Newton 法对于单根的求解具有收敛快、稳定性好、精度高等优点, 它是求解非线性方程的有效方法之一. 但它每次迭代需要计算函数值与导数值, 计算量较大. 而且当导数计算有困难时, Newton 法将无法进行.

8.3.2 弦截法

为避免计算导数, 可用差商近似导数. 在 Newton 迭代公式 (8.16) 中, 用差商 $\dfrac{f(x_n)-f(x_{n-1})}{x_n-x_{n-1}}$ 代替导数 $f'(x_n)$, 得到迭代公式

$$x_{n+1}=x_n-\frac{f(x_n)}{f(x_n)-f(x_{n-1})}(x_n-x_{n-1}) \quad (n=1,2,\cdots) \tag{8.19}$$

按式 (8.19) 计算方程 (8.1) 的近似解称为弦截法.

如图 8.4 所示, 过曲线上两点 $(x_{n-1},f(x_{n-1}))$, $(x_n,f(x_n))$ 的直线为

$$y-f(x_n)=\frac{f(x_n)-f(x_{n-1})}{x_n-x_{n-1}}(x-x_n)$$

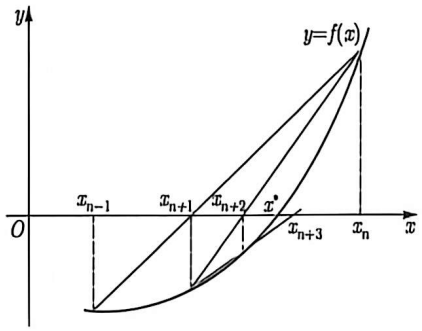


图 8.4 弦截法 (割线法)

扫描全能王 创建

它与 x 轴的交点为

$$x = x_n - \frac{f(x_n)}{f(x_n) - f(x_{n-1})}(x_n - x_{n-1}) = x_{n+1}$$

于是, 从几何上看, 弦截法是以曲线上两点的割线与 x 轴的交点作为曲线与 x 轴的交点的近似, 故弦截法又称为割线法. 另一方面, 割线的函数表达式恰为函数 $f(x)$ 的以 x_{n-1}, x_n 为节点的线性插值多项式, 按式 (8.19) 求方程 (8.1) 的近似解等价于用 $f(x)$ 的线性插值多项式的零点近似函数 $f(x)$ 的零点, 因此弦截法还称为线性插值法.

弦截法是两步法, 用弦截法求非线性方程的解, 必须给出两个初始值 x_0, x_1, 通常取根所在区间的端点即可. 一般地, 弦截法仍以 $|x_n - x_{n-1}| < \varepsilon$ 为停步准则, 也可以 $|f(x_n)| < \varepsilon$ 为停步准则.

算法 8.3　弦截法

1. 输入 $f(x)$, 初始值 x_0, x_1, 误差限 ε, 最大允许迭代次数 N.
2. 置 $f_0 = f(x_0), f_1 = f(x_1), n = 1$.
3. 计算
$$x = x_1 - \frac{f_1}{f_1 - f_0}(x_1 - x_0)$$
4. 若 $|x - x_1| < \varepsilon$, 则输出 $x^* = x$, 停机; 否则转 5 .
5. 若 $n < N$, 则置 $n+1 \Rightarrow n, x_1 \Rightarrow x_0, x \Rightarrow x_1, f_1 \Rightarrow f_0, f(x) \Rightarrow f_1$, 转 3 ; 否则, 输出失败信息, 停机.

弦截法不是不动点迭代, 前面的收敛性讨论不适用于它. 关于弦截法的收敛性有以下结果.

定理 8.6　*设 $f(x)$ 在 x^* 邻近二阶连续可微, 且 $f(x^*) = 0, f'(x^*) \neq 0$, 则存在 $\delta > 0$, 当 $x_0, x_1 \in [x^* - \delta, x^* + \delta]$ 时, 由弦截法产生的序列 $\{x_n\}$ 收敛于 x^*, 且收敛阶至少为 $(\sqrt{5}+1)/2 \approx 1.618$.*

证明参看参考文献 [9].

例 8.6　*用弦截法求方程 $f(x) = x^3 + 10x - 20 = 0$ 的根.*

解　因为 $f(1.5) < 0, f(2) > 0, [1.5, 2]$ 为有根区间. 取 $x_0 = 1.5, x_1 = 2$, 按算法 8.3 计算, 结果见表 8.6.

表 8.6　弦截法计算结果

n	x_n	$f(x_n)$
0	1.5	−1.625
1	2	8
2	1.584 415 6	−0.178 370 2
3	1.593 479 5	−0.019 078 6
4	1.594 565 1	0.000 052 56
5	1.594 562 1	-2.2×10^{-7}

扫描全能王 创建

计算结果表明, 迭代 5 次所得近似解精确到 8 位有效数字. 它的收敛速度虽低于 Newton 法, 但比例 8.2所用的简单迭代法快.

8.4 抛 物 线 法

如果考虑用 $f(x)$ 的二次插值多项式的零点来近似 $f(x)$ 的零点, 就导出抛物线法.

设已知方程 (8.1) 的根的三个近似值 x_{n-2}, x_{n-1}, x_n, 以这三点为节点的 $f(x)$ 的二次插值多项式为

$$P_2(x) = f(x_n) + f[x_n, x_{n-1}](x - x_n) + f[x_n, x_{n-1}, x_{n-2}](x - x_n)(x - x_{n-1}) \tag{8.20}$$

为简便起见, 令

$$\begin{cases} a_n = f[x_n, x_{n-1}, x_{n-2}] \\ b_n = f[x_n, x_{n-1}] + f[x_n, x_{n-1}, x_{n-2}](x_n - x_{n-1}) \\ c_n = f(x_n) \end{cases} \tag{8.21}$$

则式 (8.20) 可改写为

$$P_2(x) = a_n(x - x_n)^2 + b_n(x - x_n) + c_n$$

其零点为

$$x = x_n + \frac{-b_n \pm \sqrt{b_n^2 - 4a_n c_n}}{2a_n} = x_n - \frac{2c_n}{b_n \pm \sqrt{b_n^2 - 4a_n c_n}} \tag{8.22}$$

按式 (8.22) $f(x)$ 的二次插值多项式 $P_2(x)$ 有两个零点, 取哪个作为新的近似根? 考虑到 x_n 已是方程 (8.1) 的近似根, 新的近似根自然应在 x_n 的邻近, 故选取新近似根的原则是使得 $|x - x_n|$ 较小, 于是有

$$x_{n+1} = x_n - \frac{2c_n \operatorname{sgn}(b_n)}{|b_n| + \sqrt{b_n^2 - 4a_n c_n}} \tag{8.23}$$

按式 (8.23) 计算方程 (8.1) 的近似根称为抛物线法, 也称为 Müller 方法、二次插值法.

如图 8.5 所示, $y = P_2(x)$ 是过曲线上三点 $(x_i, f(x_i))\,(i = n-2, n-1, n)$ 的抛物线,

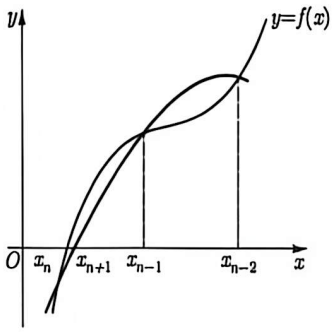


图 8.5 抛物线法

扫描全能王 创建

故抛物线法的几何意义是以过曲线上三点的抛物线与 x 轴的交点作为曲线与 x 轴交点的近似.

抛物线法的计算步骤为

算法 8.4　抛物线法

1. 给定精度 $\varepsilon > 0$, 初始值 x_0, x_1, x_2, 计算 f_0, f_1, f_2 (要求 f_0, f_1, f_2 不等).

2. 计算

$$
\begin{aligned}
a &= f[x_2, x_1, x_0] \\
b &= f[x_2, x_1] + a(x_2 - x_1) \\
c &= f_2 \\
x_3 &= x_2 - \frac{2c\,\mathrm{sgn}(b)}{|b| + \sqrt{b^2 - 4ac}} \\
f_3 &= f(x_3)
\end{aligned}
$$

3. 若 $|f_3| < \varepsilon$, 则 $x^* = x$; 否则 $x_{i+1} \Rightarrow x_i, f_{i+1} \Rightarrow f_i$ $(i = 0, 1, 2)$, 转 2 .

例 8.7　用抛物线法求方程 $f(x) = x^3 + 10x - 20 = 0$ 的根.

解　取 $x_0 = 1.5, x_1 = 1.75, x_2 = 2$, 则 $f_0 = -1.625$, $f_1 = 2.859\,375$, $f_2 = 8$. 按式 (8.21) 和式 (8.23) 计算, 得

$$
\begin{aligned}
a &= f[2, 1.75, 1.5] = 5.25 \\
b &= f[2, 1.75] + a(2 - 1.75) = 21.875 \\
c &= 8 \\
x_3 &= 2 - \frac{2 \times 8}{21.875 + \sqrt{(21.875)^2 - 4 \times 5.25 \times 8}} = 1.594\,900
\end{aligned}
$$

重复上述过程, 迭代数次可得到满足精度要求的解.

可以证明, 如果 $f(x)$ 在其零点 x^* 邻近三阶连续可微, 且初值 x_0, x_1, x_2 充分接近 x^*, 则抛物线法是收敛的. 特别地, 若 x^* 是方程 (8.1) 的单根, 收敛阶为 1.84. 另一方面, 在收敛性的证明中虽然要求初始值充分接近根 x^*, 但实际计算表明, 抛物线法对初值要求并不苛刻, 在初值不太好的情形下常常也能收敛. 它的缺点是程序较复杂, 并且在计算实根的过程中, 也常常需要采用复数运算, 增加了工作量. 因此, 抛物线法适用于当初值不太好时求方程 (8.1) 的复根的情况.

8.5　非线性方程组的解法

设有非线性方程组

扫描全能王 创建

$$\begin{cases} f_1(x_1, x_2, \cdots, x_n) = 0 \\ f_2(x_1, x_2, \cdots, x_n) = 0 \\ \cdots\cdots\cdots\cdots \\ f_n(x_1, x_2, \cdots, x_n) = 0 \end{cases} \tag{8.24}$$

若记 $\boldsymbol{x} = (x_1, x_2, \cdots, x_n)^{\mathrm{T}}$, $\boldsymbol{F}(\boldsymbol{x}) = (f_1(\boldsymbol{x}), f_2(\boldsymbol{x}), \cdots, f_n(\boldsymbol{x}))^{\mathrm{T}}$, 则方程组 (8.24) 可简记成向量形式

$$\boldsymbol{F}(\boldsymbol{x}) = \boldsymbol{0} \tag{8.25}$$

解非线性方程组的方法有许多种, 其中一类是线性化的方法, 它是将方程组中的每个方程线性化得到一个线性方程组, 由此构造迭代格式, 求得方程组 (8.24) 的近似解. 这类方法中常用的一种是 Newton 法; 另一类方法是将解非线性方程组问题化成优化问题, 然后以最优化方法求解. 最速下降法是其中最基本的方法. 本节简单介绍这两种方法.

8.5.1 解非线方程组的 Newton 法

这是 8.3 节中 Newton 法的直接推广. 设给定方程组 (8.25) 的第 k 次近似解 $\boldsymbol{x}^{(k)} = \left(x_1^{(k)}, x_2^{(k)}, \cdots, x_n^{(k)}\right)^{\mathrm{T}}$, 以函数 $\boldsymbol{F}(\boldsymbol{x})$ 在 $\boldsymbol{x}^{(k)}$ 处的一阶 Taylor 多项式近似函数, 得到线性方程组

$$\boldsymbol{F}\left(\boldsymbol{x}^{(k)}\right) + \boldsymbol{F}'\left(\boldsymbol{x}^{(k)}\right)\Delta\boldsymbol{x}^{(k)} = \boldsymbol{0}$$

即

$$\boldsymbol{F}'\left(\boldsymbol{x}^{(k)}\right)\Delta\boldsymbol{x}^{(k)} = -\boldsymbol{F}\left(\boldsymbol{x}^{(k)}\right) \tag{8.26}$$

其中

$$\boldsymbol{F}'\left(\boldsymbol{x}^{(k)}\right) = \left[\begin{array}{cccc} \dfrac{\partial f_1}{\partial x_1} & \dfrac{\partial f_1}{\partial x_2} & \cdots & \dfrac{\partial f_1}{\partial x_n} \\ \dfrac{\partial f_2}{\partial x_1} & \dfrac{\partial f_2}{\partial x_2} & \cdots & \dfrac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & & \vdots \\ \dfrac{\partial f_n}{\partial x_1} & \dfrac{\partial f_n}{\partial x_2} & \cdots & \dfrac{\partial f_n}{\partial x_n} \end{array}\right]_{x=x^{(k)}}$$

称为向量函数 $\boldsymbol{F}(\boldsymbol{x})$ 的 Jacobi 矩阵, $\Delta\boldsymbol{x}^{(k)} = \boldsymbol{x} - \boldsymbol{x}^{(k)} = \left(\Delta x_1^{(k)}, \Delta x_2^{(k)}, \cdots, \Delta x_n^{(k)}\right)^{\mathrm{T}}$. 线性方程组 (8.26) 称为 Newton 方程组. 如果 $\boldsymbol{F}(\boldsymbol{x})$ 的 Jacobi 矩阵在 $\boldsymbol{x}^{(k)}$ 处非奇异, 方程组 (8.26) 有唯一解 $\Delta\boldsymbol{x}^{(k)}$, 则 $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \Delta\boldsymbol{x}^{(k)}$ 为方程组 (8.25) 的第 $k+1$ 次近似解. 按上述过程求方程组 (8.25) 的近似解称为 Newton 方法. 可以证明, 当初始值 $\boldsymbol{x}^{(0)}$ 与解充分接近时, Newton 法是平方收敛的. 计算过程中的停步准则一般采用以下两种:

(1) $\left\|\boldsymbol{F}\left(\boldsymbol{x}^{(k+1)}\right)\right\|_\infty = \max\limits_{1\leqslant i\leqslant n}\left|f_i\left(x_1^{(k+1)}, x_2^{(k+1)}, \cdots, x_n^{(k+1)}\right)\right| < \varepsilon$;

(2) $\left\|\boldsymbol{x}^{(k+1)} - \boldsymbol{x}^{(k)}\right\|_\infty = \max\limits_{1\leqslant i\leqslant n}\left|x_i^{(k+1)} - x_i^{(k)}\right| < \varepsilon$. 当 $\left\|\boldsymbol{x}^{(k)}\right\|$ 较大时, 也可用两次迭代

扫描全能王 创建

的相对误差充分小作为停步准则.

例 8.8　求方程组

$$\begin{cases} 4x_1^2+3x_2^2=1 \\ x_1^3-8x_2^3=1 \end{cases}$$

邻近 $(0.25,-0.5)$ 的解.

解　因为

$$\boldsymbol{F}(\boldsymbol{x})=\begin{bmatrix} 4x_1^2+3x_2^2-1 \\ x_1^3-8x_2^3-1 \end{bmatrix},\quad \boldsymbol{F}'(\boldsymbol{x})=\begin{bmatrix} 8x_1 & 6x_2 \\ 3x_1^2 & -24x_2^2 \end{bmatrix}$$

$\boldsymbol{x}^{(0)}=(0.25,-0.5)^{\mathrm{T}}$, 故 Newton 方程组为

$$\boldsymbol{F}'(\boldsymbol{x}^{(0)})\Delta\boldsymbol{x}^{(0)}=\begin{bmatrix} 2 & -3 \\ 0.187 & -6 \end{bmatrix}\begin{bmatrix} \Delta x_1^{(0)} \\ \Delta x_2^{(0)} \end{bmatrix}=-\begin{bmatrix} 0 \\ 0.015\,625 \end{bmatrix}$$

其解为 $\Delta x_1^{(0)}=0.004\,098\,36$, $\Delta x_2^{(0)}=0.002\,732\,24$, 所以

$$\boldsymbol{x}^{(1)}=\boldsymbol{x}^{(0)}+\Delta\boldsymbol{x}^{(0)}=(0.254\,098\,36,-0.497\,267\,76)^{\mathrm{T}}$$

继续迭代下去, 可得近似解为 $\boldsymbol{x}^*\approx(0.254\,078\,6,-0.497\,251\,2)^{\mathrm{T}}$.

Newton 法具有收敛快, 形式简单等优点, 但它对初值的要求较高. 另外, Newton 法每迭代一次, 需计算 $n+n^2$ 个函数值及导数值, 求解一个 n 阶线性方程组, 计算量较大.

8.5.2　最速下降法

设有非线性方程组

$$\begin{cases} f_1(x_1,x_2,\cdots,x_n)=0 \\ f_2(x_1,x_2,\cdots,x_n)=0 \\ \quad\cdots\cdots\cdots\cdots \\ f_m(x_1,x_2,\cdots,x_n)=0 \end{cases} \tag{8.27}$$

构造模函数

$$\Phi(\boldsymbol{x})=\sum_{i=1}^{m}[f_i(\boldsymbol{x})]^2 \tag{8.28}$$

其中 $\boldsymbol{x}=(x_1,x_2,\cdots,x_n)^{\mathrm{T}}$, 显然, 方程组 (8.27) 的解是模函数 $\Phi(\boldsymbol{x})$ 的零点, 反之亦然. 因此, 求解非线性方程组 (8.27) 可以转化为求解下述最优化问题

$$\min_{\boldsymbol{x}\in\mathbf{R}^n}\Phi(\boldsymbol{x}) \tag{8.29}$$

这是一个无约束最优化问题, 有许多求解方法, 最基本的一种方法是最速下降法.

最速下降法的基本思想在 3.6 节中已介绍过. 为直观起见, 下面着重讨论二维的情形. 设

扫描全能王 创建

$$\Phi(x_1,x_2)=\sum_{i=1}^{m}[f_i(x_1,x_2)]^2 \tag{8.30}$$

记 $g_1(x_1,x_2)=\dfrac{\partial\Phi}{\partial x_1}$, $g_2(x_1,x_2)=\dfrac{\partial\Phi}{\partial x_2}$, 负梯度方向 $-\boldsymbol{G}(x_1,x_2)=-\left(g_1(x_1,x_2),\ g_2(x_1,x_2)\right)$ 就是点 (x_1,x_2) 处函数 $\Phi(x_1,x_2)$ 的最速下降方向. 显然, 若 $\boldsymbol{G}\neq\boldsymbol{0}$, 则沿 $-\boldsymbol{G}$ 方向 $\Phi(x_1,x_2)$ 的值必定是减少的. 因此, 从初始值 $(x_1^{(0)},x_2^{(0)})$ 出发, 沿方向 $-\boldsymbol{G}(x_1^{(0)},x_2^{(0)})$ 一定可以找到一点 $(x_1^{(1)},x_2^{(1)})$, 使得 $\Phi(x_1^{(1)},x_2^{(1)})<\Phi(x_1^{(0)},x_2^{(0)})$, 然后再从 $(x_1^{(1)},x_2^{(1)})$ 出发, 沿 $-\boldsymbol{G}(x_1^{(1)},x_2^{(1)})$ 方向找点 $(x_1^{(2)},x_2^{(2)})$, 使得 $\Phi(x_1^{(2)},x_2^{(2)})<\Phi(x_1^{(1)},x_2^{(1)})$. 如此继续下去, 就可能逼近到 $\Phi(x_1,x_2)$ 的最小点 (x_1^*,x_2^*).

从点 $(x_1^{(k)},x_2^{(k)})$ 出发, 沿方向 $-\boldsymbol{G}(x_1^{(k)},x_2^{(k)})$, 如何求出新近似解 $(x_1^{(k+1)},x_2^{(k+1)})$? 自然地, 我们希望 $(x_1^{(k+1)},x_2^{(k+1)})$ 是使得 $\Phi(x_1,x_2)$ 的值下降最多的点, 即

$$\begin{aligned}\Phi(x_1^{(k+1)},x_2^{(k+1)})&=\min_{\lambda>0}\Phi(x_1^{(k)}-\lambda g_1(x_1^{(k)},x_2^{(k)}),x_2^{(k)}-\lambda g_2(x_1^{(k)},x_2^{(k)}))\\&=\min_{\lambda>0}\varphi(\lambda)\end{aligned} \tag{8.31}$$

这是一个一维极值问题, 称为一维搜索问题, 射线方向 $-\boldsymbol{G}(x_1^{(k)},x_2^{(k)})$ 称为搜索方向. 一维搜索的方法很多, 如果 $\varphi(\lambda)$ 的形式简单, 可用一元函数求极值的方法求解; 否则, 可用对分搜索、Fibonacci 搜索、黄金分割、插值法等方法求解. 求出问题 (8.31) 的解 λ 后, 代入公式

$$x_1^{(k+1)}=x_1^{(k)}-\lambda g_1(x_1^{(k)},x_2^{(k)}),x_2^{(k+1)}=x_2^{(k)}-\lambda g_2(x_1^{(k)},\quad x_2^{(k)})$$

即得第 $k+1$ 次近似解. 按上述过程求问题 (8.30) 的解称为最速下降法, 这一过程可直接推广到 n 维的情形.

如图 8.6 所示, $u=\Phi(x_1,x_2)$ 是一空间曲面, 它与 Ox_1x_2 面的切点即为解点 (x_1^*,x_2^*). 图 8.6(b) 是曲面 $u=\Phi(x_1,x_2)$ 的等高线图, 同一条等高线上的 Φ 值是相同的. 在解点 (x_1^*,x_2^*) 邻近, 等高线是以 (x_1^*,x_2^*) 为内点的封闭曲线, 且相应的 Φ 值由外往里递减. 负梯度方向 $-\boldsymbol{G}(x_1^{(k)},x_2^{(k)})$ 是等高线在点 $(x_1^{(k)},x_2^{(k)})$ 处的内法线方向. 从 $(x_1^{(k)},x_2^{(k)})$ 出发, 沿方向为 $-\boldsymbol{G}(x_1^{(k)},x_2^{(k)})$ 的射线行进到等高线与射线相切时, 切点即为 $(x_1^{(k+1)},x_2^{(k+1)})$.

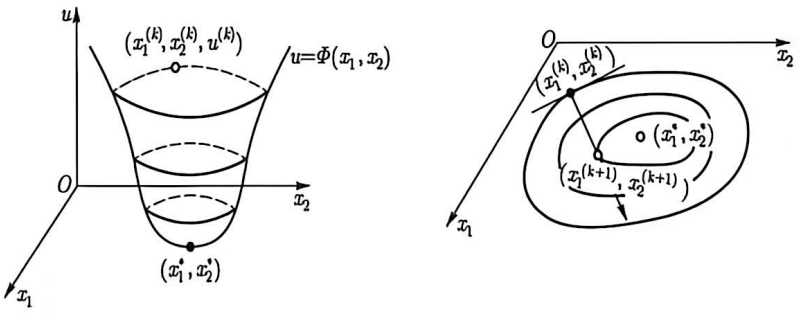


图 8.6 最速下降法

扫描全能王 创建

最速下降法的计算步骤为

算法 8.5　最速下降法

1. 选取初始点 $\boldsymbol{x}^{(0)}=(x_1^{(0)},x_2^{(0)},\cdots,x_n^{(0)})$, 置 $k=0$.
2. 计算 $\Phi(\boldsymbol{x})$ 在 $\boldsymbol{x}^{(k)}$ 处的梯度值

$$G_k=G\left(\boldsymbol{x}^{(k)}\right)=\left(\frac{\partial\Phi}{\partial x_1},\frac{\partial\Phi}{\partial x_2},\cdots,\frac{\partial\Phi}{\partial x_n}\right)_{\boldsymbol{x}=\boldsymbol{x}^{(k)}}$$

3. 若 $\|G_k\|<\varepsilon$, 得近似解 $\boldsymbol{x}^*=\boldsymbol{x}^{(k)}$; 否则, 求解一维搜索问题 $\min\limits_{\lambda>0}\Phi\left(\boldsymbol{x}^{(k)}-\lambda G_k\right)$, 得解 λ.
4. 置 $\boldsymbol{x}^{(k)}-\lambda G_k\Rightarrow\boldsymbol{x}^{(k)},k+1\Rightarrow k$, 转 2.

最速下降法计算简单, 收敛性好, 但收敛速度较慢, 它仅具有线性收敛速度. 特别当 $\Phi(x)$ 的等高线成狭长的谷时 (见图 8.7), 最速下降法收敛极慢. 因此常常将它与 Newton 法联用, 先用最速下降法求得较好的近似值, 然后用 Newton 法继续求方程组 (8.27) 的解.

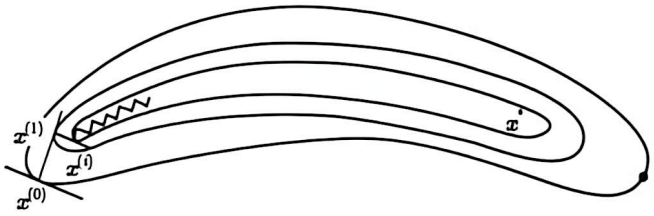


图 8.7　等高线狭长

需要说明的是, 最优化方法在实际使用时常常不作精确的一维搜索, 而只要求 $\boldsymbol{x}^{(k+1)}$ 满足某些条件, 如使 $\Phi(\boldsymbol{x})$ 值下降且位移 $\boldsymbol{x}^{(k+1)}-\boldsymbol{x}^{(k)}$ 不太小等.

8.6　算法实现与应用实例

8.6.1　高温炉的温度计算

机电工程中常需要制造半导体装置, 如太阳能电池、集成电路等, 这中间需要对一些材料进行高温热处理. 典型的高温炉由加热元件及火墙构成.设 x 表示高温炉内的温度, T 表示炉外周围环境的温度. 由于热量的对流和辐射, 炉内温度将变化, 已知温度的变化率与炉内和炉外的温差成比率. 根据实验观察, 温度变化过程可处理为如下的模型:

$$\frac{\mathrm{d}x}{\mathrm{d}t}=\alpha(T-x)+\beta(T^4-x^4)+\gamma u^2.$$

扫描全能王 创建

其中 α, β 和 γ 是正常数, 依赖于高温炉的保温能力. $\alpha(T-x)$ 表示对流散热, $\beta(T^4-x^4)$ 表示辐射散热, u 表示加热元件的电流.

在许多热处理过程中, 温度的控制极为重要. 设加热元件的电流 u 是常数, 为得到稳定状态的温度 x, 令 $\dfrac{\mathrm{d}x}{\mathrm{d}t}=0$, 上述微分方程转化为非线性方程

$$\alpha(T-x)+\beta(T^4-x^4)+\gamma u^2=0$$

问题归结为求解非线性方程的根的问题. 现取数据如下:

$$u=21\ \mathrm{A},\quad T=20\ ^\circ\mathrm{C}=293.15\ \mathrm{K},\quad \alpha=50\ \mathrm{min}^{-1}$$

$$\beta=2\times10^{-7}\ \mathrm{min}^{-1}\cdot\mathrm{K}^{-3},\quad \lambda=800\ \mathrm{K}\cdot\mathrm{min}^{-1}\cdot\mathrm{A}^2$$

代入方程, 得

$$f(x)=3.689\times10^5-50x-2\times10^{-7}x^4=0$$

可以采用北太天元软件, 编制脚本文件, 采用二分法求解方程.

```
function y=pfun(x)
  y=3.689e5-50*x-2*x*x*x*x/(1e-7) ;
end
```

```
% 二分法的实现求解 fun(x)=0, fh=fun, x0=[x1,x2] 为有根区间, tol 为误差限
function y=fzero( fh, x0, tol)
  d=(x0(2)-x0(1));
  while d>tol
    y1=feval(fh, x0(1));
    y2=feval(fh, x0(2));
    xc=(x0(1)+x0(2))/2.0;
    d=(x0(2)-x0(1))/2;
    yc=feval(fh, xc);
    if yc==0
      break;
    else
      if (y1*yc)>0
      x0(1)=xc;
      else
      x0(2)=xc;
      end
    end
  end
  y=xc;
end
```


扫描全能王 创建

这里我们选 x0 为 x0=[0, 2000], tol=0.01,

≫ fh=@pfun

≫ x=fzero(fh, x, tol)　% 调用函数 fzero()

得到结果为 x=1 118.5. 实际上, 如果 x0=[−2 000, 0], 我们得出负值, 是方程的根 x=−1 210.5, 但不是问题的解.

也可以直接采用北太天元软件自带的求解多项式函数零点的函数 roots(c) 求解, c 为多项式由高到低的系数.

≫ c=[2e-7,0,0,-50, 3.689e5]

≫ roots(c)

8.6.2　悬索垂度与张力的计算

公路和铁路设计中常出现高架悬索桥梁, 由于桥梁的质量, 在设计中需计算各个支撑部件所承受的张力. 设高架悬索系统如图 8.8, 其中 a 表示悬索的跨度, x 是悬索的垂度, m 是悬索承受的质量. 设悬索承受的质量是均匀分布的, $g = 9.78\ \mathrm{m/s^2}$ 表示重力加速度, 则悬索承受的负荷密度为 $w = mg/a$. 若不计温度变化的影响, 悬索端点的张力由公式

$$T = \frac{wa}{2}\sqrt{1+\left(\frac{a}{4x}\right)^2}$$

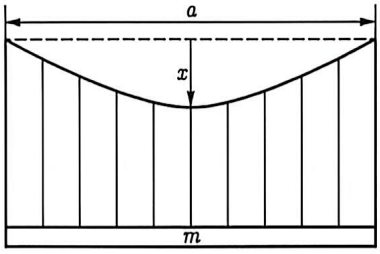


图 8.8　高架悬索桥梁

确定, 为此需计算悬索的垂度 x. 设悬索的长度为 $L > a$, 垂度 x 近似满足如下的非线性代数方程

$$L = a\left[1+\frac{8}{3}\left(\frac{x}{a}\right)^2-\frac{32}{5}\left(\frac{x}{a}\right)^4+\frac{256}{7}\left(\frac{x}{a}\right)^6\right]$$

不妨设 $a = 120$ m, $L = 125$ m, $m = 1\ 500$ kg, 则悬索的负荷密度 $w = 122.25$ N/m. 采用 Newton 迭代法求解非线性方程, 取迭代初值 $x_0 = a/2$, 误差精度 $\varepsilon = 10^{-6}$ 开始计算, 迭代到第 7 步, 得到结果 $x = 15.27$ m, 悬索端点承受张力 $T = 16\ 168.50$ N. 由张力计算公式知: 悬索的垂度 x 越大, 悬索受到的张力 T 越小, 因此可调节悬索的长度 L, 增加悬索的垂度 x, 减小其承受的张力 T. 采用北太天元软件实现 Newton 迭代法, 编写脚本文件 iter_newton.m, 计算悬索的长度与垂度及压力的关系, 程序如下:

扫描全能王 创建

```
function iter_newton(x0, Nmax, tol)
a=120; % 悬索的高度 (m)
m=1500; % 悬索的质量 (kg)
g=9.78; % 重力加速度
w=m*g/a;
fx=@(x,L) (a*(1+8/3*(x/a)^2-32/5*(x/a)^4+256/7*(x/a)^6))-L;
% 匿名方式定义函数 fx(x,L), 悬索垂度和长度的满足的方程 fx(x,L)=0
dfx=@(x) ((16/3*(x/a)-32*4/5*(x/a)^3+256*6/7*(x/a)^5));
% 匿名方式定义 fx(x,L) 对 x 的导函数 dfx(x)
for j=1:6
  Lx(j)=a+5*j;
  y0=x0;
  for i=1:Nmax
    y1=y0-fx(y0,Lx(j))/dfx(y0); %Newton 迭代法
    if(abs(y1-y0)<tol)
      break; % 验证是否满足误差要求
    else
      y0=y1; % 不满足误差要求, 以新的迭代值为初值继续迭代
    end
  end
  Tx(j)=y1; % 存储悬索垂度
  T(j)=(w*a/2)*sqrt(1+(a/(4*y1))^2); % 存储悬索端点张力
end
Lx   Tx   T % 依次输出与悬索长度 Lx 对应的垂度值 Tx 及张力值 T
end
```

表 8.7 显示了对于悬索的不同长度, 数值计算所得悬索的垂度及其端点所受的张力.

表 8.7　与悬索长度 L 对应的垂度 x 和张力 T 的计算结果

L/m	125	130	135	140	145	150
x/m	15.27	21.94	27.19	31.63	35.45	38.75
T/N	16 168.5	12 426.8	10 922.1	10 109.1	9 608.6	9 275.9

评注九

本章重点讨论了单个非线性方程的情形, 但在基本概念和收敛性方面, 方程组的情形和单个方程的情形类似.

简单迭代法的理论基础是求迭代函数 $\varphi(x)$ 的不动点. 迭代收敛的充分条件是在有根区间 $[a,b]$ 上, 有 $|\varphi'(x)|<1$. 简单迭代法可推广到方程组的情形.

扫描全能王 创建

Newton 迭代法收敛速度很快, 在单根的情况下, Newton 迭代法是二阶收敛的. Newton 迭代法对初值的要求较高, 初值选得不好时, Newton 迭代序列有可能是发散的, 此时可先用二分法或简单迭代法获得有一定精度的近似值, 再将此近似值作为 Newton 迭代法的初值开始迭代计算, 或将方法改进为 Newton 下山法

$$x_{k+1}=x_k-\lambda\frac{f(x_k)}{f'(x_k)}\quad(k=0,1,2,\cdots)$$

其中 λ $(0<\lambda<1)$ 称为下山因子. 适当选择下山因子, 可使迭代序列收敛. 下山因子的选取没有统一公式, 一般通过试算确定.

弦截法和抛物线法是多点迭代法, 它们属于插值型方法. 弦截法的收敛阶是 $r=\dfrac{1+\sqrt{5}}{2}\approx 1.618$, 抛物线法的收敛阶是 $r\approx 1.85$. 抛物线法适用于求方程的复根.

Newton 迭代法可以方便地推广到 n 维非线性方程组的情形. 此时需要引入 Jacobi 矩阵 $\boldsymbol{J}(x)=\boldsymbol{F}'\left(\boldsymbol{x}^{(k)}\right)$, 计算中可用中心差商近似 $\boldsymbol{J}(x)$ 中的各偏导数, 然后用直接法或用迭代法求解线性代数方程组. 针对 Newton 迭代法的弱点, 有许多改进的形式, 它们统称为 Newton 型方法, 例如, DFP 算法、BFGS 算法、Fletcher-Freeman 方法和 Gill-Murray 方法都是比较有效的 [11]. 非线性方程组的求解问题可以转化为一个无约束优化问题, 本章仅介绍了最速下降法. 最速下降法是一个经典的下降算法, 它简单易行, 但收敛速度缓慢, 目前使用得不多. 然而在理论上, 它是其他下降算法的基础. 求解大型无约束问题的有效方法是共轭梯度法, 计算时需要求模函数的导数. 如果模函数不可导, 或计算导数过于复杂, 可以采用不需要计算导数的直接法, 如模式搜索法和单纯形法等.

本章知识要点　二分法; 简单迭代法; 牛顿法; 弦截法.

算法历史及背后的数学家

习题八

1. 用二分法求解下列方程, 要求误差不超过 10^{-5}:

(1) $x-\ln x=2$ 在区间 $[2,4]$ 内的根;

(2) $x\mathrm{e}^x-1=0$ 在区间 $[0,1]$ 内的根;

(3) $x^3+4x^2-10=0$ 在区间 $[1,2]$ 内的根.

2. 已知方程 $x^3-x^2-1=0$ 在 $x_0=1.5$ 邻近有根, 试判断下列迭代格式:

(1) $x_{n+1}=1+\dfrac{1}{x_n^2}$;

(2) $x_{n+1}=\sqrt{\dfrac{1}{x_n-1}}$;

(3) $x_{n+1}=\sqrt[3]{1+x_n^2}$.

扫描全能王 创建

在 $x_0 = 1.5$ 邻近的收敛性. 选一种迭代格式计算 1.5 邻近的根, 要求结果具有 5 位有效数字.

3. 用简单迭代法求下列方程的根, 要求 $|x_{n+1} - x_n| < 10^{-5}$:

(1) $x - \cos x = 0$;

(2) $4x - \mathrm{e}^{-x} = 0$;

(3) $x^3 - 2x - 5 = 0$.

4. 用 Steffensen 方法求方程 $x^3 - x^2 - 1 = 0$ 在区间 $[1,2]$ 内的根, 要求结果具有 5 位有效数字, 并与第 2 题的结果进行比较.

5. 分别用简单迭代法与 Steffensen 方法求方程 $x\mathrm{e}^x - 1 = 0$ 的根, 要求误差不超过 10^{-5}.

6. 对于方程 $f(x) = 0$, 使用迭代格式

$$x_{n+1} = x_n + cf(x_n) \quad (n = 0, 1, 2, \cdots)$$

若 $f'(x) \neq 0$, 如何选取 c 才能保证迭代收敛.

7. 设有解方程 $12 - 3x + 2\cos x = 0$ 的迭代法

$$x_{n+1} = 4 + \frac{2}{3}\cos x_n$$

(1) 证明对 $\forall x_0 \in \mathbf{R}$, 此迭代法均收敛到方程的根;

(2) 取 $x_0 = 4$, 用此迭代法求方程根的近似值, 使其误差不超过 $\frac{1}{2} \times 10^{-5}$;

(3) 此迭代法的收敛阶是多少? 说明理由.

8. 证明由迭代格式

$$x_{n+1} = \frac{x_n}{2} + \frac{1}{x_n} \quad (n = 0, 1, 2, \cdots)$$

产生的迭代序列 $\{x_n\}$, 对任意 $x_0 > 1$ 均收敛到 $\sqrt{2}$, 且是二阶收敛的.

9. 证明当 x_0 充分接近 $\sqrt{a}$ 时, 按迭代公式

$$x_{n+1} = \frac{x_n(x_n^2 + 3a)}{3x_n^2 + a} \quad (n = 0, 1, 2, \cdots)$$

产生的迭代序列 $\{x_n\}$ 收敛于 $\sqrt{a}$, 且收敛阶为 3 .

10. 用 Newton 法求解下列方程, 要求 $|x_{n+1} - x_n| < 10^{-6}$:

(1) $x - \ln x = 2$;

(2) $x\mathrm{e}^x - 1 = 0$;

(3) $x + \sin x = 1$.

11. 试用 Newton 法推导出 $\sqrt[3]{a}$ 的迭代格式, 并求 $\sqrt[3]{3}$ 的近似值, 要求误差不超过 10^{-5}.

12. 证明方程 $f(x) = x^3 - x - 1 = 0$ 在区间 $[1,2]$ 内仅有一个根. 取 $x_0 = 1.5$, 用 Newton 法求此根, 要求精确到 5 位有效数字, 并说明其收敛阶数为 2.

13. 若 x^* 是方程 $f(x) = 0$ 的 r 重根, 即 $f(x) = (x - x^*)^r \varphi(x)$, 其中 $\varphi(x^*) \neq 0$, 试证明:

扫描全能王 创建

(1) Newton 法在 x^* 邻近是线性收敛的;

(2) 改进的 Newton 法

$$x_{n+1} = x_n - \frac{rf(x_n)}{f'(x_n)} \quad (n = 0, 1, 2, \cdots)$$

14. 设 x^* 是方程 $f(x) = 0$ 的重根, 令

$$u(x) = \frac{f(x)}{f'(x)}$$

试说明 x^* 方程 $u(x) = 0$ 的单根, 并用 Newton 法到处相应的迭代格式.

15. 利用上题导出的迭代格式, 求方程

$$x^4 - 0.4x^3 + 1.04x^2 - 0.4x + 0.04 = 0$$

在 0.1 邻近的重根.

16. 试用割线法求解下列方程, 要求 $|x_{n+1} - x_n| < 10^{-5}$:

(1) $x - \ln x = 2$;

(2) $5x - \cos x - 1 = 0$;

(3) $x = 3 - 2^x$.

17. 用 Newton 法求方程组

$$\begin{cases} x^2 + y^2 - 5 = 0 \\ (x+1)y - 3x - 1 = 0 \end{cases}$$

在 $(1, 1)$ 邻近的解, 迭代 3 次.

18. 求下列方程组的解:

(1) $\begin{cases} x = x^2 + y^2, \\ y = x^2 - y^2, \end{cases} \quad (x_0, y_0) = (0.8, 0.4)$;

(2) $\begin{cases} x - 0.7\sin x - 0.2\cos y = 0, \\ y - 0.7\cos x + 0.2\sin y = 0, \end{cases} \quad (x_0, y_0) = (0.5, 0.5)$.

数值实验八

1. 设有方程 $f(x) = x^2 + \sin 10x - 1 = 0$, 先作出函数 $y = f(x)$ 的图形, 观察方程的有根区间, 构造收敛的迭代格式并编程求出方程 $f(x) = 0$ 的所有根.

2. 求方程 $x = \mathrm{e}^{-x}$ 在 $x = 0.5$ 附近的根, 按如下算法编写程序, 求方程的近似解. 要求精确到 10^{-8}:

(1) 取 $\varphi(x) = \mathrm{e}^{-x}$, 用简单迭代法 $x_{n+1} = \varphi(x_n)$ 计算;

(2) 用加速收敛的迭代格式 $x_{n+1} = \lambda\varphi(x_n) + (1 - \lambda)x_n, \lambda = 0.625$ 计算.

3. 先作图分析下列方程的有根区间, 编写 Newton 法和弦截法的程序, 求解下列方程:

(1) $5x^{23} - 6x^7 + 8x^6 - 5x^2 = 0$;

扫描全能王 创建

(2) $\mathrm{e}^{-2x+6}\cos(10x)+\mathrm{e}^{-3x}\sin(2x)=\mathrm{e}^{6}$.

4. 试编写求解非线性方程组的 Newton 法和最速下降法的程序, 求解下列方程组:

(1) $\begin{cases} x^2+y^2=1, \\ 0.75x^2-y=-0.9 \end{cases}$ 在点 $(-1,0.1)$ 和 $(0.5,0.9)$ 附近的根;

(2) $\begin{cases} x+3y^3+2z^2=\dfrac{1}{2}, \\ x^2+3y+z^3=2, \\ x^3+2y+2z^2=\dfrac{1}{2} \end{cases}$ 在点$(-1,0.1,0.7)$ 附近的根;

(3) $\begin{cases} x^2\mathrm{e}^{-\frac{xy^2}{2}}+\mathrm{e}^{-\frac{y^2}{2}}\sin(xy)=0, \\ x^2\cos\left(x+y^2\right)+y^2\mathrm{e}^{x+y}=0 \end{cases}$ 在点 $(-1,1),(-0.2,4)$ 和 $(0.1,-2)$ 附近的根.

扫描全能王 创建

/第九章/

常微分方程数值解法

许多实际问题的数学模型是微分方程或微分方程组的定解问题, 如物体运动、电路振荡、化学反应及生物群体的变化等. 能用解析方法求出精确解的微分方程为数不多, 而且有的方程即使有解析解, 也可能由于解表达式非常复杂而不易计算. 因此有必要研究微分方程的数值解法.

本章主要讨论一阶常微分方程的初值问题, 其一般形式是

$$\begin{cases} \dfrac{\mathrm{d}y}{\mathrm{d}x} = f(x,y), \quad a \leqslant x \leqslant b \\ y(a) = y_0 \end{cases} \tag{9.1}$$

在下面的讨论中, 我们总假定函数 $f(x,y)$ 连续, 且关于 y 满足 Lipschitz (利普希茨) 条件, 即存在常数 L, 使得

$$|f(x,y) - f(x,\bar{y})| \leqslant L|y - \bar{y}|$$

这样, 由常微分方程理论知, 初值问题 (9.1) 的解必定存在唯一.

所谓数值解法, 就是求问题 (9.1) 的解 $y(x)$ 在若干点

$$a = x_0 < x_1 < x_2 < \cdots < x_N = b$$

处的近似值 y_n $(n = 1,2,\cdots,N)$ 的方法, y_n $(n = 1,2,\cdots,N)$ 称为问题 (9.1) 的数值解, $h_n = x_{n+1} - x_n$ 称为由 x_n 到 x_{n+1} 的步长. 今后如无特别说明, 我们总取步长为常量 h.

建立数值解法, 首先要将微分方程离散化, 一般采用以下几种方法:

1. 用差商近似导数

若用向前差商 $\dfrac{y(x_{n+1}) - y(x_n)}{h}$ 代替 $y'(x_n)$ 代入问题 (9.1) 中的微分方程, 则得

$$\frac{y(x_{n+1}) - y(x_n)}{h} \approx f(x_n, y(x_n)) \quad (n = 0,1,\cdots)$$

化简得

$$y(x_{n+1}) \approx y(x_n) + hf(x_n, y(x_n))$$

如果用 $y(x_n)$ 的近似值 y_n 代入上式右端, 所得结果作为 $y(x_{n+1})$ 的近似值, 记为 y_{n+1}, 则有

$$y_{n+1} = y_n + hf(x_n, y_n) \quad (n = 0,1,\cdots) \tag{9.2}$$

这样, 问题 (9.1) 的近似解可通过求解下述问题

$$\begin{cases} y_{n+1} = y_n + hf(x_n, y_n) \quad (n = 0,1,\cdots) \\ y_0 = y(a) \end{cases} \tag{9.3}$$

得到, 按式 (9.2) 由初值 y_0 可逐次算出 $y_1, y_2, \cdots$. 式 (9.3) 是个离散化的问题, 称为差分方

扫描全能王 创建

程初值问题.

需要说明的是, 用不同的差商近似导数, 将得到不同的计算公式.

2. 用数值积分方法

将问题 (9.1) 的解表达成积分形式, 用数值积分方法离散. 例如, 对微分方程两端积分, 得

$$y(x_{n+1})-y(x_n)=\int_{x_n}^{x_{n+1}} f(x,y(x))\mathrm{d}x \quad (n=0,1,\cdots) \tag{9.4}$$

用 y_{n+1},y_n 代替 $y(x_{n+1}),y(x_n)$, 对右端积分采用取左端点的矩形公式, 即

$$\int_{x_n}^{x_{n+1}} f(x,y(x))\mathrm{d}x \approx hf(x_n,y_n)$$

则由式 (9.4) 可得

$$y_{n+1}-y_n=hf(x_n,y_n)$$

于是问题 (9.1) 的近似解可由以下公式求出

$$\begin{cases} y_{n+1}=y_n+hf(x_n,y_n) \quad (n=0,1,\cdots) \\ y_0=y(a) \end{cases}$$

完全类似地, 对右端积分采用取右端点的矩形公式或其他数值积分方法, 又可得到不同的计算公式.

3. 用 Taylor 多项式近似

将函数 $y(x)$ 在 x_n 处展开, 取一次 Taylor 多项式近似, 则得

$$y(x_{n+1})=y\left(x_n+h\right)\approx y(x_n)+hy'(x_n)=y(x_n)+hf(x_n,y(x_n))$$

再将 $y(x_n)$ 的近似值 y_n 代入上式右端, 所得结果作为 $y(x_{n+1})$ 的近似值 y_{n+1}, 得到离散化的计算公式

$$y_{n+1}=y_n+hf(x_n,y_n)$$

以上三种方法都是将微分方程离散化的常用方法, 每一类方法又可导出不同形式的计算公式. 其中的 Taylor 展开法, 不仅可以得到求数值解的公式, 而且容易估计截断误差, 故本章在推导数值求解公式时主要用这种方法.

在上面的例子中, 我们给出了解初值问题 (9.1) 的一种最简单的数值公式 (9.3), 虽然由于它的精度较低而不常使用, 但这一公式的导出过程能较清楚地说明构造数值求解公式的基本思想, 且几何意义明确. 因此我们首先介绍这种方法.

9.1 Euler 方法及其改进方法

9.1.1 Euler 方法

Euler 方法就是用差分方程初值问题

扫描全能王 创建

$$\begin{cases} y_{n+1} = y_n + hf(x_n, y_n) \quad (n = 0, 1, \cdots) \\ y_0 = y(a) \end{cases}$$

的解来近似微分方程初值问题 (9.1) 的解, 即由公式 (9.2) 依次算出 $y(x_n)$ 的近似值 y_n $(n = 1, 2, \cdots)$.

例 9.1　用 Euler 方法求初值问题

$$\begin{cases} \dfrac{\mathrm{d}y}{\mathrm{d}x} = x - y, \quad 0 \leqslant x \leqslant 1 \\ y(0) = 0 \end{cases}$$

的数值解.

解　此问题的精确解为 $y = x + \mathrm{e}^{-x} - 1$, 对此问题 Euler 公式的具体形式为

$$y_{n+1} = y_n + h(x_n - y_n) \quad (n = 0, 1, \cdots)$$

若取 $h = 0.1, N = 10$, 由初值 $y_0 = 0$ 出发按上式计算, 所得数值结果见表 9.1.

表 9.1　Euler 方法数值结果

x_n	y_n	$y(x_n)$	$y(x_n) - y_n$
0	0.000 000	0.000 000	0.000 000
0.1	0.000 000	0.004 837	0.004 837
0.2	0.010 000	0.018 731	0.008 731
0.3	0.029 000	0.040 818	0.011 818
0.4	0.056 100	0.070 320	0.014 220
0.5	0.090 490	0.106 531	0.016 041
0.6	0.131 441	0.148 812	0.017 371
0.7	0.178 297	0.196 585	0.018 288
0.8	0.230 467	0.249 329	0.018 862
0.9	0.287 420	0.306 570	0.019 150
1.0	0.348 678	0.367 879	0.019 201

从几何上看, 微分方程 $y' = f(x, y)$ 在 xOy 平面上确定了一个向量场: 点 (x, y) 处的方向斜率为 $f(x, y)$. 问题 (9.1) 的解是过点 (x_0, y_0) 的积分曲线 $y = y(x)$, 且此曲线上每点的切向都与向量场在这点的方向一致. 而 Euler 方法所给出的解是过点 (x_0, y_0) 的一条折线 $P_0P_1 \cdots P_N$ 上的转折点, 折线上每一段的方向都与向量场在该线段的左端点处的方向一致 (见图 9.1), 自然地, 可用折线 $P_0P_1 \cdots P_N$ 近似问题 (9.1) 的解曲线, 因此 Euler 方法又称为 Euler 折线法.

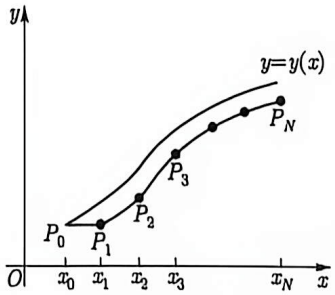


图 9.1　例 9.1 的数值解和解析解

扫描全能王 创建

9.1.2 向后 Euler 法

如果在离散微分方程时, 用向后差商近似导数, 即

$$y'(x_{n+1}) \approx \frac{y(x_{n+1}) - y(x_n)}{h}$$

则得计算公式

$$\begin{cases} y_{n+1} = y_n + hf(x_{n+1}, y_{n+1}) \\ y_0 = y(a) \end{cases} \tag{9.5}$$

这就是向后 Euler 方法.

向后 Euler 方法与 Euler 方法形式上相似, 但计算时却复杂得多. Euler 公式是显式的, 可以直接计算. 向后 Euler 公式的右端含有 y_{n+1}, 不能直接计算出 y_{n+1}, 称为隐式公式, 需要迭代求解. 一般先用 Euler 公式算出迭代初值 $y_{n+1}^{(0)} = y_n + hf(x_n, y_n)$, 再用向后 Euler 公式迭代

$$y_{n+1}^{(k+1)} = y_n + hf(x_{n+1}, y_{n+1}^{(k)}) \quad (k = 0, 1, \cdots)$$

当 $\left|y_{n+1}^{(k+1)} - y_{n+1}^{(k)}\right| < \varepsilon$ 时, 终止迭代, 取 $y_{n+1} = y_{n+1}^{(k+1)}$.

下面考虑向后 Euler 公式迭代的收敛性. 由于 $f(x, y)$ 关于 y 满足 Lipschitz 条件, 即

$$|f(x, y) - f(x, \tilde{y})| \leqslant L|y - \tilde{y}|$$

那么

$$\begin{aligned} \left|y_{n+1}^{(k+1)} - y_{n+1}^{(k)}\right| &= h\left|f(x_{n+1}, y_{n+1}^{(k)}) - f(x_{n+1}, y_{n+1}^{(k-1)})\right| \\ &\leqslant hL\left|y_{n+1}^{(k)} - y_{n+1}^{(k-1)}\right| \leqslant \cdots \\ &\leqslant (hL)^{(k)}\left|y_{n+1}^{(1)} - y_{n+1}^{(0)}\right| \end{aligned}$$

由此得出, 当 $hL < 1$ 时, 向后 Euler 方法迭代收敛, 称 $hL < 1$ 为迭代收敛的条件.

9.1.3 梯形公式

利用数值积分方法离散微分方程时, 如果用梯形公式计算式 (9.4) 中右端积分, 即

$$\int_{x_n}^{x_{n+1}} f(x, y(x))\mathrm{d}x \approx \frac{h}{2}\left[f(x_n, y(x_n)) + f\left(x_{n+1}, y(x_{n+1})\right)\right]$$

并用 y_n, y_{n+1} 代替 $y(x_n), y(x_{n+1})$, 得求解初值问题 (9.1) 的梯形公式

$$y_{n+1} = y_n + \frac{h}{2}\left[f(x_n, y_n) + f\left(x_{n+1}, y_{n+1}\right)\right] \tag{9.6}$$

梯形公式也是隐式公式, 需用迭代法求解, 迭代格式为

$$\begin{cases} y_{n+1}^{(0)} = y_n + hf(x_n, y_n) \\ y_{n+1}^{(k+1)} = y_n + \dfrac{h}{2}\left[f(x_n, y_n) + f(x_{n+1}, y_{n+1}^{(k)})\right] \end{cases} \quad (k = 0, 1, \cdots) \tag{9.7}$$

扫描全能王 创建

同样由于 $f(x,y)$ 关于 y 满足 Lipschitz 条件, 格式 (9.7) 产生的序列 $\left\{y_{n+1}^{(k)}\right\}$ 满足

$$\begin{aligned}\left|y_{n+1}^{(k+1)}-y_{n+1}^{(k)}\right| &= \frac{h}{2}\left|f(x_{n+1},y_{n+1}^{(k)})-f(x_{n+1},y_{n+1}^{(k-1)})\right| \\ &\leqslant \cdots \leqslant \left(\frac{hL}{2}\right)^{(k)}\left|y_{n+1}^{(1)}-y_{n+1}^{(0)}\right|\end{aligned}$$

因此, 当 $\dfrac{hL}{2}<1$ 时, 迭代格式 (9.7) 收敛.

若计算的精度要求不是太高, 迭代公式每步仅迭代一次, 从而导出一种新的方法——改进的 Euler 方法.

9.1.4　改进 Euler 法

按式 (9.7) 计算问题 (9.1) 的数值解时, 如果每步只迭代一次, 相当于将 Euler 公式与梯形公式结合使用: 先用 Euler 公式求 y_{n+1} 的一个初步近似值 $\bar{y}_{n+1}$, 称为预测值, 然后用梯形公式校正求得近似值 y_{n+1}, 即

$$\begin{cases}\bar{y}_{n+1}=y_n+hf(x_n,y_n) & \text{预测}\\ y_{n+1}=y_n+\dfrac{h}{2}\left[f(x_n,y_n)+f\left(x_{n+1},\bar{y}_{n+1}\right)\right] & \text{校正}\end{cases} \tag{9.8}$$

式 (9.8) 称为由 Euler 公式和梯形公式得到的预测–校正系统, 也叫改进 Euler 法, 它是显式单步法.

算法 9.1　改进 Euler 方法

1. 输入 $a,b,f(x,y)$, 整数 N, 初值 y_0.
2. 置 $h=\dfrac{b-a}{N},n=0,x=a,y=y_0$

   输出 (x,y).
3. 计算

$$\begin{aligned}y_p&=y+hf(x,y)\\ y_q&=y+hf\left(x+h,y_p\right)\end{aligned}$$

   置 $\dfrac{1}{2}\left(y_p+y_q\right)\Rightarrow y, x+h\Rightarrow x$

   输出 (x,y).
4. 若 $n<N-1$, 则置 $n+1\Rightarrow n$, 转 3 ; 否则停机.

为便于编制程序上机, 式 (9.8) 常改写成

$$\begin{cases}y_p=y_n+hf(x_n,y_n)\\ y_q=y_n+hf\left(x_n+h,y_p\right)\\ y_{n+1}=\dfrac{1}{2}\left(y_p+y_q\right)\end{cases} \tag{9.9}$$

扫描全能王 创建

例 9.2 用改进 Euler 法求初值问题

$$\begin{cases} y' = x - y, & 0 \leqslant x \leqslant 1 \\ y(0) = 0 \end{cases}$$

的数值解, 取 $h = 0.1, N = 10$.

解 按式 (9.9)

$$y_p = y_n + hf(x_n, y_n) = y_n + 0.1(x_n - y_n)$$

$$y_q = y_n + 0.1\,(x_n + 0.1 - y_p)$$

$$y_{n+1} = \frac{1}{2}\,(y_p + y_q)$$

数值结果见表 9.2.

表 9.2 改进 Euler 算法的数值结果

x_n	y_n	$y(x_n)$	$y(x_n) - y_n$
0	0.000 000	0.000 000	0.000 000
0.1	0.005 000	0.004 837	−0.000 163
0.2	0.019 025	0.018 731	−0.000 294
0.3	0.041 218	0.040 818	−0.000 400
0.4	0.070 802	0.070 320	−0.000 482
0.5	0.107 076	0.106 531	−0.000 545
0.6	0.149 404	0.148 812	−0.000 592
0.7	0.197 211	0.196 585	−0.000 626
0.8	0.249 976	0.249 329	−0.000 647
0.9	0.307 228	0.306 570	−0.000 658
1.0	0.368 541	0.367 879	−0.000 662

与例 9.1 的计算结果比较, 改进 Euler 法的精度明显高于 Euler 法.

9.1.5 单步法的误差估计

前面介绍的几种方法, 在计算 y_{n+1} 时都只用到前一步的值 y_n, 这类方法统称为单步法, 单步法的一般形式为

$$y_{n+1} = y_n + h\varphi\,(x_n, y_n, x_{n+1}, y_{n+1}, h) \tag{9.10}$$

其中函数 φ 与微分方程的右端项 $f(x, y)$ 有关, 若 φ 中不含 y_{n+1}, 则方法为显式的, 否则为隐式的. 为便于误差分析, 先考虑计算一步所产生的误差.

定义 9.1 设 $y(x)$ 是问题 (9.1) 的精确解, 则

$$R_{n+1} = y(x_{n+1}) - y(x_n) - h\varphi(x_n, y(x_n), x_{n+1}, y(x_{n+1}), h) \tag{9.11}$$

称为单步法 (9.10) 的局部截断误差, 简称截断误差.

扫描全能王 创建

局部截断误差通常由 Taylor 展开得到, 如对 Euler 法有

$$R_{n+1} = y(x_{n+1}) - y(x_n) - hf(x_n, y(x_n)) = y(x_n + h) - y(x_n) - hy'(x_n)$$
$$= \frac{1}{2}h^2 y''(\xi) \tag{9.12}$$

由误差表示的不同形式, Euler 法的局部截断误差也可表示为

$$R_{n+1} = \frac{1}{2}h^2 y''(x_n) + O(h^3) = O(h^2)$$

实际计算时, y_n 是 $y(x_n)$ 的近似值, 因此, 计算过程中除每步所产生的局部截断误差外, 还有因前面的计算不准确而引起的误差.

定义 9.2　*在不考虑舍入误差的情况下, 问题 (9.1) 的精确解 $y(x_{n+1})$ 与数值解 y_{n+1} 之差称为整体截断误差, 记为*

$$e_{n+1} = y(x_{n+1}) - y_{n+1}$$

下面讨论 Euler 方法的整体截断误差.

为简便起见, 假定函数 $f(x, y)$ 充分光滑, 问题 (9.1) 的解 $y(x)$ 在 $[a, b]$ 上二阶连续可微, 于是由式 (9.12), 局部截断误差有界, 即存在 $M > 0$, 使得对任意 $x \in [a, b]$, 都有 $|y''(x)| \leqslant M$, 从而有

$$|R_{n+1}| = |y(x_{n+1}) - [y(x_n) + hf(x_n, y(x_n))]|$$
$$= \frac{h^2}{2}|y''(\xi)| \leqslant \frac{h^2}{2}M \quad (n = 0, 1, 2, \cdots)$$

记 $\bar{y}_{n+1} = y(x_n) + hf(x_n, y(x_n))$, 于是

$$|e_{n+1}| = |y(x_{n+1}) - y_{n+1}|$$
$$\leqslant |y(x_{n+1}) - \bar{y}_{n+1}| + |\bar{y}_{n+1} - y_{n+1}|$$
$$= |R_{n+1}| + |y(x_n) + hf(x_n, y(x_n)) - y_n - hf(x_n, y_n)|$$
$$\leqslant |R_{n+1}| + |y(x_n) - y_n| + h\,|f(x_n, y(x_n)) - f(x_n, y_n)|$$
$$\leqslant |R_{n+1}| + (1 + hL)\,|y(x_n) - y_n|$$
$$\leqslant \frac{h^2}{2}M + (1 + hL)\,|e_n|$$

反复递推得

$$|e_{n+1}| \leqslant \frac{h^2}{2}M + (1 + hL)\left[\frac{h^2 M}{2} + (1 + hL)\,|e_{n-1}|\right]$$
$$\leqslant \frac{h^2}{2}M \sum_{k=0}^{n}(1 + hL)^k$$
$$= \frac{h^2 M}{2}\frac{(1 + hL)^{n+1} - 1}{(1 + hL) - 1}$$

扫描全能王 创建

$$= \frac{hM}{2L}\left[(1+hL)^{n+1}-1\right]$$

因为 $(n+1)h \leqslant b-a$, 从而有

$$(1+hL)^{n+1} \leqslant (1+hL)^{\frac{b-a}{h}} < \mathrm{e}^{L(b-a)}$$

所以

$$|e_{n+1}| \leqslant \frac{hM}{2L}\left[\mathrm{e}^{L(b-a)}-1\right] \tag{9.13}$$

式 (9.8) 表明, Euler 方法的整体截断误差与 h 同阶. 当 $h \to 0$ 时, $e_N \to 0$.

定义 9.3 *若单步法 (9.10) 的局部截断误差为 $O(h^{p+1})$, 则称该方法是 p 阶的.*

p 的值越大, 方法 (9.10) 的精度越高. 估计式 (9.12) 说明, Euler 方法是一阶的, 它的精度不高.

梯形公式的局部截断误差为

$$\begin{aligned}
R_{n+1} =& y(x_{n+1}) - \left\{y(x_n) + \frac{h}{2}\left[f(x_n, y(x_n)) + f\left(x_{n+1}, y(x_{n+1})\right)\right]\right\} \\
=& \left[y(x_n) + hy'(x_n) + \frac{1}{2}h^2y''(x_n) + \frac{1}{3!}h^3y'''(x_n) + O\left(h^4\right)\right] - \\
& \left\{y(x_n) + \frac{h}{2}\left[y'(x_n) + y'(x_n) + hy''(x_n) + \frac{1}{2}h^2y'''(x_n) + O(h^3)\right]\right\} \\
=& -\frac{h^3}{12}y'''(x_n) + O\left(h^4\right)
\end{aligned}$$

故梯形公式是二阶方法. 类似地可以证明, 向后 Euler 方法是一阶方法, 改进的 Euler 方法是二阶方法.

9.2 Runge-Kutta 法

9.2.1 Runge-Kutta 法的基本思想

回顾 Euler 方法的误差估计, 用一阶 Taylor 多项式近似函数得到 Euler 公式, 故其局部截断误差为一阶 Taylor 余项 $O(h^2)$. 完全类似地, 若用 p 阶 Taylor 多项式近似函数, 即

$$y_{n+1} = y(x_n) + hy'(x_n) + \frac{h^2}{2!}y''(x_n) + \cdots + \frac{h^p}{p!}y^{(p)}(x_n)$$

其中 $y'(x) = f(x,y), y''(x) = f'_x(x,y) + f'_y(x,y)f(x,y), \cdots$, 则局部截断误差应为 p 阶 Taylor 余项 $O(h^{p+1})$. 由此得到启示: 可以通过提高 Taylor 多项式的次数来提高算法的阶数, 以得到高精度的数值方法. 但若直接对 $y(x)$ 用高次 Taylor 多项式近似, 则因公式中出现 f 的各阶偏导数导致计算太复杂, 计算量大而不实用. 如果将 Euler 公式与改进 Euler 公式分别写成以下形式

扫描全能王 创建

Euler 公式

$$\begin{cases} y_{n+1} = y_n + hK_1 \\ K_1 = f(x_n, y_n) \end{cases}$$

改进 Euler 公式

$$\begin{cases} y_{n+1} = y_n + h\left(\dfrac{1}{2}K_1 + \dfrac{1}{2}K_2\right) \\ K_1 = f(x_n, y_n) \\ K_2 = f\left(x_n + h, y_n + hK_1\right) \end{cases}$$

这两组公式都是用函数 $f(x,y)$ 在某些点上的值的线性组合来计算 $y(x_{n+1})$ 的近似值 y_{n+1}. Euler 公式每步计算一次 $f(x,y)$ 的值, 它是 $y(x_{n+1})$ 在 x_n 处的一阶 Taylor 多项式, 因而是一阶方法. 改进 Euler 公式每步需计算二次 $f(x,y)$ 的值, 它在 (x_n, y_n) 处的 Taylor 展开式与 $y(x_{n+1})$ 在 x_n 处的 Taylor 展开式的前三项完全相同, 故是二阶方法. 于是可以考虑用函数 $f(x,y)$ 在若干点上的函数值的线性组合来构造近似公式, 构造时要求近似公式在 (x_n, y_n) 处的 Taylor 展开式与解 $y(x)$ 在 x_n 处的 Taylor 展开式的前面几项重合, 从而使近似公式达到所需要的阶数. 这样既避免了计算函数 $f(x,y)$ 的偏导数, 又提高了方法的精度, 这就是 Runge-Kutta(龙格－库塔) 方法 (简称 RK 方法) 的基本思想.

9.2.2　RK 方法的构造

一般地, RK 方法设近似公式为

$$\begin{cases} y_{n+1} = y_n + h\sum\limits_{i=1}^{p} c_i K_i \\ K_1 = f(x_n, y_n) \\ K_i = f\left(x_n + a_i h, y_n + h\sum\limits_{j=1}^{i-1} b_{ij} K_j\right) \quad (i = 2, 3, \cdots, p) \end{cases} \tag{9.14}$$

其中 a_i, b_{ij}, c_i 都是参数, 确定它们的原则是使近似公式在 (x_n, y_n) 处的 Taylor 展开式与 $y(x)$ 在 x_n 处的 Taylor 展开式的前面的项尽可能多地重合, 这样就使近似公式有尽可能高的精度. 以 $p = 2$ 为例, 近似公式为

$$\begin{cases} y_{n+1} = y_n + h\left(c_1 K_1 + c_2 K_2\right) \\ K_1 = f(x_n, y_n) \\ K_2 = f\left(x_n + a_2 h, y_n + h b_{21} K_1\right) \end{cases} \tag{9.15}$$

式 (9.15) 在 (x_n, y_n) 处的 Taylor 展开式为

$$\begin{aligned} y_{n+1} =& y_n + h\left[c_1 f(x_n, y_n) + c_2 f\left(x_n + a_2 h, y_n + h b_{21} f(x_n, y_n)\right)\right] \\ =& y_n + h\left\{c_1 f(x_n, y_n) + c_2\left[f(x_n, y_n) + a_2 h f'_x(x_n, y_n) + b_{21} h f'_y(x_n, y_n) f(x_n, y_n)\right]\right\} + O(h^3) \\ =& y_n + (c_1 + c_2) f(x_n, y_n) h + c_2\left[a_2 f'_x(x_n, y_n) + b_{21} f'_y(x_n, y_n) f(x_n, y_n)\right] h^2 + O(h^3) \end{aligned} \tag{9.16}$$

扫描全能王 创建

$y(x_{n+1})$ 在 x_n 处的 Taylor 展开式为

$$\begin{aligned} y(x_{n+1}) =& y(x_n) + hy'(x_n) + \frac{h^2}{2}y''(x_n) + O(h^3) \\ =& y_n + f(x_n, y_n)h + \frac{h^2}{2}\left[f'_x(x_n, y_n) + f'_y(x_n, y_n)f(x_n, y_n)\right] + O(h^3) \end{aligned} \tag{9.17}$$

要使近似公式 (9.15) 的局部截断误差为 $O(h^3)$, 则应要求式 (9.16) 和式 (9.17) 的前三项相同, 于是有

$$\begin{cases} c_1 + c_2 = 1 \\ c_2 a_2 = \dfrac{1}{2} \\ c_2 b_{21} = \dfrac{1}{2} \end{cases} \tag{9.18}$$

式 (9.18) 有无穷多组解, 它的每一组解代入式 (9.15) 得到的近似公式, 局部截断误差均为 $O(h^3)$, 故这些方法统称为二阶方法. 例如, 取 $c_1 = c_2 = \dfrac{1}{2}, a_2 = b_{21} = 1$, 近似公式为

$$\begin{cases} y_{n+1} = y_n + \dfrac{h}{2}(K_1 + K_2) \\ K_1 = f(x_n, y_n) \\ K_2 = f(x_n + h, y_n + hK_1) \end{cases}$$

这就是改进 Euler 公式. 如果取 $c_1 = 0, c_2 = 1, a_2 = b_{21} = \dfrac{1}{2}$, 则得

$$\begin{cases} y_{n+1} = y_n + hK_2 \\ K_1 = f(x_n, y_n) \\ K_2 = f\left(x_n + \dfrac{h}{2}, y_n + \dfrac{h}{2}K_1\right) \end{cases}$$

这也是常用的二阶公式, 称为中点公式.

理论上可以证明, 无论怎样选取参数, 式 (9.15) 都不可能具有更高的精度. 因此, 每步计算两个函数值, 只能得出二阶公式.

类似地, 对 $p = 3$ 和 $p = 4$ 的情形, 通过更复杂的计算, 可以导出三阶和四阶 RK 公式, 其中常用的三阶和四阶 RK 公式为

$$\begin{cases} y_{n+1} = y_n + \dfrac{h}{6}(K_1 + 4K_2 + K_3) \\ K_1 = f(x_n, y_n) \\ K_2 = f\left(x_n + \dfrac{h}{2}, y_n + \dfrac{h}{2}K_1\right) \\ K_3 = f(x_n + h, y_n - hK_1 + 2hK_2) \end{cases}$$

和

扫描全能王 创建

$$\begin{cases} y_{n+1}=y_n+\dfrac{h}{6}(K_1+2K_2+2K_3+K_4) \\ K_1=f(x_n,y_n) \\ K_2=f\left(x_n+\dfrac{h}{2},y_n+\dfrac{h}{2}K_1\right) \\ K_3=f\left(x_n+\dfrac{h}{2},y_n+\dfrac{h}{2}K_2\right) \\ K_4=f\left(x_n+h,y_n+hK_3\right) \end{cases} \tag{9.19}$$

式 (9.19) 称为经典形式的四阶 RK 公式, 通常说四阶 RK 方法就是指用式 (9.19) 求解.

算法 9.2　四阶 RK 方法

1. 输入 $a,b,f(x,y),N,y_0$.
2. 置 $h=\dfrac{b-a}{N},n=1,x_0=a$.
3. 计算
   $x=x_0+h$
   $K_1=f(x_0,y_0)$
   $K_2=f\left(x_0+\dfrac{h}{2},y_0+\dfrac{h}{2}K_1\right)$
   $K_3=f\left(x_0+\dfrac{h}{2},y_0+\dfrac{h}{2}K_2\right)$
   $K_4=f\left(x_0+h,y_0+hK_3\right)$
   $y=y_0+\dfrac{h}{6}\left[K_1+2\left(K_2+K_3\right)+K_4\right]$
   输出 (x,y).
4. 若 $n<N$, 则置 $n+1\Rightarrow n,x\Rightarrow x_0,y\Rightarrow y_0$, 转 3; 否则停机.

例 9.3　用四阶 RK 方法求初值问题

$$\begin{cases} y'=x-y, & 0\leqslant x\leqslant 1 \\ y(0)=0 \end{cases}$$

的数值解, 取 $h=0.1,n=10$.

解　公式 (9.19) 在此题中的具体形式为

$$\begin{cases} y_{n+1}=y_n+\dfrac{0.1}{6}(K_1+2K_2+2K_3+K_4) \\ K_1=x_n-y_n \\ K_2=x_n+\dfrac{h}{2}-\left(y_n+\dfrac{h}{2}K_1\right)=0.95(x_n-y_n)+0.05 \\ K_3=x_n+\dfrac{h}{2}-\left(y_n+\dfrac{h}{2}K_2\right)=0.952\,5(x_n-y_n)+0.047\,5 \\ K_4=x_n+h-(y_n+hK_3)=0.904\,75(x_n-y_n)+0.095\,25 \end{cases}$$

扫描全能王 创建

计算结果见表 9.3.

表 9.3 四阶 RK 方法数值结果

x_n	y_n	$y(x_n)$	$y(x_n)-y_n$
0	0	0	0
0.1	0.004 837 500 000 00	0.004 837 418 035 96	-0.82×10^{-7}
0.2	0.018 730 901 406 25	0.018 730 753 077 98	-0.15×10^{-6}
0.3	0.040 818 422 001 18	0.040 818 220 681 72	-0.20×10^{-6}
0.4	0.070 320 288 917 49	0.070 320 046 035 64	-0.24×10^{-6}
0.5	0.106 530 934 423 38	0.106 530 659 712 63	-0.27×10^{-6}
0.6	0.148 811 934 376 32	0.148 811 636 094 03	-0.30×10^{-6}
0.7	0.196 585 618 671 23	0.196 585 303 791 41	-0.31×10^{-6}
0.8	0.249 329 289 734 43	0.249 328 964 117 22	-0.33×10^{-6}
0.9	0.306 569 991 200 08	0.306 569 659 740 60	-0.33×10^{-6}
1.0	0.367 879 774 412 50	0.367 879 441 171 44	-0.33×10^{-6}

与例 9.1、例 9.2 的计算结果相比, 显然四阶 RK 方法的精度高得多.

需要说明的是, 当 $p=1,2,3,4$ 时, RK 公式的最高阶数恰好是 p, 当 $p>4$ 时, RK 公式的最高阶数不是 p, 如 $p=5$ 时, RK 公式的最高阶数仍为 $4, p=6$ 时, RK 公式的最高阶数为 5. RK 方法的导出基于 Taylor 展开, 故它要求所求问题的解具有较高的光滑度. 当解充分光滑时, 四阶 RK 方法确实优于改进 Euler 法. 对大量实际问题来说, 四阶 RK 一般可以达到精度要求. 如果解的光滑性差, 则用四阶 RK 方法求问题(9.1) 的数值解的效果可能不如改进 Euler 法. 因此, 实际计算时, 需根据问题的具体情况来选择合适的算法.

9.2.3 变步长的 RK 方法

由于问题 (9.1) 的解函数 $y(x)$ 的变化是不均匀的, 如果用等步长求数值解, 则可能产生有些点处精度过高, 有些点处精度过低的情况, 为保证一定的精度, 必须取较小的步长. 但这样既增加了计算量, 也导致误差的增加, 因而实际计算时, 往往采用事后估计误差、自动调整步长的 RK 方法.

设用 p 阶方法计算 y_{n+1}, 记从 y_n 出发, 以步长 h 计算一步所得 $y(x_{n+1})$ 的近似值为 $y_{n+1}^{(h)}$, 以步长 $\dfrac{h}{2}$ 计算两步得 $y(x_{n+1})$ 的近似值为 $y_{n+1}^{\left(\frac{h}{2}\right)}$. 由于 p 阶公式的局部截断误差为 $O(h^{p+1})$, 且 $y^{(p+1)}(x)$ 在小区间 $[x_n, x_{n+1}]$ 上变化不大, 故有

$$y(x_{n+1})-y_{n+1}^{(h)}\approx ch^{p+1} \tag{9.20}$$

$$y(x_{n+1})-y_{n+1}^{\left(\frac{h}{2}\right)}\approx 2c\left(\frac{h}{2}\right)^{p+1} \tag{9.21}$$

将式 (9.21) 乘 2^p 减式 (9.20) 得

$$(2^p-1)\,y(x_{n+1})-2^p y_{n+1}^{\left(\frac{h}{2}\right)}+y_{n+1}^{(h)}\approx 0$$

扫描全能王 创建

从而有

$$\left|y(x_{n+1})-y_{n+1}^{(h)}\right| \approx \frac{2^p}{2^p-1}\left|y_{n+1}^{(h)}-y_{n+1}^{\left(\frac{h}{2}\right)}\right| = \Delta \tag{9.22}$$

这样, 可以事后误差估计式 (9.22) 中 Δ 的大小来选择合适的步长.

变步长的 RK 方法按以下过程进行: 先以 h 为步长从点 x_n 出发求得 $y(x_n+h)$ 的一个近似值 $y_{n+1}^{(h)}$, 再以 $\frac{h}{2}$ 为步长从 x_n 出发计算两步求得 $y(x_n+h)$ 的又一个近似值 $y_{n+1}^{\left(\frac{h}{2}\right)}$. 如果

$$\Delta = \frac{2^p}{2^p-1}\times\left|y_{n+1}^{(h)}-y_{n+1}^{\left(\frac{h}{2}\right)}\right| < \varepsilon$$

且相差不多, 这里 ε 为事先给定的精度, 则取 $y(x_{n+1}) \approx y_{n+1} = y_{n+1}^{(h)}$, 并以 h 为步长计算 x_{n+2} 处的近似值 y_{n+2}; 如果 $\Delta \ll \varepsilon$, 则表明步长过小, 应反复将步长加倍进行计算, 直到 $\Delta > \varepsilon$ 为止. 此时前一次的计算结果为 y_{n+1}, 步长即为合适步长, 可以此步长继续计算; 如果 $\Delta > \varepsilon$, 则表明步长过大, 应将步长减半再计算, 直到满足 $\Delta < \varepsilon$ 为止.

9.3　线性多步法

以上所介绍的各种数值解法都是单步法, 单步法在计算 y_{n+1} 时, 只用到前一步的信息 y_n.

为了提高精度, 需重新计算多个点处的函数值, 如 RK 方法, 故计算量较大. 如何通过较多地利用前面的已知信息, 如 $y_n, y_{n-1}, \cdots, y_{n-r}$, 来构造高精度的算法计算 y_{n+1}, 这就是多步法的基本思想. 多步法中最常用的是线性多步法, 它的计算公式中只出现 $y_{n+1}, y_n, \cdots, y_{n-r}$ 及 $f(x_{n+1}, y_{n+1}), \cdots, f(x_{n-r}, y_{n-r})$ 的一次项, 其一般形式为

$$y_{n+1} = \sum_{i=0}^{r}\alpha_i y_{n-i} + h\sum_{i=-1}^{r}\beta_i f_{n-i} \tag{9.23}$$

其中 α_i, β_i 均为常数, $f_k = f(x_k, y_k)\,(k = n+1, n, \cdots, n-r)$. 若 $\beta_{-1} = 0$, 公式 (9.23) 为显式, $\beta_{-1} \neq 0$, 公式为隐式. 构造线性多步公式常用 Taylor 展开和数值积分的方法.

9.3.1　线性多步公式的导出

利用 Taylor 展开导出线性多步公式的基本方法是: 将线性多步公式 (9.23) 在 x_n 处进行 Taylor 展开, 然后与 $y(x_{n+1})$ 在 x_n 处的 Taylor 展开式相比较, 要求它们前面的项重合, 由此确定参数 α_i, β_i. 下面以 $r = 1$ 为例说明.

设初值问题 (9.1) 的解 $y(x)$ 充分光滑, 待定的线性两步公式为

$$y_{n+1} = \alpha_0 y_n + \alpha_1 y_{n-1} + h\left(\beta_{-1} f_{n+1} + \beta_0 f_n + \beta_1 f_{n-1}\right) \tag{9.24}$$

记 $y_n^{(k)} = y^{(k)}(x_n)(k = 1, 2, \cdots)$, 则 $y(x)$ 在 x_n 处的 Taylor 展开式为

扫描全能王 创建

$$y(x)=y_n+y_n'(x-x_n)+\frac{y_n''}{2}(x-x_n)^2+\cdots+\frac{y_n^{(p)}}{p!}(x-x_n)^p+O\left((x-x_n)^{p+1}\right) \tag{9.25}$$

假设前 n 步计算结果都是准确的, 即 $y_i=y(x_i), y'(x_i)=f(x_i,y_i)(i\leqslant n)$, 则

$$y_{n-1}=y(x_n-h)=y_n-y_n'h+\frac{y_n''}{2!}h^2-\frac{y_n'''}{3!}h^3+\frac{y_n^{(4)}}{4!}h^4-\frac{y_n^{(5)}}{5!}h^5+O(h^6)$$

$$f_{n-1}=f\left(x_{n-1},y_{n-1}\right)=y'(x_{n-1})=y_n'-y_n''h+\frac{y_n'''}{2!}h^2-\frac{y_n^{(4)}}{3!}h^3+\frac{y_n^{(5)}}{4!}h^4+O(h^5)$$

$$f_n=f(x_n,y_n)=y_n'$$

$$f_{n+1}=f(x_{n+1},y_{n+1})\approx y'(x_{n+1})=y_n'+y_n''h+\frac{y_n'''}{2!}h^2+\frac{y_n^{(4)}}{3!}h^3+\frac{y_n^{(5)}}{4!}h^4+O(h^5)$$

将以上各式代入式 (9.24) 并整理, 得

$$\begin{aligned}y_{n+1}=&(\alpha_0+\alpha_1)y_n+(-\alpha_1+\beta_{-1}+\beta_0+\beta_1)y_n'h+\left(\frac{\alpha_1}{2}+\beta_{-1}-\beta_1\right)y_n''h^2+\\&\left(-\frac{\alpha_1}{6}+\frac{\beta_{-1}}{2}+\frac{\beta_1}{2}\right)y_n'''h^3+\left(\frac{\alpha_1}{24}+\frac{\beta_{-1}}{6}-\frac{\beta_1}{6}\right)y_n^{(4)}h^4+\\&\left(-\frac{\alpha_1}{120}+\frac{\beta_{-1}}{24}+\frac{\beta_1}{24}\right)y_n^{(5)}h^5+O(h^6)\end{aligned} \tag{9.26}$$

为使公式 (9.24) 有 p 阶精度, 只需使式 (9.26) 与 $y(x_{n+1})$ 在 x_n 处的 Taylor 展开式

$$y(x_{n+1})=y_n+y_n'h+\frac{y_n''}{2!}h^2+\cdots+\frac{y_n^{(5)}}{5!}h^5+O(h^6) \tag{9.27}$$

的前 $p+1$ 项重合, 因为式 (9.26) 中有 5 个待定系数, 对比 Taylor 展开式的系数可得方程组

$$\begin{cases}\alpha_0+\alpha_1=1\\-\alpha_1+\beta_{-1}+\beta_0+\beta_1=1\\\dfrac{1}{2}\alpha_1+\beta_{-1}-\beta_1=\dfrac{1}{2}\\-\dfrac{1}{6}\alpha_1+\dfrac{1}{2}\beta_{-1}+\dfrac{1}{2}\beta_1=\dfrac{1}{6}\\\dfrac{1}{24}\alpha_1+\dfrac{1}{6}\beta_{-1}-\dfrac{1}{6}\beta_1=\dfrac{1}{24}\end{cases} \tag{9.28}$$

由推导过程知, 如果选取参数 α_i,β_i, 使其满足式 (9.28) 中前 $p+1$ 个方程 ($p=1,2,3,4$), 则近似公式 (9.24) 为 p 阶公式. 如 $\alpha_0=1$, $\alpha_1=0$, $\beta_{-1}=\beta_0=\dfrac{1}{2}$, $\beta_1=0$ 满足方程组 (9.28) 的前三个方程, 故公式

$$y_{n+1}=y_n+\frac{h}{2}\left(f_{n+1}+f_n\right)$$

扫描全能王 创建

为二阶公式. 解方程组 (9.28) 得

$$\alpha_0 = 0, \quad \alpha_1 = 1, \quad \beta_{-1} = \beta_1 = \frac{1}{3}, \quad \beta_0 = \frac{4}{3}$$

相应的线性二步公式为

$$y_{n+1} = y_{n-1} + \frac{h}{3}\left(f_{n+1} + 4f_n + f_{n-1}\right) \tag{9.29}$$

式 (9.29) 称为 Simpson 公式, 它是四阶公式, 局部截断误差为

$$R_{n+1} = -\frac{1}{90}h^5 y_n^{(5)} + O(h^6)$$

由此可知, 线性二步公式至多是四阶公式.

一般地, 线性多步公式 (9.23) 中有 $2r+3$ 个待定参数, 如果令式 (9.23) 右端在 x_n 处的 Taylor 展开式与 $y(x_{n+1})$ 在 x_n 处的 Taylor 展开式的前 $p+1$ 项系数对应相等, 可得方程组

$$\begin{cases} \sum\limits_{i=0}^{r} \alpha_i = 1 \\ \sum\limits_{i=1}^{r} (-i)^k \alpha_i + k \sum\limits_{i=-1}^{r} (-i)^{k-1} \beta_i = 1 \quad (k = 1, 2, \cdots, p) \end{cases} \tag{9.30}$$

其解所对应的公式 (9.23) 具有 p 阶精度, 局部截断误差为

$$\boldsymbol{R}_{n+1} = \frac{h^{p+1}}{(p+1)!}\left[1 - \sum_{i=1}^{r}(-i)^{p+1}\alpha_i - (p+1)\sum_{i=1}^{r}(-i)^p \beta_i\right] y_n^{(p+1)} + O\left(h^{p+2}\right) \tag{9.31}$$

显然, 线性多步公式 (9.23) 至多可达到 $2r+2$ 阶精度.

9.3.2　常用的线性多步公式

(一) Adams (亚当斯) 公式

取 $r = 3$, 并令 $\alpha_1 = \alpha_2 = \alpha_3 = \beta_{-1} = 0$, 由方程组

$$\begin{cases} \sum\limits_{i=0}^{3} \alpha_i = 1 \\ \sum\limits_{i=1}^{3} (-i)^k \alpha_i + k \sum\limits_{i=-1}^{3} (-i)^{k-1} \beta_i = 1 \quad (k = 1, 2, 3, 4) \end{cases} \tag{9.32}$$

可解得

$$\alpha_0 = 1, \quad \beta_0 = \frac{55}{24}, \quad \beta_1 = -\frac{59}{24}, \quad \beta_2 = \frac{37}{24}, \quad \beta_3 = -\frac{9}{24}$$

相应的线性多步公式为

$$y_{n+1} = y_n + \frac{h}{24}\left(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}\right) \tag{9.33}$$

因为 $\beta_{-1} = 0$, 式 (9.33) 称为 Adams 显式公式, 它是四阶公式, 局部截断误差为

扫描全能王 创建

$$
\begin{aligned}
R_{n+1} &= \frac{h^5}{5!}\left[1-\sum_{i=1}^{3}(-i)^5\alpha_i-5\sum_{i=-1}^{3}(-i)^4\beta_i\right]y_n^{(5)}+O(h^6) \quad (9-34)\\
&= \frac{251}{720}h^5y_n^{(5)}+O(h^6)
\end{aligned}
\tag{9.34}
$$

如果令 $\alpha_1=\alpha_2=\alpha_3=\beta_3=0$, 由方程组 (9.32) 可得解

$$
\alpha_0=1,\quad \beta_{-1}=\frac{9}{24},\quad \beta_0=\frac{19}{24},\quad \beta_1=-\frac{5}{24},\quad \beta_2=\frac{1}{24}
$$

相应的线性多步公式

$$
y_{n+1}=y_n+\frac{h}{24}(9f_{n+1}+19f_n-5f_{n-1}+f_{n-2}) \tag{9.35}
$$

称为四阶 Adams 隐式公式, 其局部截断误差为

$$
R_{n+1}=-\frac{19}{720}h^5y_n^{(5)}+O(h^6) \tag{9.36}
$$

Adams 公式 (9.33) (9.35) 也可以利用数值积分方法得到, 其基本思想是首先将初值问题 (9.1) 化成等价的积分形式

$$
y(x_{n+1})-y(x_n)=\int_{x_n}^{x_{n+1}}f(x,y(x))\mathrm{d}x=\int_{x_n}^{x_{n+1}}F(x)\mathrm{d}x \tag{9.37}
$$

其中 $F(x)=f(x,y(x))$, 然后用过节点 $x_n,x_{n-1},\cdots,x_{n-r}$ 或 $x_{n+1},x_n,\cdots,x_{n-r+1}$ 的 $F(x)$ 的 r 次插值多项式 $\varphi_r(x)$ 代替 $F(x)$ 求积分, 即得 $r+1$ 阶 Adams 显式或隐式公式. 例如, 当 $r=3$ 时, 过节点 $x_n,x_{n-1},x_{n-2},x_{n-3},F(x)$ 的三次插值多项式为

$$
L_3(x)=\sum_{i=0}^{3}l_i(x)F\left(x_{n-i}\right)
$$

其中

$$
l_i(x)=\frac{(x-x_n)(x-x_{n-1})(x-x_{n-2})(x-x_{n-3})}{(x-x_{n-i})\prod\limits_{j=0,j\neq i}^{3}(x_{n-i}-x_{n-j})}\quad (i=0,1,2,3)
$$

将 $L_3(x)$ 代替 $F(x)$ 计算式 (9.37) 右端的积分, 得

$$
\begin{aligned}
y(x_{n+1})-y(x_n) &\approx \int_{x_n}^{x_{n+1}}L_3(x)\mathrm{d}x\\
&= \sum_{i=0}^{3}\left[\int_{x_n}^{x_{n+1}}l_i(x)\mathrm{d}x\right]F\left(x_{n-i}\right)\\
&= F(x_n)\int_{x_n}^{x_{n+1}}\frac{(x-x_{n-1})(x-x_{n-2})(x-x_{n-3})}{6h^3}\,\mathrm{d}x+\\
&\quad F(x_{n-1})\int_{x_n}^{x_{n+1}}\frac{(x-x_n)(x-x_{n-2})(x-x_{n-3})}{-2h^3}\,\mathrm{d}x+
\end{aligned}
$$

扫描全能王 创建

$$F(x_{n-2})\int_{x_n}^{x_{n+1}}\frac{(x-x_n)(x-x_{n-1})(x-x_{n-3})}{2h^3}\,\mathrm{d}x+$$
$$F(x_{n-3})\int_{x_n}^{x_{n+1}}\frac{(x-x_n)(x-x_{n-1})(x-x_{n-2})}{-6h^3}\,\mathrm{d}x$$
$$=\frac{h}{24}\left[55F(x_n)-59F(x_{n-1})+37F(x_{n-2})-9F(x_{n-3})\right]$$

用 y_n, y_{n+1} 代替 $y(x_n), y(x_{n+1}), f_k=f(x_k,y_k)$ 代替 $F(x_k)=f(x_k,y(x_k))(k=n, n-1, n-2, n-3)$, 则得

$$y_{n+1}=y_n+\frac{h}{24}(55f_n-59f_{n-1}+37f_{n-2}-9f_{n-3})$$

这就是四阶 Adams 显式公式. 由于积分区间在插值区间 $[x_{n-3}, x_n]$ 外面, 故又称为四阶 Adams 外插公式. 由插值余项公式可得其局部截断误差为

$$R_{n+1}=\int_{x_n}^{x_{n+1}}\frac{F^{(4)}(\xi_x)}{4!}\prod_{j=0}^{3}(x-x_{n-j})\,\mathrm{d}x=\int_{x_n}^{x_{n+1}}\frac{y^{(5)}(\xi_x)}{4!}\prod_{j=0}^{3}(x-x_{n-j})\,\mathrm{d}x$$

由积分中值定理, 存在 $\eta\in(x_{n-3}, x_{n+1})$, 使得

$$R_{n+1}=\frac{y^{(5)}(\eta)}{4!}\int_{x_n}^{x_{n+1}}\prod_{j=0}^{3}(x-x_{n-j})\,\mathrm{d}x=\frac{251}{720}h^5y^{(5)}(\eta)$$

如果以过节点 $x_{n+1}, x_n, x_{n-1}, x_{n-2}$ 的 $F(x)$ 的三次插值多项式

$$L_3(x)=\sum_{i=-1}^{2}l_i(x)F(x_{n-i})$$

其中

$$l_i(x)=\frac{\prod\limits_{j=-1}^{2}(x-x_{n-j})}{(x-x_{n-i})\prod\limits_{\substack{j=-1\\ j\neq i}}^{2}(x_{n-i}-x_{n-j})}\quad(i=-1,0,1,2)$$

代替 $F(x)$ 计算式 (9.37) 右端的积分, 即得四阶 Adams 隐式公式

$$y_{n+1}=y_n+\frac{h}{24}\quad(9f_{n+1}+19f_n-5f_{n-1}+f_{n-2})$$

其局部截断误差为

$$R_{n+1}=-\frac{19}{720}h^5y^{(5)}(\eta),\quad x_{n-2}<\eta<x_{n+1}$$

由于积分区间在插值区间 $[x_{n-2}, x_{n+1}]$ 内, 故 Adams 隐式公式又称为 Adams 内插公式.

(二) Milne (米尔恩) 公式

取 $r=3$, 并令 $\alpha_0=\alpha_1=\alpha_2=\beta_{-1}=0$, 由方程组 (9.31) 可解出 $\alpha_3=1$, $\beta_0=\frac{8}{3}$,

扫描全能王 创建

$\beta_1 = -\frac{4}{3}$, $\beta_2 = \frac{8}{3}$, $\beta_3 = 0$, 相应的线性多步公式

$$y_{n+1} = y_{n-3} + \frac{4}{3}h(2f_n - f_{n-1} + 2f_{n-2}) \tag{9.38}$$

称为 Milne 公式, 其局部截断误差为

$$R_{n+1} = \frac{14}{45}h^5 y_n^{(5)} + O(h^6) \tag{9.39}$$

Milne 公式是四阶四步显式公式.

(三) Hamming (汉明) 公式

取 $r = 2$, 并令 $\alpha_1 = \beta_2 = 0$, 可得到 Hamming 公式

$$y_{n+1} = \frac{1}{8}(9y_n - y_{n-2}) + \frac{3}{8}h\left(f_{n+1} + 2f_n - f_{n-1}\right) \tag{9.40}$$

其局部截断误差为

$$R_{n+1} = -\frac{1}{40}h^5 y_n^{(5)} + O(h^6) \tag{9.41}$$

Hamming 公式是四阶三步隐式公式.

利用线性多步法求解初值问题 (9.1) 时, 必须先用其他方法算出开始几个点的近似值. 一般可用同阶的单步法.

例 9.4　分别用四阶 Adams 显式和隐式公式求初值问题

$$\begin{cases} y' = x - y, \quad 0 \leqslant x \leqslant 1 \\ y(0) = 0 \end{cases}$$

的数值解, 取 $h = 0.1$.

解　据题意, $x_n = nh = 0.1n$, $f_n = x_n - y_n$, 由四阶 Adams 显式公式 (9.33) 有

$$\begin{aligned} y_{n+1} &= y_n + \frac{h}{24}(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}) \\ &= \frac{1}{24}(18.5y_n + 5.9y_{n-1} - 3.7y_{n-2} + 0.9y_{n-3} + 0.24n + 0.12) \quad (n = 3, 4, \cdots, 9) \end{aligned}$$

由四阶 Adams 隐式公式 (9.35) 有

$$\begin{aligned} y_{n+1} &= y_n + \frac{h}{24}(9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}) \\ &= \frac{1}{24}(-0.9y_{n+1} + 22.1y_n + 0.5y_{n-1} - 0.1y_{n-2} + 0.24n + 0.12) \end{aligned}$$

由上式可解出

$$y_{n+1} = \frac{1}{24.9}(22.1y_n + 0.5y_{n-1} - 0.1y_{n-2} + 0.24n + 0.12) \quad (n = 2, 3, \cdots, 9)$$

利用精确解 $y = \mathrm{e}^{-x} + x - 1$ 求出起步值 (一般可用四阶 RK 公式计算起步值) 后, 按上面的公式计算, 结果见表 9.4.

扫描全能王 创建

表 9.4　Adams 公式数值结果

x_n	Adams 显式法		Adams 隐式法	
	y_n	$\lvert y(x_n)-y_n\rvert$	y_n	$\lvert y(x_n)-y_n\rvert$
0.0	0		0	
0.1	0.004 837 42		0.004 837 42	
0.2	0.018 730 75		0.018 730 75	
0.3	0.040 818 22		0.040 818 01	2.1×10^{-7}
0.4	0.070 322 92	2.87×10^{-6}	0.070 319 66	3.8×10^{-7}
0.5	0.106 535 48	4.82×10^{-6}	0.106 530 14	5.2×10^{-7}
0.6	0.148 818 41	6.77×10^{-6}	0.148 811 01	6.3×10^{-7}
0.7	0.196 593 39	8.09×10^{-6}	0.196 584 59	7.1×10^{-7}
0.8	0.249 338 16	9.19×10^{-6}	0.249 328 19	7.7×10^{-7}
0.9	0.306 579 61	9.95×10^{-6}	0.306 568 85	8.1×10^{-7}
1.0	0.367 889 96	1.052×10^{-6}	0.367 878 60	8.4×10^{-7}

从表 9.4 可以看出, 四阶 Adams 隐式法比显式法的精度高, 比较这两种方法的局部截断误差公式 (9.34) 和式 (9.36), 可以说明这一现象. 一般地, 同阶的隐式法比显式法精确, 而且数值稳定性也好. 但在隐式公式中, 通常很难解出 y_{n+1}, 需要用迭代法求解, 这样又增加了计算量. 因此实际计算时, 很少单独使用显式公式或隐式公式, 而是将它们联合使用: 先用显式公式求出 $y(x_{n+1})$ 的预测值, 记作 $\bar{y}_{n+1}$, 再用隐式公式对预测值进行校正, 求出 $y(x_{n+1})$ 的近似值 y_{n+1}.

9.3.3　预测–校正系统

用显式公式计算预测值, 然后用隐式公式进行校正, 得到近似值 y_{n+1}, 这样一组计算公式称为预测–校正系统. 一般地, 采用同阶的隐式公式与显式公式. 常用的预测–校正系统有两种:

Adams 预测–校正公式

$$\begin{cases} \hat{y}_{n+1}=y_n+\dfrac{h}{24}\left(55f_n-59f_{n-1}+37f_{n-2}-9f_{n-3}\right) & \text{预测} \\ y_{n+1}=y_n+\dfrac{h}{24}\left[9f\left(x_{n+1},\bar{y}_{n+1}\right)+19f_n-5f_{n-1}+f_{n-2}\right] & \text{校正} \end{cases} \tag{9.42}$$

Milne-Hamming 预测–校正公式

$$\begin{cases} \bar{y}_{n+1}=y_{n-3}+\dfrac{4}{3}h(2f_n-f_{n-1}+2f_{n-2}) \\ y_{n+1}=\dfrac{1}{8}(9y_n-y_{n-2})+\dfrac{3}{8}h\left[f\left(x_{n+1},\bar{y}_{n+1}\right)+2f_n-f_{n-1}\right] \end{cases} \tag{9.43}$$

注 9.1　以上两种预测–校正公式均为四阶公式, 其起步值通常用四阶 RK 公式计算; 有时为提高精度, 校正公式可迭代进行多次, 但迭代次数一般不超过 3 次.

为减少一次迭代所产生的误差, 常常用局部截断误差进一步修正预测值与校正值, 得到

扫描全能王 创建

更精确的预测–校正公式. 下面分别对 Adams 预测–校正公式与 Milne-Hamming 公式进行讨论.

由 Adams 公式的局部截断误差公式

$$y(x_{n+1}) - \bar{y}_{n+1} = \frac{251}{720}h^5 y_n^{(5)} + O(h^6)$$

$$y(x_{n+1}) - y_{n+1} = -\frac{19}{720}h^5 y_n^{(5)} + O(h^6)$$

两式相减, 得

$$y_{n+1} - \bar{y}_{n+1} = \frac{270}{720}h^5 y_n^{(5)} + O(h^6)$$

从而有

$$h^5 y_n^{(5)} \approx \frac{720}{270}(y_{n+1} - \bar{y}_{n+1})$$

将上式代入式 (9.34) 和式 (9.35), 得

$$y(x_{n+1}) - \bar{y}_{n+1} \approx \frac{251}{270}(y_{n+1} - \bar{y}_{n+1}) \tag{9.44}$$

$$y(x_{n+1}) - y_{n+1} \approx -\frac{19}{270}(y_{n+1} - \bar{y}_{n+1}) \tag{9.45}$$

用式 (9.44) 和式 (9.45) 修正公式 (9.42), 就得到多环节的 Adams 预测–校正公式

$$\begin{cases} p_{n+1} = y_n + \dfrac{h}{24}(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}) \quad \text{预测} \\ m_{n+1} = p_{n+1} + \dfrac{251}{270}(c_n - p_n) \quad \text{改进} \\ c_{n+1} = y_n + \dfrac{h}{24}[9f(x_{n+1}, m_{n+1}) + 19f_n - 5f_{n-1} + f_{n-2}] \quad \text{校正} \\ y_{n+1} = c_{n+1} - \dfrac{19}{270}(c_{n+1} - p_{n+1}) \quad \text{改进} \end{cases} \tag{9.46}$$

完全类似地, 可以导出多环节的 Milne-Hamming 预测–校正公式 (也称为 Hamming 方法)

$$\begin{cases} p_{n+1} = y_{n-3} + \dfrac{4}{3}h(2f_n - f_{n-1} + 2f_{n-2}) \\ m_{n+1} = p_{n+1} + \dfrac{112}{121}(c_n - p_n) \\ c_{n+1} = \dfrac{1}{8}(9y_n - y_{n-2}) + \dfrac{3}{8}h[f(x_{n+1}, m_{n+1}) + 2f_n - f_{n-1}] \\ y_{n+1} = c_{n+1} - \dfrac{9}{121}(c_{n+1} - p_{n+1}) \end{cases} \tag{9.47}$$

上述预测–校正公式的优点是每算一步只需计算两个函数值, 计算量小于四阶 RK 方法, 而且在计算过程中已大致估计出误差. 它们不足之处在于必须借助于别的方法计算开始几个函数值, 计算过程中不易变步长. 算法 9.3 给出了 Hamming 算法的流程图.

扫描全能王 创建

算法 9.3　Hamming 算法

1. 输入 $a, b, f(x,y), N, y_0$.
2. 置 $h=\dfrac{b-a}{N}, x_0=a, n=1$.
3. 计算

$$
\begin{aligned}
f_{n-1} &= f(x_{n-1}, y_{n-1}) \\
K_1 &= hf_{n-1} \\
K_2 &= hf\left(x_{n-1}+\frac{h}{2}, y_{n-1}+\frac{K_1}{2}\right) \\
K_3 &= hf\left(x_{n-1}+\frac{h}{2}, y_{n-1}+\frac{K_2}{2}\right) \\
K_4 &= hf(x_{n-1}+h, y_{n-1}+K_3) \\
y_n &= y_{n-1}+\frac{1}{6}(K_1+2K_2+2K_3+K_4) \\
x_n &= a+nh
\end{aligned}
$$

4. 输出 (x_n, y_n).
5. 若 $n<3$, 则置 $n+1\Rightarrow n$, 返回 3; 否则, 置 $n+1\Rightarrow n, 0\Rightarrow p_0, 0\Rightarrow c_0$, 转 6 .
6. 计算

$$
\begin{aligned}
f_3 &= f(x_3, y_3) \\
x &= x_3+h \\
p &= y_0+\frac{4}{3}(2f_3-f_2+2f_1) \\
m &= p+\frac{112}{121}(c_0-p_0) \\
c &= \frac{1}{8}(9y_3-y_1)+\frac{3}{8}h[f(x,m)+2f_3-f_2] \\
y &= c-\frac{9}{121}(c-p)
\end{aligned}
$$

输出 (x,y).

7. 若 $n<N'$, 则置 $n+1\Rightarrow n, x_{j+1}\Rightarrow x_j, y_{j+1}\Rightarrow y_j, f_{j+1}\Rightarrow f_j(j=0,1,2), x\Rightarrow x_3, y\Rightarrow y_3, p\Rightarrow p_0, c\Rightarrow c_0$, 转 6 ; 否则停机.

9.4　相容性、收敛性与稳定性

9.4.1　相容性、收敛性

上面所介绍的方法都是用离散化的手段, 将微分方程初值问题化为差分方程初值问题来求解的. 这些转化是否合理? 即当 $h\to 0$ 时, 差分方程是否能无限逼近微分方程, 差分方

扫描全能王 创建

程的解 y_n 是否能无限逼近微分方程初值问题的精确解 $y(x_n)$, 这就是相容性与收敛性问题. 这里仅对显式单步法进行讨论.

用显式单步法

$$y_{n+1} = y_n + h\varphi\left(x_n, y_n, h\right)$$

其中函数 φ 称为增量函数, 求解初值问题 (9.1), 即用差分方程初值问题

$$\begin{cases} y_{n+1} = y_n + h\varphi\left(x_n, y_n, h\right) \\ y\left(x_0\right) = y_0 \end{cases} \tag{9.48}$$

的解作为问题 (9.1) 的近似解. 如果近似是合理的, 则应有

$$\frac{y(x+h) - y(x)}{h} - \varphi(x, y(x), h) \to 0 \quad (h \to 0) \tag{9.49}$$

其中 $y(x)$ 为问题 (9.1) 的解. 因为

$$\lim_{h \to 0} \frac{y(x+h) - y(x)}{h} = y'(x) = f(x, y)$$

故由式 (9.49) 得

$$\lim_{h \to 0} \varphi(x, y, h) = f(x, y)$$

如果增量函数 $\varphi(x, y, h)$ 关于 h 连续, 则有

$$\varphi(x, y, 0) = f(x, y) \tag{9.50}$$

式 (9.50) 称为相容性条件. 如果增量函数 $\varphi(x, y, h)$ 满足条件式 (9.50), 则称显式单步法与初值问题 (9.1) 相容. 显然, 满足相容性条件式 (9.50) 是可以用单步法求解初值问题 (9.1) 的必要条件.

容易验证, Euler 法与改进 Euler 法均满足相容性条件. 事实上, 对 Euler 法, 增量函数为

$$\varphi(x, y, h) = f(x, y)$$

自然满足条件式 (9.50), 改进 Euler 法的增量函数为

$$\varphi(x, y, h) = \frac{1}{2}[f(x, y) + f(x + h, y + hf(x, y))]$$

因为 $f(x, y)$ 连续, 从而有

$$\varphi(x, y, 0) = \frac{1}{2}[f(x, y) + f(x, y)] = f(x, y)$$

所以, Euler 法与改进 Euler 法均与初值问题 (9.1) 相容. 一般地, 如果显式单步法有 p 阶精度 ($p > 0$), 则其局部截断误差为

$$y(x + h) - [y(x) + h\varphi(x, y(x), h)] = O(h^{p+1})$$

扫描全能王 创建

上式两端同除以 h, 得

$$\frac{y(x+h)-y(x)}{h}-\varphi(x,y,h)=O\left(h^p\right)$$

令 $h\to 0$, 如果 $\varphi(x,y,h)$ 连续, 则有

$$y'(x)-\varphi(x,y,0)=0$$

即

$$f(x,y)-\varphi(x,y,0)=0$$

所以 $p>0$ 的显式单步法均与问题 (9.1) 相容. 由此即得各阶 RK 方法与初值问题 (9.1) 相容.

一种数值方法称为是收敛的, 如果对于任意初值 y_0 及任意 $x\in(a,b]$, 都有

$$\lim_{h\to 0} y_n=y(x)\quad (x=a+nh)$$

其中 $y(x)$ 为初值问题 (9.1) 的精确解.

按定义, 数值方法的收敛性需根据该方法的整体截断误差 ε_{n+1} 来判定. 本书 9.1节中已导出, Euler 方法的整体截断误差有估计式

$$|e_{n+1}|\leqslant\frac{hM}{2L}\left|e^{L(b-a)}-1\right|=O(h)$$

当 $h\to 0$ 时, $e_n\to 0$, 故 Euler 方法收敛. 类似地, 可以讨论显式单步法的收敛性.

定理 9.1　*设显式单步法具有 p 阶精度, 其增量函数 $\varphi(x,y,h)$ 关于 y 满足 Lipschitz 条件, 问题 (9.1) 的初值是精确的, 即 $y\left(x_0\right)=y_0$, 则显式单步法的整体截断误差为*

$$e_{n+1}=y(x_{n+1})-y_{n+1}=O(h^p)$$

读者可仿 Euler 法的整体截断误差估计过程自行证明此定理. 提示: 显式单步法具有 p 阶精度, 即其局部截断误差为 $O(h^{p+1})$, 故存在 $M>0$, 使得

$$|R_{n+1}|=|y(x_{n+1})-y(x_n)-h\varphi(x_n,y(x_n),h)|\leqslant Mh^{p+1}$$

推论 9.1　*设显式单步法具有 p 阶精度, 增量函数 $\varphi(x,y,h)$ 在区域*

$$S: a\leqslant x\leqslant b,\quad -\infty<y<+\infty, 0\leqslant h\leqslant h_0$$

内连续, 且关于 y 满足 Lipschitz 条件, 则显式单步法是收敛的.

当 $f(x,y)$ 在区域 $D: a\leqslant x\leqslant b, -\infty<y<+\infty$ 上连续, 且关于 y 满足 Lipschitz 条件时, 改进 Euler 方法, 各阶 RK 方法的增量函数 $\varphi(x,y,h)$ 在区域 S 上连续, 且关于 y 满足 Lipschitz 条件, 因而它们都是收敛的.

关于显式单步法收敛的更一般结果是

定理 9.2　*设增量函数 $\varphi(x,y,h)$ 在区域 S 上连续, 且关于 y 满足 Lipschitz 条件, 则显式单步法收敛的充分必要条件是相容性条件式 (9.50) 成立.*

证明请看参考文献 [9].

扫描全能王 创建

9.4.2 稳定性

在收敛性的讨论中, 我们是在每步计算都是准确的前提下, 估计由近似公式所产生的误差, 没有考虑舍入误差. 实际计算中, 除了由数值方法所产生的截断误差外, 还有因数字舍入而产生的误差. 一种数值方法, 即使它满足相容性条件, 也是收敛的, 但若在计算过程中, 舍入误差的积累越来越大, 那么它的解 y_n 作为问题 (9.1) 的近似解也可能严重失真. 影响舍入误差的因素很多, 这里我们只关心它在传播过程中的增长情况, 这就是数值方法的稳定性问题. 具体地说, 如果用某种数值方法求解初值问题, 当步长 h 固定时, 在节点值 y_n 上产生误差 (扰动) δ, 而由此引起节点值 y_m $(m>n)$ 的误差 δ_m 的绝对值均不超过 $|\delta|$, 则称此方法是绝对稳定的. 稳定性问题比较复杂, 而上面的定义又依赖于微分方程本身, 实际讨论时, 均就下述模型方程进行

$$y'=\lambda y \tag{9.51}$$

其中 λ 为复常数.

定义 9.4 用某种数值方法求解模型方程 (9.51), 对固定的步长 h, 如果由计算节点值 y_n 时产生的误差 δ 所引起后面节点值 y_m $(m>n)$ 的误差 δ_m 的绝对值均不超过 $|\delta|$, 则称该方法对于所用步长 h 和复数 λ 是绝对稳定的. 使得方法绝对稳定的 h 和 λ 的全体称为该方法的绝对稳定区域. 特别地, 如果绝对稳定区域包含整个左半平面, 则称该方法是 A-稳定的. 绝对稳定区域与实轴的交称为绝对稳定区间.

(1) Euler 方法的稳定性

Euler 方法用于模型方程式 (9.51), 计算公式为

$$y_{n+1}=(1+h\lambda)y_n \tag{9.52}$$

设实际计算 y_n 时产生误差 δ, 它使节点值 y_{n+1} 产生误差 δ_{n+1}, 记 $\tilde{y}_n=y_n+\delta$, $\tilde{y}_{n+1}=y_{n+1}+\delta_{n+1}$, 由式 (9.52) 有

$$\tilde{y}_{n+1}=y_{n+1}+\delta_{n+1}=(1+h\lambda)\tilde{y}_n=(1+h\lambda)y_n+(1+h\lambda)\delta$$

于是有

$$\delta_{n+1}=(1+h\lambda)\delta \tag{9.53}$$

为使误差不扩大, 仅需

$$|1+h\lambda|\leqslant 1 \tag{9.54}$$

因此, Euler 方法的绝对稳定区域为 $|1+h\lambda|\leqslant 1$, 绝对稳定区间为 $-2\leqslant h\lambda\leqslant 0$. 如图 9.2 所示, 这是 $h\lambda$ 平面上以 -1 为中心的单位圆. 当 $h\lambda$ 在此圆内时, Euler 方法是稳定的; 若 $h\lambda$ 在此圆外, 则 Euler 方法不稳定.

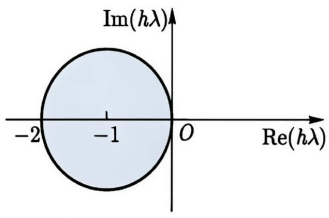


图 9.2 Euler 方法的稳定区域

(2) 梯形公式的稳定性

对模型方程式 (9.51), 梯形公式的具体表达式为

扫描全能王 创建

$$y_{n+1}=y_n+\frac{h}{2}(\lambda y_n+\lambda y_{n+1})$$

即

$$y_{n+1}=\frac{1+\dfrac{h\lambda}{2}}{1-\dfrac{h\lambda}{2}}y_n \tag{9.55}$$

类似于 Euler 方法的讨论可得, 梯形公式的绝对稳定区域为

$$\left|\frac{1+\dfrac{h\lambda}{2}}{1-\dfrac{h\lambda}{2}}\right|\leqslant 1 \tag{9.56}$$

即

$$\left|1+\frac{h\lambda}{2}\right|\leqslant\left|1-\frac{h\lambda}{2}\right|$$

化简得

$$\mathrm{Re}(h\lambda)\leqslant 0$$

因此, 梯形公式的绝对稳定区域为 $h\lambda$ 平面的左半平面 (见图 9.3). 特别地, 当 λ 为负实数时, 对任意的 $h>0$, 梯形公式都是稳定的.

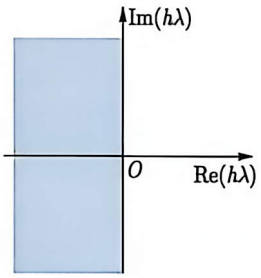


图 9.3　梯形公式的稳定区域

(3) RK 方法的稳定性

与前面的讨论相仿, 将 RK 方法用于模型方程 (9.51), 可得二、三、四阶 RK 方法的绝对稳定区域分别为

$$\left|1+h\lambda+\frac{1}{2}(h\lambda)^2\right|\leqslant 1$$

$$\left|1+h\lambda+\frac{1}{2}(h\lambda)^2+\frac{1}{6}(h\lambda)^3\right|\leqslant 1$$

$$\left|1+h\lambda+\frac{1}{2}(h\lambda)^2+\frac{1}{6}(h\lambda)^3+\frac{1}{24}(h\lambda)^4\right|\leqslant 1$$

其图形见图 9.4, 当 λ 为实数时, 四阶 RK 方法的绝对稳定区间为 $[-2.78,0]$.

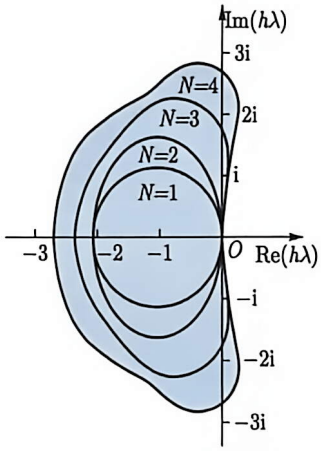


图 9.4　RK 方法的稳定区域

例 9.5　用四阶 RK 方法求解初值问题

$$\begin{cases}y'=-20y, & 0\leqslant x\leqslant 1\\ y(0)=1\end{cases}$$

分别取 $h=0.1,0.2$. 将所得结果与精确解 $y=\mathrm{e}^{-20x}$ 比较, 并说明比较结果.

解　对此问题, 四阶 RK 方法的计算公式为

扫描全能王 创建

$$K_1 = -20y_n, \quad K_2 = -20\left(y_n + \frac{h}{2}K_1\right)$$

$$K_3 = -20\left(y_n + \frac{h}{2}K_2\right), \quad K_4 = -20(y_n + hK_3)$$

$$y_{n+1} = y_n + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4)$$

将初值 $y_0 = y(0) = 1$ 代入, 计算结果见表 9.5.

表 9.5 四阶 RK 方法稳定性数值结果

x_n	$y_n(h=0.1)$	$y(x_n)-y_n$	$y_n(h=0.2)$	$y(x_n)-y_n$
0.0	1		1	
0.1	0.333 333 3	−0.197 998 1		
0.2	0.111 111 1	−0.092 795 5	5	−4.981 684 4
0.3	0.037 037 0	−0.034 558 2		
0.4	0.012 345 6	−0.012 010 1	25	−24.999 665
0.5	0.004 115 2	−0.004 069 8		
0.6	0.001 371 7	−0.001 365 6	125	−124.999 9
0.7	$4.572\,47 \times 10^{-4}$	$-4.564\,15 \times 10^{-4}$		
0.8	$1.524\,15 \times 10^{-4}$	$-1.523\,03 \times 10^{-4}$	625	−625.000 0
0.9	$5.080\,52 \times 10^{-5}$	-5.079×10^{5}		
1.0	$1.881\,67 \times 10^{-6}$	$-1.879\,61 \times 10^{-6}$	3 125	−3 125.000

从表 9.5 可以看出, 当 $h = 0.1$ 时, 虽然计算结果的相对误差很大, 但计算过程是稳定的. 而当 $h = 0.2$ 时, 计算过程不稳定. 这是因为 $\lambda = -20$, 当 $h = 0.1$ 时, $\lambda h = -2$ 属于稳定区间 $[-2.78, 0]$, 当 $h = 0.2$ 时, $\lambda h = -4$ 不属于稳定区间.

对于一般方程 $y' = f(x, y)$, 在考虑稳定性对步长的限制时, 常用 $\dfrac{\partial f}{\partial y}$ 代替 λ, 只要步长的选取使得 $h\lambda$ 在所用方法的绝对稳定区域内即可. 例如, 若用 Euler 方法求解初值问题

$$\begin{cases} y' = -\dfrac{20xy}{1+x^2}, & 0 \leqslant x \leqslant 2 \\ y(0) = 1 \end{cases}$$

因为 $\lambda = \dfrac{\partial f}{\partial y} = -\dfrac{20x}{1+x^2} < 0$, 且 $\max\limits_{0 \leqslant x \leqslant 2} |\lambda| = 10$, 要使 $|1 + h\lambda| \leqslant 1$, 只要 $|1 - 10h| \leqslant 1$, 即 $0 \leqslant x \leqslant 0.2$. 这就是说, 只要取步长 $0 < h \leqslant 0.2$, 用 Euler 方法求解上述问题是绝对稳定的. 如果用四阶 RK 方法, 则需限制步长 h, 使得 $-2.78 \leqslant h\lambda < 0$, 即 $0 < h < 0.278$.

从稳定性定义及数值结果可以看出, 绝对稳定区域越大, 对步长 h 的限制越小, 方法的稳定性越好. 因而梯形公式与 RK 方法的稳定性均优于 Euler 方法.

9.4.3 线性多步法的收敛性

关于线性多步法的收敛性, 有如下限制性结果.

扫描全能王 创建

定理 9.3　显式的线性多步法不可能是 A-稳定的; A-稳定的隐式线性多步法的阶不超过 2, 而在所有 A-稳定二阶方法中, 式 (9.6) 具有最小的局部阶段误差系数.

为了确定一个多步方法的稳定区域, 介绍下面的方法.

定义 9.5　称 k 次复系数多项式

$$\phi(r) \xlongequal{\text{def}} c_k r^k + c_{k-1} r^{k-1} + \cdots + c_1 r + c_0 \tag{9.57}$$

(这里 $c_k \neq 0$, $c_0 \neq 0$) 为 Schur (舒尔) 多项式, 如果它的根 r_s 满足

$$|r_s| \leqslant 1, \quad s = 1, 2, \cdots, k.$$

为判断一个多项式是否为 Schur 多项式, 由式 (9.57) 构造多项式

$$\hat{\phi}(r) \xlongequal{\text{def}} c_k^* r^k + c_{k-1}^* r^{k-1} + \cdots + c_1^* r + c_0^*$$

其中 c_j^* 表示 c_j 的共轭复数 ($j = 1, 2, \cdots, k$) 和多项式

$$\phi_1(r) = \frac{1}{r}[\hat{\phi}(0)\phi(r) - \phi(0)\hat{\phi}(r)]$$

显然, $\phi_1(r)$ 是一个不超过 $k-1$ 次的多项式.

Schur 准则: $\phi(r)$ 是 Schur 多项式当且仅当 $|\hat{\phi}(0)| > |\phi(0)|$, 且 $\phi_1(r)$ 是 Schur 多项式.

由 Schur 准则容易判断某点 $h\lambda$ 是否在一个方法的稳定域内. 类似前一部分的讨论和定义 9.5易得如下定理.

定理 9.4　多步法 (9.23) 在 $h\lambda$ 是绝对稳定的充要条件是多项式 $\Pi(\xi, h\lambda) = \rho(\xi) - h\lambda\sigma(\xi)$ 是 Schur 多项式.

其中 $\Pi(\xi, h\lambda)$ 是这样定义的: 将线性多步法公式 (9.23)

$$y_{n+1} = \sum_{i=0}^{r} \alpha_i y_{n-i} + h \sum_{i=-1}^{r} \beta_i f_{n-i}$$

应用于模型方程 $y' = \lambda y$ 得到常系数线性差分方程

$$y_{n+1} = \sum_{i=0}^{r} \alpha_i y_{n-i} + h\lambda \sum_{i=-1}^{r} \beta_i y_{n-i}$$

该方程的特征多项式记为 $\Pi(\xi, h\lambda) = \rho(\xi) - h\lambda\sigma(\xi)$.

线性多步法 (9.23) 的绝对稳定区域就是使得 $\Pi(\xi, h\lambda) = 0$ 的根位于单位圆内的 $h\lambda$ 的集合 S, 它的边界记为 ∂S. 由于 $\Pi(\xi, h\lambda) = 0$ 的根是关于 $h\lambda$ 的连续函数, 于是, 当有 $h\lambda$ 使得 $\Pi(\xi, h\lambda) = 0$ 的一个根位于单位圆周上时, 此 $h\lambda$ 必位于 ∂S 上, 即是当

$$\Pi(\mathrm{e}^{\mathrm{i}\theta}, h\lambda) = \rho(\mathrm{e}^{\mathrm{i}\theta}) - h\lambda\sigma(\mathrm{e}^{\mathrm{i}\theta})$$

时, $h\lambda \in \partial S$. 这样便得到满足

$$h\lambda = \frac{\rho(\mathrm{e}^{\mathrm{i}\theta})}{\sigma(\mathrm{e}^{\mathrm{i}\theta})}$$

的轨迹, 它便是 ∂S. 这种确定稳定区域边界的方法称为根轨迹方法. 利用 Schur 准则和根轨迹方法便可以确定线性多步法的绝对稳定区域. 更多的关于线性多步法的收敛性与稳定性的工作, 有兴趣的读者可参看文献 [2, 9].

扫描全能王 创建

9.5 微分方程组的数值解法

9.5.1 一阶微分方程组的数值解法

设有一阶微分方程组的初值问题

$$\begin{cases} y_i' = f_i(x, y_1, y_2, \cdots, y_m) \\ y_i(a) = y_{i0} \end{cases} \quad (i = 1, 2, \cdots, m) \tag{9.58}$$

若记 $\boldsymbol{y} = (y_1, y_2, \cdots, y_m)^{\mathrm{T}}, \boldsymbol{y}_0 = (y_{10}, y_{20}, \cdots, y_{m0})^{\mathrm{T}}, \boldsymbol{f} = (f_1, f_2, \cdots, f_m)^{\mathrm{T}}$, 则初值问题式 (9.58) 可写成如下向量形式

$$\begin{cases} \boldsymbol{y}' = \boldsymbol{f}(x, \boldsymbol{y}) \\ \boldsymbol{y}(a) = \boldsymbol{y}_0 \end{cases} \tag{9.59}$$

如果向量函数 $\boldsymbol{f}(x, \boldsymbol{y})$ 在区域

$$D : a \leqslant x \leqslant b, \quad \boldsymbol{y} \in \mathbf{R}^m$$

连续, 且关于 $\boldsymbol{y}$ 满足 Lipschitz 条件, 即存在 $L > 0$, 使得对 $\forall x \in [a, b], \boldsymbol{y}_1, \boldsymbol{y}_2 \in \mathbf{R}^m$, 都有

$$\|\boldsymbol{f}(x, \boldsymbol{y}_1) - \boldsymbol{f}(x, \boldsymbol{y}_2)\| \leqslant L \|\boldsymbol{y}_1 - \boldsymbol{y}_2\|$$

那么问题 (9.59) 在 $[a, b]$ 上存在唯一解 $\boldsymbol{y} = \boldsymbol{y}(x)$.

问题 (9.59) 和式 (9.1) 形式上完全相同, 故对初值问题 (9.1) 所建立的各种数值解法可全部用于求解问题 (9.59), 只需将 y 换成向量 $\boldsymbol{y}, f(x, y)$ 换成向量函数 $\boldsymbol{f}(x, \boldsymbol{y})$ 即可.

例如, 对问题 (9.59), Euler 方法的计算公式为

$$\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + h\boldsymbol{f}(x_n, \boldsymbol{y}_n) \tag{9.60}$$

其中 $\boldsymbol{y}_n = (y_{1n}, y_{2n}, \cdots, y_{mn})^{\mathrm{T}}$, 其分量形式为

$$y_{in+1} = y_{in} + hf_i(x_n, y_{1n}, y_{2n}, \cdots, y_{mn}) \quad (i = 1, 2, \cdots, m)$$

改进 Euler 法的计算公式为

$$\begin{cases} \boldsymbol{y}_{n+1} = \boldsymbol{y}_n + \dfrac{h}{2}(\boldsymbol{K}_1 + \boldsymbol{K}_2) \\ \boldsymbol{K}_1 = \boldsymbol{f}(x_n, \boldsymbol{y}_n) \\ \boldsymbol{K}_2 = \boldsymbol{f}(x_n + h, \boldsymbol{y}_n + h\boldsymbol{K}_1) \end{cases} \tag{9.61}$$

经典四阶 RK 公式为

$$\begin{cases} \boldsymbol{y}_{n+1} = \boldsymbol{y}_n + \dfrac{h}{6}(\boldsymbol{K}_1 + 2\boldsymbol{K}_2 + 2\boldsymbol{K}_3 + \boldsymbol{K}_4) \\ \boldsymbol{K}_1 = \boldsymbol{f}(x_n, \boldsymbol{y}_n) \\ \boldsymbol{K}_2 = \boldsymbol{f}\left(x_n + \dfrac{h}{2}, \boldsymbol{y}_n + \dfrac{h}{2}\boldsymbol{K}_1\right) \\ \boldsymbol{K}_3 = \boldsymbol{f}\left(x_n + \dfrac{h}{2}, \boldsymbol{y}_n + \dfrac{h}{2}\boldsymbol{K}_2\right) \\ \boldsymbol{K}_4 = \boldsymbol{f}(x_n + h, \boldsymbol{y}_n + h\boldsymbol{K}_3) \end{cases} \tag{9.62}$$

扫描全能王 创建

其中 $K_i=(K_{1i},K_{2i},\cdots,K_{mi})^{\mathrm{T}}\,(i=1,2,3,4)$, 式 (9.62) 的分量形式为

$$\begin{cases} y_{in+1}=y_{in}+\dfrac{h}{6}\left(K_{i1}+2K_{i2}+2K_{i3}+K_{i4}\right) \\ K_{i1}=f_i\left(x_n,y_{1n},y_{2n},\cdots,y_{mn}\right) \\ K_{i2}=f_i\left(x_n+\dfrac{h}{2},y_{1n}+\dfrac{h}{2}K_{11},\cdots,y_{mn}+\dfrac{h}{2}K_{m1}\right) \quad (i=1,2,\cdots,m) \\ K_{i3}=f_i\left(x_n+\dfrac{h}{2},y_{1n}+\dfrac{h}{2}K_{12},\cdots,y_{mn}+\dfrac{h}{2}K_{m2}\right) \\ K_{i4}=f_i\left(x_n+h,y_{1n}+hK_{13},\cdots,y_{mn}+hK_{m3}\right) \end{cases} \tag{9.63}$$

一般地, 用于问题 (9.59) 的显式单步法公式为

$$\boldsymbol{y}_{n+1}=\boldsymbol{y}_n+h\boldsymbol{\Phi}\left(x_n,\boldsymbol{y}_n,h\right) \tag{9.64}$$

其中 $\boldsymbol{\Phi}=(\varphi_1,\varphi_2,\cdots,\varphi_m)^{\mathrm{T}}$ 是 $m+2$ 元的 m 维向量函数. 为便于理解, 下面用含有两个方程的方程组 $(m=2)$ 具体说明.

设有初值问题

$$\begin{cases} y'=f(x,y,z), \quad y(a)=y_0 \\ z'=g(x,y,z), \quad z(a)=z_0 \end{cases} \tag{9.65}$$

它的经典四阶 RK 公式为

$$\begin{cases} y_{n+1}=y_n+\dfrac{h}{6}(K_{11}+2K_{12}+2K_{13}+K_{14}) \\ z_{n+1}=z_n+\dfrac{h}{6}(K_{21}+2K_{22}+2K_{23}+K_{24}) \end{cases} \tag{9.66}$$

$$\begin{cases} K_{11}=f\left(x_n,y_n,z_n\right) \\ K_{21}=g\left(x_n,y_n,z_n\right) \\ K_{12}=f\left(x_n+\dfrac{h}{2},y_n+\dfrac{h}{2}K_{11},z_n+\dfrac{h}{2}K_{21}\right) \\ K_{22}=g\left(x_n+\dfrac{h}{2},y_n+\dfrac{h}{2}K_{11},z_n+\dfrac{h}{2}K_{21}\right) \\ K_{13}=f\left(x_n+\dfrac{h}{2},y_n+\dfrac{h}{2}K_{12},z_n+\dfrac{h}{2}K_{22}\right) \\ K_{23}=g\left(x_n+\dfrac{h}{2},y_n+\dfrac{h}{2}K_{12},z_n+\dfrac{h}{2}K_{22}\right) \\ K_{14}=f\left(x_n+h,y_n+hK_{13},z_n+hK_{23}\right) \\ K_{24}=g\left(x_n+h,y_n+hK_{13},z_n+hK_{23}\right) \end{cases} \tag{9.67}$$

计算过程为从节点 x_n 处的近似解 y_n,z_n 出发, 按式 (9.67) 顺序计算 $K_{11},K_{21},K_{12},K_{22},K_{13},K_{23},K_{14},K_{24}$, 将所得结果代入式 (9.66), 即得 x_{n+1} 处的近似解 y_{n+1},z_{n+1}.

类似于本书 9.4节的讨论, 可以建立一阶方程组的数值解法的相容性、收敛性及稳定性概念与结论.

扫描全能王 创建

9.5.2 高阶微分方程的数值解法

高阶微分方程的初值问题可以通过变量代换化为一阶微分方程组初值问题.

设有 m 阶常微分方程初值问题

$$\begin{cases} y^{(m)} = f\left(x, y, y', \cdots, y^{(m-1)}\right), \quad a \leqslant x \leqslant b \\ y(a) = y_0, y'(a) = y_0^{(1)}, \cdots, y^{(m-1)}(a) = y_0^{(m-1)} \end{cases} \tag{9.68}$$

引入新变量 $y_1 = y, y_2 = y', \cdots, y_m = y^{(m-1)}$, 问题 (9.68) 就化为一阶微分方程组初值问题

$$\begin{cases} y_1' = y_2, & y_1(a) = y_0 \\ y_2' = y_3, & y_2(a) = y_0^{(1)} \\ \cdots\cdots\cdots\cdots & \\ y_{m-1}' = y_m, & y_{m-1}(a) = y_0^{(m-2)} \\ y_m' = f\left(x, y_1, y_2, \cdots, y_m\right), & y_m(a) = y_0^{(m-1)} \end{cases} \tag{9.69}$$

用数值方法求解问题 (9.69), 就可以得到问题 (9.68) 的数值解.

例 9.6 用四阶 RK 方法求解初值问题

$$\begin{cases} y'' = 5\mathrm{e}^{2x}\sin x - 2y + 2y', \quad 0 \leqslant x \leqslant 1 \\ y(0) = -2, y'(0) = -3 \end{cases} \tag{9.70}$$

取 $h = 0.1$.

解 引入新变量 $z = y'$, 问题 (9.70) 化为如下形式

$$\begin{cases} y' = z, & y(0) = -2 \\ z' = 5\mathrm{e}^{2x}\sin x - 2y + 2z, & z(0) = -3 \end{cases}$$

这是形如式 (9.65) 的问题, 按式 (9.66)、式 (9.67) 求解. 因为 $x_0 = 0$, $y_0 = -2$, $z_0 = -3$, 由式 (9.67) 得

$$K_{11} = z_0 = -3$$

$$K_{21} = 5\mathrm{e}^{2x_0}\sin x_0 - 2y_0 + 2z_0 = 4 - 6 = -2$$

$$K_{12} = z_0 + \frac{h}{2}K_{21} = -3 - 0.1 = -3.1$$

$$K_{22} = 5\mathrm{e}^{h}\sin\frac{h}{2} - 2\left(y_0 + \frac{h}{2}K_{11}\right) + 2\left(z_0 + \frac{h}{2}K_{21}\right) = -1.623\ 822\ 4$$

$$K_{13} = z_0 + \frac{h}{2}K_{22} = -3.081\ 191\ 1$$

$$K_{23} = 5\mathrm{e}^{0.1}\sin 0.05 - 2\left(-2 - \frac{0.1}{2}\times 3.1\right) + 2\left(-3 - \frac{0.1}{2}\times 1.623\ 822\ 4\right)$$

$$= -1.576\ 204\ 6$$

$$K_{14} = z_0 + hK_{23} = -3.157\ 620\ 5$$

扫描全能王 创建

$$K_{24} = 5\mathrm{e}^{0.2}\sin 0.1 - 2(-2 - 0.1 \times 3.081\ 191\ 1) + 2(-3 - 0.1 \times 1.576\ 204\ 6)$$
$$= -1.089\ 318\ 6$$

将上述结果代入式 (9.66) 得

$$y_1 = y_0 + \frac{h}{6}(K_{11} + 2K_{12} + 2K_{13} + K_{14}) = -2.308\ 666\ 7$$
$$z_1 = z_0 + \frac{h}{6}(K_{21} + 2K_{22} + 2K_{23} + K_{24}) = -3.158\ 156\ 2$$

类似地, 可计算以后各节点处的近似解, 计算结果见表 9.6.

表 9.6　例 9.6 的计算结果

x_n	y_n	z_n	$y(x_n) - y_n$
0	−2	−3	0
0.1	−2.308 666 7	−3.158 156 2	1.85×10^{-6}
0.2	−2.627 799 4	−3.200 744 8	4.39×10^{-6}
0.3	−2.943 007 2	−3.068 319 1	7.31×10^{-6}
0.4	−3.233 061 6	−2.682 910 2	1.06×10^{-5}
0.5	−3.467 833 3	−1.943 690 5	1.39×10^{-5}
0.6	−3.605 759 5	−0.721 904 4	1.72×10^{-5}
0.7	−3.590 764 8	1.144 985 1	2.10×10^{-5}
0.8	−3.348 556 7	3.859 959 0	2.29×10^{-5}
0.9	−2.782 214 5	0.767 390 8	2.46×10^{-5}
1.0	−1.766 994 3	1.289 383 2	2.26×10^{-5}

问题 (9.70) 的精确解为 $y = \mathrm{e}^{2x}(\sin x - 2\cos x)$. 将它与近似解比较即得误差 (见表 9.6 中最后一列).

9.5.3　刚性方程组

在自动控制、电力系统、化学反应等领域中, 常会出现一类特殊的常微分方程组. 例如, 初值问题

$$\begin{bmatrix} y_1' \\ y_2' \end{bmatrix} = \begin{bmatrix} -0.01 & -299.99 \\ 0 & -300 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}, \quad \begin{matrix} y_1(0) = 2 \\ y_2(0) = 1 \end{matrix} \tag{9.71}$$

其解析解为

$$\begin{cases} y_1 = \mathrm{e}^{-0.01x} + \mathrm{e}^{-300x} \\ y_2 = \mathrm{e}^{-300x} \end{cases}$$

在解析解的表达式中, 当 $x \to +\infty$ 时, e^{-300x} 变化迅速, 称为解的快瞬态部分; $\mathrm{e}^{-0.01x}$ 相对变化缓慢, 称为解的慢瞬态部分. $y_1(x)$ 含有慢瞬态部分和快瞬态部分, 整体变化缓慢, $y_2(x)$ 仅含有快瞬态部分, 迅速趋于零.

扫描全能王 创建

另一方面, 如果用数值方法求解这类问题, 由于方法绝对稳定域的限制, 将对步长的选取十分苛刻. 例如, 用四阶 RK 方法求解方程组 (9.71), 由于方程组 (9.71) 的系数矩阵的特征值为 $\lambda_1=-300,\lambda_2=-0.01$, 为使计算稳定, 要求 $-300h\in(-2.785,0)$, 即要求步长 $h<0.009\,28$, 计算到 $x=10$ 时, 需要计算 1 078 步, 这就需要相当大的计算量. 而且由于步长过小, 计算步数过多, 使得舍入误差积累, 影响解的真实性. 这就是微分方程的刚性 (stiff) 现象.

定义 9.6 设线性方程组

$$\boldsymbol{y}'(x)=\boldsymbol{A}\boldsymbol{y}(x)+\boldsymbol{b}(x) \tag{9.72}$$

的系数矩阵 $\boldsymbol{A}$ 的特征值为 $\lambda_1,\lambda_2,\cdots,\lambda_n$. 若

(1) $\operatorname{Re}\lambda_j<0, j=1,2,\cdots,n$

(2) $s=\dfrac{\max\limits_{1\leqslant j\leqslant n}|\operatorname{Re}\lambda_j|}{\min\limits_{1\leqslant j\leqslant n}|\operatorname{Re}\lambda_j|}\gg 1$

则称方程组 (9.72) 为刚性方程组 (或病态方程组), s 为刚性比.

方程组 (9.71) 的刚性比 $s=30\,000$, 是一个严重刚性的方程组.

对于刚性方程组的数值求解, 应采用步长 h 不受稳定性限制的 A-稳定方法, 如隐式 Euler 法、隐式多步法或隐式 RK 法, 并用 Newton 迭代法求解隐式方程组. 实际计算中常用的有以下几种方法:

二阶隐式 RK 方法:

$$\begin{cases}\boldsymbol{y}_{n+1}=\boldsymbol{y}_n+h\boldsymbol{K}_1\\ \boldsymbol{K}_1=\boldsymbol{f}\left(x_n+\dfrac{h}{2},\boldsymbol{y}_n+\dfrac{h}{2}\boldsymbol{K}_1\right)\end{cases} \tag{9.73}$$

二阶隐式 RK 方法:

$$\begin{cases}\boldsymbol{y}_{n+1}=\boldsymbol{y}_n+\dfrac{h}{2}(\boldsymbol{K}_1+\boldsymbol{K}_2)\\ \boldsymbol{K}_1=\boldsymbol{f}(x_n,\boldsymbol{y}_n)\\ \boldsymbol{K}_2=\boldsymbol{f}\left(x_n+h,\boldsymbol{y}_n+\dfrac{h}{2}(\boldsymbol{K}_1+\boldsymbol{K}_2)\right)\end{cases} \tag{9.74}$$

四阶隐式 RK 方法:

$$\begin{cases}\boldsymbol{y}_{n+1}=\boldsymbol{y}_n+\dfrac{h}{2}(\boldsymbol{K}_1+\boldsymbol{K}_2)\\ \boldsymbol{K}_1=\boldsymbol{f}\left(x_n+h,\boldsymbol{y}_n+\dfrac{h}{4}\left(\boldsymbol{K}_1+\left(1+\dfrac{2\sqrt{3}}{3}\right)\boldsymbol{K}_2\right)\right)\\ \boldsymbol{K}_2=\boldsymbol{f}\left(x_n+\left(\dfrac{1}{2}-\dfrac{\sqrt{3}}{6}\right)h,\boldsymbol{y}_n+\dfrac{h}{4}\left(\left(1-\dfrac{2\sqrt{3}}{3}\right)\boldsymbol{K}_1+\boldsymbol{K}_2\right)\right)\end{cases} \tag{9.75}$$

例如, 对于初值问题 (9.71), 采用隐式 RK 公式 (9.73) 迭代一次计算, 若取步长大于 $h=0.006\,6$, 计算结果错误. 在数值算例部分给出了利用北太天元软件编写的代码及计算结果. 刚性微分方程的数值计算是一个复杂而有困难的过程, 其他的一些数值计算方法可参阅文献 [13].

扫描全能王 创建

9.6　边值问题的数值解法

科学研究和工程计算中还会出现另外一类常微分方程求解问题. 例如, 二阶常微分方程

$$y'' = f(x, y, y'), \quad a < x < b \tag{9.76}$$

在自变量 x 的两个不同点处给出解 $y(x)$ 的条件, 这类问题称为边值问题, 所给条件称为边界条件. 方程式 (9.76) 的边界条件有三种:

第一边界条件: $y(a) = \alpha, y(b) = \beta$

第二边界条件 : $y'(a) = \alpha, y'(b) = \beta$

第三边界条件: $y(a) + \alpha_0 y'(a) = \alpha, y(b) + \beta_0 y'(b) = \beta$

其中 $\alpha, \beta, \alpha_0, \beta_0$ 均为常数. 求解边值问题的方法很多, 本节简单介绍打靶法和有限差分法.

为简便起见, 本节专门讨论二阶线性微分方程的边值问题

$$\begin{cases} y'' + p(x)y' + q(x)y = f(x), \quad a < x < b \\ y(a) = \alpha, \quad y(b) = \beta \end{cases} \tag{9.77}$$

9.6.1　打靶法

打靶法可用于求解高阶线性或非线性常微分方程, 它的基本思路是: 将边值问题化为初值问题, 用前面介绍的求解初值问题的各种方法计算近似解, 适当调整初值条件, 使得近似解逼近给定的边界条件. 这种方法很像瞄准目标打靶, 故称为打靶法或试射法. 具体求解步骤如下:

(1) 先预测 $y(x)$ 的导数在点 $x = a$ 处的值 $y'(a) = t_1$, 将边值问题 (9.77) 化为初值问题

$$\begin{cases} y'' + p(x)y' + q(x)y = f(x), \quad a < x < b \\ y(a) = \alpha, \quad y'(a) = t_1 \end{cases} \tag{9.78}$$

解初值问题 (9.78), 得数值解 $y_1(x)$.

(2) 比较 $y_1(b) = \beta_1$ 与 β, 若 $\beta_1 = \beta$ 或 $|\beta_1 - \beta| < \varepsilon$, 则 $y_1(x)$ 即为所求的数值解.

(3) 若不是 (2) 的情况, 修改 $y'(a)$ 的值. 例如, 取 $y'(a) = t_2 = \dfrac{\beta}{\beta_1} t_1$, 用 t_2 代替方程 (9.78) 中的 t_1, 再解初值问题 (9.78) 得解 $y_2(x)$.

(4) 再比较 $y_2(b) = \beta_2$ 与 β, 若 $\beta_2 = \beta$ 或 $|\beta_2 - \beta| < \varepsilon$, 则 $y_2(x)$ 即为所求的数值解; 否则继续修改 t_2, 直至 $\beta_k = \beta$ 或 $|\beta_k - \beta| < \varepsilon$.

一般 t_k 可按公式

$$t_k = t_{k-1} - \frac{\beta_{k-1} - \beta}{\beta_{k-1} - \beta_{k-2}} (t_{k-1} - t_{k-2}), \quad k = 2, 3, \cdots \tag{9.79}$$

逐步修正.

扫描全能王 创建

9.6.2 有限差分法

有限差分法的基本思想是将区间 $[a,b]$ 剖分, 在各节点上用差商代替导数, 将微分方程离散成差分方程, 求解差分方程组, 从而得到解在各节点上的近似值. 具体方法如下:

将区间 $[a,b]$ 作 n 等分, 则步长 $h=(b-a)/n$, 节点 $x_i=a+ih$ $(i=0,1,\cdots,n)$. 在内部节点 x_i $(i=1,2,\cdots,n-1)$ 上, 方程组 (9.77) 为

$$y''(x_i)+p(x_i)y'(x_i)+q(x_i)y(x_i)=f(x_i),\quad i=1,2,\cdots,n-1 \tag{9.80}$$

用中心差商近似导数

$$y'(x_i)=\frac{1}{2h}\left[y(x_{i+1})-y(x_{i-1})\right]-\frac{h^2}{6}y^{(3)}(\xi_i),\quad x_{i-1}<\xi_i<x_{i+1}$$

$$y''(x_i)=\frac{1}{h^2}\left[y(x_{i+1})-2y(x_i)+y(x_{i-1})\right]-\frac{h^2}{12}y^{(4)}(\eta_i),\quad x_{i-1}<\eta_i<x_{i+1}$$

将以上两式代入式 (9.80), 得

$$\begin{aligned}\frac{1}{h^2}\left[y(x_{i+1})-2y(x_i)\right.&\left.+y(x_{i-1})\right]+\frac{1}{2h}p(x_i)\left[y(x_{i+1})-y(x_{i-1})\right]\\&+q(x_i)y(x_i)=f(x_i)+O(h^2),\quad i=1,2,\cdots,n-1\end{aligned}$$

省略无穷小量 $O(h^2)$, 分别以 p_i,q_i,f_i,y_i 代替 $p(x_i),q(x_i),f(x_i),y(x_i)$, 若取边界条件 $y(a)=\alpha,y(b)=\beta$, 则得差分方程组

$$\begin{cases}(-2+h^2q_1)y_1+\left(1+\dfrac{h}{2}p_1\right)y_2=h^2f_1-\left(1-\dfrac{h}{2}p_1\right)\alpha\\ \left(1-\dfrac{h}{2}p_i\right)y_{i-1}+(-2+h^2q_i)y_i+\left(1+\dfrac{h}{2}p_i\right)y_{i+1}=h^2f_i,\quad i=2,3,\cdots,n-2\\ \left(1-\dfrac{h}{2}p_{n-1}\right)y_{n-2}+(-2+h^2q_{n-1})y_{n-1}=h^2f_{n-1}-\left(1+\dfrac{h}{2}p_{n-1}\right)\beta\end{cases} \tag{9.81}$$

式 (9.81) 是三对角线性方程组, 可用追赶法求解. 其他边界条件也可得到形式相近的线性方程组.

可以证明: 若解函数 $y(x)$ 充分光滑, 则差分格式 (9.81) 是二阶方法, 即

$$\max_{0\leqslant i\leqslant n}|y(x_i)-y_i|=O(h^2)$$

例 9.7　试分别取 $n=10,20,40$ 用差分法求解边值问题

$$\begin{cases}y''+\dfrac{x}{1-x}y'-\dfrac{1}{1-x}y=x\\ y(0)=1,\quad y(1)=2\mathrm{e}\end{cases}$$

并将计算结果与精确解 $y(x)=2\mathrm{e}^x+2x-x^2-1$ 进行比较.

解　取 $n=10,h=0.1$, 式 (9.81) 为

扫描全能王 创建

$$\begin{bmatrix} -2.010\,0 & 1.000\,0 & & & & & & & \\ 0.994\,4 & -2.011\,1 & & & & & & & \\ & 0.987\,6 & & & & & & & \\ & & & & & & & & \\ & & & & & & & & \\ & & & & & & & & \\ & & & & & & & & \\ & & & & & 1.116\,7 & & & \\ & & & & 0.883\,3 & -2.033\,3 & 1.200\,0 & & \\ & & & & & 0.800\,0 & -2.050 & & \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \\ y_8 \\ y_9 \end{bmatrix} = \begin{bmatrix} -0.010\,0 \\ -0.009\,0 \\ -0.008\,0 \\ -0.007\,0 \\ -0.006\,0 \\ -0.005\,0 \\ -0.004\,0 \\ -0.003\,0 \\ -0.002\,0 \end{bmatrix}$$

解方程组即可得近似解, 计算结果见表 9.7. 根据表 9.7 中数据知: 当 h 越来越小时, 数值解的精度也在逐步提高.

表 9.7　例 9.7 的计算结果

x_n	$y_n(h=0.1)$	$y_n(h=0.05)$	$y_n(h=0.025)$	$y(x_n)$
0.0	1.000 0	1.000 0	1.000 0	1.000 0
0.1	1.389 8	1.394 4	1.397 2	1.400 3
0.2	1.783 4	1.792 0	1.797 1	1.802 8
0.3	2.183 5	2.195 0	2.201 9	2.209 7
0.4	2.592 7	2.606 3	2.614 5	2.623 6
0.5	3.014 2	3.028 8	3.037 5	3.047 4
0.6	3.451 3	3.465 8	3.474 4	3.484 2
0.7	3.907 8	3.920 8	3.928 6	3.937 5
0.8	4.387 7	4.397 9	4.404 1	4.411 1
0.9	4.895 6	4.901 6	4.905 1	4.909 2
1.0	5.436 6	5.436 6	5.436 6	5.436 6

有限差分法的其他格式以及各种格式的稳定性、收敛性等问题可参阅偏微分方程数值解的相关教材.

9.7　Hamilton 系统保结构算法简介

经典力学有三种等价的基本形式, 即 Newton 力学、Lagrange 力学和 Hamilton 力学. 这三种不同形式都可以用某种具有特定结构的常微分方程来表示. 虽然三种形式在数学上等价, 但是不同的表述形式对于相应方程的数值模拟可以带来很大差异, 对它们各自的数值格式的构造和分析都具有重要影响. 所以在设计计算格式之前, 我们应该选择一种适合的数学形式.

冯康先生首先认识到 Hamilton 形式具有重要的优势, 而且这种数学表达形式的优势

扫描全能王 创建

可以使得与之匹配的数值方法获得更优的数值结果. 这个思想被称为 “冯氏大定理”. 绝大多数真实的, 耗散效应可以忽略不计的系统都可以表示成 Hamilton 系统的形式, 所以对 Hamilton 系统进行数值研究具有重要意义. 自 20 世纪 80 年代以来, Hamilton 系统数值方法的研究成为计算数学的一个重要课题. 本节对 Hamilton 系统保结构算法做一些简单介绍. 进一步的介绍可以参考文献 [26].

经典 Hamilton 系统表示为

$$\dot{p}(t) = -\frac{\partial H(p,q)}{\partial q}, \quad \dot{q}(t) = -\frac{\partial H(p,q)}{\partial p} \tag{9.82}$$

其中 $p = (p_1, p_2, \cdots, p_N)^T$ 称为广义动量, $q = (q_1, q_2, \cdots, q_N)^{\mathrm{T}}$ 称为广义位置, $H(p,q)$ 为 Hamilton 函数, 表示系统能量. 加上适当的初值 $p(0) = p_0$, $q(0) = q_0$, 就可构成一个常微分初值问题. 该方程表示系统位置和动量随着时间的演化. Hamilton 系统有两个重要的特征, 即系统的能量守恒和辛结构.

辛结构的几何解释是系统的相流在各个坐标组合 (p_i, q_i) 中的投影面积之和不变. 这个重要结构使得系统具有非常多的重要性质, 如没有渐进稳定的平衡点, 系统的相空间体积守恒等. 保结构算法的基本思想, 即数值算法应该尽可能地保持原系统的重要结构. 基于这个思想, 对 Hamilton 系统而言, 我们构造的数值方法应该保持系统的辛结构或者能量不变. 设 $y_n = (p_n^{\mathrm{T}}, q_n^{\mathrm{T}})^{\mathrm{T}}$, $n \geqslant 0$, 则单步方法, 如 RK 方法, 可以定义一个离散映射

$$y_{n+1} = \Phi_h(y_n), \quad n \geqslant 0$$

称一个算法是保辛的, 如果

$$\left(\frac{\partial \Phi_h(y_n)}{\partial (y_n)}\right)^{\mathrm{T}} J \left(\frac{\partial \Phi_h(y_n)}{\partial (y_n)}\right) = J.$$

根据这个定义, 可以证明中点格式是一个辛格式. 称一个算法是保能量的, 如果 $H(p_n, q_n) = H(p_0, q_0)$, $n \geqslant 1$.

对 Hamilton 系统而言, 保结构算法的研究取得了丰硕的成果, 这些研究大致分为两大类, 即保辛算法和保能量算法. 由于辛和能量都是 Hamilton 系统的重要特征, 人们自然希望寻找同时保辛和保能量的方法, 但是葛忠和 Marsden (马斯登) 证明, 通常情况下对于非可积 Hamilton 系统, 辛算法是不能保持能量的. 这个结果也从一个侧面反映了离散动力系统和连续系统的内在差异.

9.8 算法实现与应用实例

北太天元软件提供了求解常微分方程函数.

算法实现

基于算法 9.2 的四阶 RK 的程序如下:

扫描全能王 创建

```
function yrk4=rk4(fxy,a,b,y0,n)
h=(b-a)/n ;
x=a:h:b;
yrk4= zeros(n,1);
yrk4(1)=y0;
for i = 1 : n − 1
   k1 = fxy(x(i), yrk 4 (i));
   k2 = fxy(x(i) + h/2, yrk4(i) + h ∗ k1/2);
   k3 = fxy(x(i) + h/2, yrk 4(i) + h ∗ k2/2);
   k4 = fxy(x(i) + h, yrk4(i) + h ∗ k3);
   yrk 4(i + 1) = yrk 4(i) + h ∗ (k1 + 2 ∗ k2 + 2 ∗ k3 + k4)/6;
end;
plot(x,yrk4,'-o');
```

实际上也可以直接调用北太天元中的命令 ode23(　), ode45(　), ode78(　), ode89(　), 数字表示采用的 RK 方法的阶数. 例如可以用 ode23(　) 求解 van der Pol 方程

$$y'' - \mu(1 - y^2)y' + y = 0$$

荷兰电气工程师 Balthasar van der Pol (巴尔塔萨 • 范德波尔, 1889—1959) 于 1927 年 9 月在 *Nature* 杂志上发文, 报告了他在霓虹灯实验中观察到真空管放大器的电流存在具有极限环的振荡现象. van der Pol 写下了这个张弛振荡器 (relaxation oscillator) 的微分方程:

$$\ddot{x} - k\left(1 - x^2\right)\dot{x} + x = bk\lambda\cos(\lambda t + \alpha)$$

其中 k, b, λ, α 为参数. 这个方程和 Rayleigh (瑞利, 1842—1919) 的方程是等价的, 它还可以看作是 Liénard (李纳德, 1869—1958) 方程的一种特别情形. 不过, 这个方程极限环的存在性是 van der Pol 在 $k > 0$ 和 $b = 0$ 时在实验中证实的. 在 $k > 0$ 和 $b \neq 0$ 以及 $\lambda \neq 0$ 即有驱动信号输入时, 在某种自然频率下他听到了来自系统内部的“毫无规律的品声”. 这篇文章很可能是观察到物理混沌现象最早的实验报告, 只不过那时还没有混沌这个概念. 可以直接用北太天元软件自带函数求解 van der Pol 方程并生成数值解图像 (图 9.5):

```
function dydt = vdp1(t,y)
dydt = [y(2); (1 − y(1) ∗ y(1)) ∗ y(2) − y(1)];
end
% 编写脚本文件 vdpsolver.m 调用北太天元软件函数 ode23()
odefun = @vdp1;
tspan = [0 20];
y0 = [2; 0];
options = odeset('RelTol',1e-3,'AbsTol',1e-5);
[t, y]= ode23(odefun,tspan,y0,options);
plot(t,y(:,1),'-o',t);
```


扫描全能王 创建

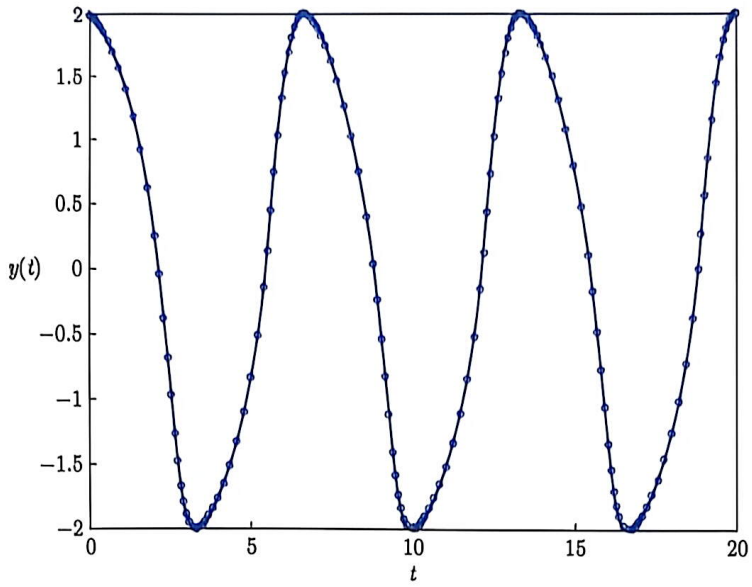


图 9.5 van der Pol 方程数值解

刚性方程组程序

对于初值问题 (9.71), 采用隐式 RK 公式 (9.73) 迭代一次计算, 编程如下:

```
function f=dyfn(t, y)
f(1)=-0.01 * y(1)-299.99 * y(2) ;
f(2)=-300 * y(2);
end
```

```
function [x,y]=RK11(xspan,y0,h)
x=xspan(1): h: xspan(2) ;
y=zeros(length(y0), length (x)) ;
y(:, 1)=y0(:)
for i=1:length(x)-1
k1=dyfn( x(i), y(:, i));
y(:, i+1)=y(:, i)+h * k 1
k2= dyfn(x(i+1), y(:, i+1));
y(:, i+1)=y(:, i)+h *(k 1+k 2) / 2;
end
end
```

取步长 $h = 0.005$, 键入命令

$\gg$ [x, y] = RK11([0 300], [2; 1], 0.005);

$\gg$ plot(x, y(1), x, y(2)); axis([−5, 300])

生成数值解图像 (图 9.6)

扫描全能王 创建

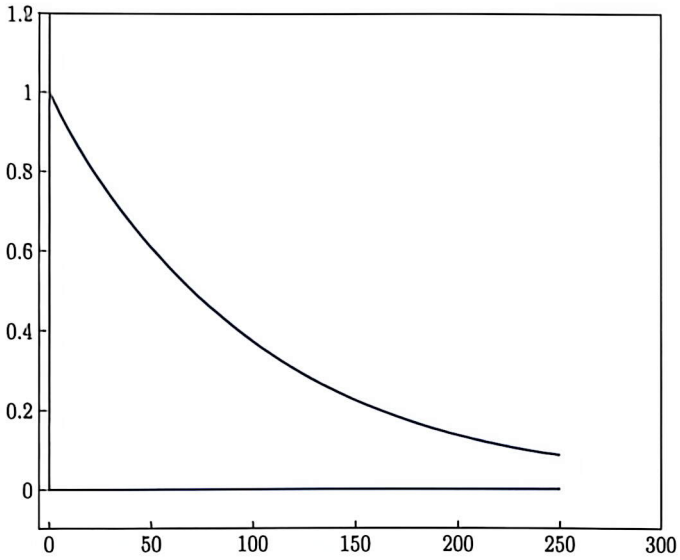


图 9.6　h=0.005 方程数值解

评注九

常微分方程初值问题的数值解法主要分单步法和多步法. Euler 公式、改进的 Euler 公式以及各阶 Runge-Kutta 方法都是单步法. 对于一般的初值问题, 通常采用改进的 Euler 公式, 就能保证二阶精度. 如果方程的右端项 $f(x,y)$ 足够光滑, 计算精度要求较高, 经典 Runge-Kutta 法是不错的方法. 实际计算中常用的是变步长的 Runge-Kutta 法, 这样既能保证精度, 又能节省计算量. 如果计算 $f(x,y)$ 代价较高, 可选用线性多步法或预测–校正公式. 若要获得更高精度 (如误差小于 10^{-10}), 需要采用更高精度的计算公式, 再加上逐次分半步长或外推技巧. 收敛性、相容性和稳定性是微分方程数值解的重要概念, 实际计算中应先分析方法的稳定性, 选取步长 h 使得 λh 落在稳定区域内, 只有稳定的方法才能得到正确的结果.

对于微分方程组、刚性方程组以及边值问题, 本章仅介绍了最基本的方法.

本章知识要点　Runge-Kutta 算法; 线性多步法.

算法历史及背后的数学家

习题九

1. 用 Euler 方法求下列初值问题的数值解, 取 $h=0.1, N=10$, 计算结果保留 4 位小数, 并与精确解进行比较:

扫描全能王 创建

(1) $\begin{cases} y' = x + y, & 0 \leqslant x \leqslant 1 \\ y(0) = 1 \end{cases}$

(2) $\begin{cases} y' = y - \dfrac{2x}{y}, & 0 \leqslant x \leqslant 1 \\ y(0) = 1 \end{cases}$

2. 用 Euler 方法求初值问题

$$\begin{cases} y' = 1 - \dfrac{2xy}{1+x^2}, & 0 \leqslant x \leqslant 2 \\ y(0) = 0 \end{cases}$$

的数值解, 取 $h = 0.5$, 计算结果取 5 位小数, 并与精确解 $y(x) = \dfrac{x\,(3+x^2)}{3\,(1+x^2)}$ 比较.

3. 用改进 Euler 法求下列问题的数值解, 取 $h = 0.1$, 计算结果保留 4 位小数:

(1) $\begin{cases} y' = x + y, & 0 \leqslant x \leqslant 1 \\ y(0) = 1 \end{cases}$

(2) $\begin{cases} y' = y - \dfrac{2x}{y}, & 0 \leqslant x \leqslant 1 \\ y(0) = 1 \end{cases}$

4. 分别用改进 Euler 法与四阶 RK 方法求初值问题

$$\begin{cases} y' = 1 - \dfrac{2xy}{1+x^2}, & 0 \leqslant x \leqslant 2 \\ y(0) = 0 \end{cases}$$

的数值解, 取 $h = 0.5$, 计算结果取 5 位小数, 并与第 2 题的计算结果作比较.

5. 用 Euler 方法及改进 Euler 方法计算积分

$$I(x) = \int_0^x \mathrm{e}^{t^2}\,\mathrm{d}t$$

在 $x = 1$ 处的近似值, 取 $h = 0.1$, 计算结果保留 4 位小数.

6. 用中点公式及四阶 RK 公式求初值问题

$$\begin{cases} y' = x + y \\ y(0) = 1 \end{cases}$$

的解 $y(x)$ 在 $x = 1$ 处的近似值, 取 $h = 0.2$, 计算结果保留 4 位小数.

7. 用四阶 RK 方法求初值问题

$$\begin{cases} y' + y = 0, & 0 \leqslant x \leqslant 1 \\ y(0) = 1 \end{cases}$$

的数值解, 取 $h = 0.1$, 计算结果保留 4 位小数.

扫描全能王 创建

8. 选取参数 a, b, 使下述形式的 RK 公式为二阶公式

$$\begin{cases} y_{n+1} = y_n + hK_2 \\ K_1 = f(x_n, y_n) \\ K_2 = f\left(x_n + ah, y_n + hbK_1\right) \end{cases}$$

9. 分别用四阶 Adams 显式和隐式公式求初值问题

$$\begin{cases} y' = -xy^2, \quad 0 \leqslant x \leqslant 1 \\ y(0) = 2 \end{cases}$$

的数值解, 取 $h = 0.2$.

10. 试证二步公式

$$y_{n+1} = \frac{1}{2}\left(y_n + y_{n-1}\right) + \frac{h}{4}\left(4f_{n+1} - f_n + 3f_{n-1}\right) \quad (n = 1, 2, \cdots)$$

是二阶方法, 并估计其局部误差.

11. 设有求解常微分方程初值问题

$$\begin{cases} y' = f(x, y) \\ y\left(x_0\right) = y_0 \end{cases}$$

的二步公式

$$y_{n+1} = y_n + \frac{h}{12}\left(5f_{n+1} + 8f_n - f_{n-1}\right)$$

试求该公式的局部截断误差, 并回答该方法是几阶精度的.

12. 试用 Taylor 展开构造形如

$$y_{n+1} = y_{n-3} + h\left(\alpha f_n + \beta f_{n-1}\right)$$

的四步公式, 使它具有尽可能高的阶, 并写出其局部截断误差.

13. 对初值问题

$$\begin{cases} y' = y \\ y(0) = 1 \end{cases}$$

用改进 Euler 法求近似解的表达式, 并证明当步长 h 趋于 0 时, 近似解收敛于精确解 $y = \mathrm{e}^x$.

14. 试求向后 Euler 方法的绝对稳定区域.

15. 对初值问题

$$\begin{cases} y' = -1\,000\left(y - x^2\right) + 2x, \quad 0 \leqslant x \leqslant 1 \\ y(0) = 0 \end{cases}$$

扫描全能王 创建

用 Euler 方法求数值解, 分别取步长为 $h=0.1, h=0.01$, 将所得结果与精确解 $y=x^2$ 比较, 并用稳定性条件说明比较结果.

16. 用改进 Euler 法求问题

$$\begin{cases} y''-2y'+2y=\mathrm{e}^x\sin x, & 0\leqslant x\leqslant 1 \\ y(0)=0, y'(0)=-\dfrac{1}{2} \end{cases}$$

的数值解, 取 $h=0.1$.

17. 用四阶 RK 方法求解第 16 题中问题, 并比较结果.

18. 写出用四阶 RK 方法求解问题

$$\begin{cases} y'''+2y''-y'-2y=\mathrm{e}^x, & 0\leqslant x\leqslant 3 \\ y(0)=1, y'(0)=2, y''(0)=0 \end{cases}$$

的计算公式, 取 $h=0.2$.

19. 取 $t_0=2$, 用打靶法求两点边值问题

$$\begin{cases} y''-(y')^2+x=0 \\ y(0)=1, y(0.4)=1.8 \end{cases}$$

要求误差小于 $\varepsilon=0.5\times 10^{-4}$.

20. 取步长 $h=0.1$, 用有限差分法求解两点边值问题

$$\begin{cases} -y''+y=(\pi^2+1)\sin(\pi x) \\ y(0)=0, y(1)=0 \end{cases}$$

数值实验九

1. 设有常微分方程初值问题

$$\begin{cases} y'=-y+2\cos x, & 0<x<\pi \\ y(0)=1 \end{cases}$$

其精确解为 $y=\cos x+\sin x$. 选取步长 h 使四阶 Adams 预测 $-$ 校正算法和经典 RK 法均稳定, 分别用这两种方法求解微分方程, 将数值解与精确解进行比较, 输出结果. 其中多步法需要的初值由经典 RK 法提供.

2. 考虑 Rossler (罗斯勒) 化学反应方程

$$\begin{cases} x'=-y-z \\ y'=x+ay, \quad x(0)=y(0)=z(0)=d \\ z'=(x-c)z+b \end{cases}$$

扫描全能王 创建

将 a,b,c,d 设为参数, 编程求数值解. 取 $a=0.2$, $b=0.8$, $c=4.5$, $d=1$, 绘制数值解的三维相轨迹, 并作出在 xOy 平面上的投影.

3. 选择状态函数, 将高阶非线性微分方程

$$\begin{cases} x''-2xzx'=3x^2yt^2 \\ y''-\mathrm{e}^y y'=4xzt^2 \quad x(1)=y(1)=z(1)=3 \\ z''-2tz'=2t\mathrm{e}^{xy} \quad x'(1)=y'(1)=z'(1)=2 \end{cases}$$

转化为一阶显式微分方程组, 编程求其数值解, 并绘制出解的相空间曲线.

4. 考虑反应速度方程组

$$\begin{cases} y_1'=-0.04y_1+10^4y_2y_3 \\ y_2'=0.04y_1-10^4y_2y_3-3\times 10^7y_2^2 \\ y_3'=3\times 10^7y_2^2 \end{cases}$$

初值为 $y_1(0)=1$, $y_2(0)=y_3(0)=0$. 这是一个刚性方程组, 试用 ode45( ) 计算, 结果如何? 如不能得到较精确的解应该如何处理?

5. 方程

$$x^2y''-x(x+2)y'+(x+2)y=0$$

的通解为 $y=C_1x+C_2x\mathrm{e}^x$, 若边界条件为 $y(1)=e$, $y(2)=2\mathrm{e}^2$, 确定方程的特解. 取 $h=0.1,0.05,0.025$, 用差分法编程求解该边值问题, 并研究步长 h 减半时误差的变化情况.

扫描全能王 创建

/附录/

北太天元及其应用简介

北太天元是面向科学计算与工程计算的国产通用型科学计算软件. 本软件提供科学计算、可视化、交互式程序设计, 具备强大的底层数学函数库, 支持数值计算、数据分析、数据可视化、数据优化、算法开发等工作, 并通过 SDK 与 API 接口, 扩展支持各类学科与行业场景, 为各领域科学家与工程师提供优质、可靠的科学计算环境.

北太天元操作面板介绍

安装完北太天元之后, 在桌面会自动生成北太天元图标. 双击图标, 即见到如附图 1 的操作面板.

附图 1　北太天元操作面板

1. 命令行窗口为输入命令的主要区域, 它由命令提示符 (≫) 指示, 北太天元的一般计算与基本控制都可以通过命令行窗口实现;

2. 工作区显示了目前内存中所有的变量名、数据结构等;

3. 历史记录面板主要记录了命令行输入的历史, 可以通过在命令行窗口使用方向键的上 (↑) 按键触发.

4. 快捷菜单栏可以实现文件的打开、新建、保存、数据导入、工作界面布局设置、工

扫描全能王 创建

作路径设置、问题反馈等功能

5. 快捷工具栏可以实现文件的打开、新建、保存、程序的运行、调试、代码输入的撤销与恢复等功能

基本功能

北太天元环境的运行行为就像一个很复杂的计算器, 使用者可以在 ≫ 命令提示符后输入命令. 北太天元语言是一个解释型语言, 换言之, 使用者输入一个命令, 北太天元就会自动将其翻译为机器语言并执行.

基本数据格式、结构和输入方式

北太天元中的所有计算都是通过双精度量进行的, 在内存中的数据精度是双精度的, 但其显示格式可以不同. 在命令窗口中, 键入常用的命令 format 可以实现数据的不同显示格式之间的切换. format 命令的主要用法见附表 1:

附表 1　format 命令表

命令	格式
format	(默认值) 短格式, 仅显示小数点后 4 位
format short	短格式, 仅显示小数点后 4 位
format long	长格式, 显示小数点后 15 位 (双精度) 或 7 位 (单精度)
format short e	短格式科学计数法, 显示小数点后 4 位
format long e	长格式科学计数法, 显示小数点后 15 位 (双精度) 或 7 位 (单精度)

北太天元中的数据格式是矩阵是矩阵, 单个变量默认为是 1×1 矩阵. 变量名是以字母开头, 由字母、数字和下划线组成的字符串, 字母的大小写是有区别的. 北太天元中的特殊变量见附表 2:

附表 2　特殊变量表

特殊变量名	说明
pi	π 值
eps	机器极小值 $= 2.2204\text{e}-16$
ans	默认的表示计算结果的变量名
inf	无穷大
NaN	不定量, 如分母出现零
i,j	虚数符号 $\text{i} = \text{j} = \sqrt{-1}$

生成矩阵的方式有多种, 生成小型矩阵的常用方法是直接从命令窗口输入. 整个矩阵以“[ ]”作为首尾, 矩阵的行与行之间用分号“;”相隔, 每行中元素用逗号“,”或空格分隔.

扫描全能王 创建

例如, 在命令窗口键入:

≫ A = [1,2,3;4,5,6;7,8,9]

命令行窗口会显示结果

≫ A =

3 × 3 double

1　2　3

4　5　6

7　8　9

表示变量 A 是一个 3 × 3 矩阵. 如果在上述命令的末尾加上 “;” , 则同样生成矩阵 A, 但并不在命令窗口中显示结果. 形成大规模矩阵的常用方法是使用 load 命令, 直接将数据文件读入. 对于按步长逐渐增加的一维数组, 有更简单的创建方式, 如

≫x=0:0.1:10;

生成了从 0 开始, 每次递增 0.1, 直至 10 的 101 个数据构成的一位数组. 也可调用库函数 linspace 生成一维数组. 附表 3 列出了一些特殊函数, 以便形成常用的特殊矩阵.

附表 3　特殊矩阵生成函数表

函数符号	功能
zero (m,n)	生成 m 行 n 列的全零矩阵
ones (m,n)	生成 m 行 n 列的全 1 矩阵
eye (m,n)	生成 m 行 n 列的对角线元素为 1 的矩阵
rand(m,n)	生成 m 行 n 列的随机矩阵

基本运算符

北太天元提供的常用运算符号见附表 4:

附表 4　常用运算符号表

运算符	使用形式	功能
+	A + B	执行矩阵加法运算, 要求 A、B 维数相同
−	A − B	执行矩阵减法运算, 要求 A、B 维数相同
*	A * B	执行矩阵相乘运算, 要求 A、B 维数满足矩阵相乘的条件
/	B/A	执行矩阵右除运算, 其值为矩阵 $B/A = BA^{-1}$
\	A\B	执行矩阵左除运算, 其值为矩阵 $A\backslash B = A^{-1}B$
.*	A. * B	数组乘法, 运算结果为两个矩阵对应元素相乘, 要求 A、B 维数相同
./	A./B	数组右除, 矩阵 A 中各个元素除以矩阵 B 中对应元素
′	A′	执行矩阵的共轭转置
.′	A.′	执行矩阵的转置

此外, 调用库函数 inv(A)、det(A)、eig(A)、cond(A)、trace(A) 和 rank(A), 可分别得到方阵 A 的逆矩阵、行列式值、特征值、条件数、迹和秩; 调用库函数 pinv(B), 可以求解矩阵 B 的伪逆, 如果 B 是 $m \times n$ 矩阵, 则伪逆的维数为 $n \times m$. 对于非奇异矩阵 B 来

扫描全能王 创建

说, 有 $\mathrm{pinv}(B) = \mathrm{inv}(B)$.

北太天元中的关系运算符和逻辑运算符见附表 5:

附表 5　关系运算和逻辑运算符号表

关系运算符	说明	逻辑运算符	说明
==	相等	&	逻辑与运算
~=	不等于 |	逻辑或运算
<	小于	~	逻辑非运算
<=	小于或等于	xor	逻辑异或运算
>	大于		
>=	大于或等于		

变量与一些命令

北太天元的变量命名规则: 可以是字母、中文字、下划线开头, 后面可以跟字母、中文字、下划线或数字各种组合字符串, 变量名区分字母大小写.

北太天元语句的一般形式为: 变量 = 表达式;

这里的“=”表示赋值, 与 C 语言的用法相同. 需要注意的是

1. 变量在使用之前必须赋值;

2. 当表达式返回未非分配给任何变量的结果时, 系统会将结果分配给默认变量 ans, 该变量在之后可以使用.

除去从工作区查看变量之外, 我们还可以通过 who 命令检查所有的变量, 例如

≫ who

当前工作区变量有: a　　ans

≫ whos

当前工作区变量信息如下:

变量名: a

基本信息: 3×3 double

变量名: ans

基本信息: 1×1 double

除去 who 与 whos, 北太天元还提供了很多的控制命令 (附表 6):

附表 6　控制命令表

命令	功能
clear	清除工作空间的所有变量
clear x	清除工作空间的变量 x
clear all	清除工作空间的所有变量, 函数和 bex 文件
clc	清除命令窗口的内容
clf	清除当前的图形

扫描全能王 创建

(续表)

命令	功能
close	关闭当前的图形窗口
close all	关闭所有的图形窗口
exist	检查文件或者变量的存在
global	声明变量为全局变量
help	搜索帮助主题
quit	停止北太天元

在北太天元中输入≫ help xxx, 可以查看内置函数 xxx 的使用帮助;

≫ help plot

立即得到关于函数 plot 的详细资料:

二维线图

plot(X,Y) 创建 Y 中数据对 X 中对应值的二维线图.

如果 X 和 Y 都是向量, 则它们的长度必须相同.plot 函数绘制 Y 对 X 的图.

如果 X 和 Y 均为矩阵, 则它们的大小必须相同.plot 函数绘制 Y 的列对 X 的列的图.

如果 X 或 Y 中的一个是向量而另一个是矩阵, 则矩阵的各维中必须有一维与向量的长度相等.

……

输入 ≫ help, 可以查看北太天元提供的所有函数命令. 事实上, 北太天元提供了各种有用的命令来处理系统, 例如将当前工作作为文件保存在工作区中, 并在以后加载文件. 它还为其他与系统相关的活动提供各种命令, 例如显示日期, 在目录中列出文件, 显示当前目录等. 感兴趣的用户可以前往北太天元官网与官方社区学习使用.

循环语句与条件语句

北太天元的控制语句与其他语言的类似, 编写简洁, 意义明确.

for 循环语句

例如

```
≫ m=10 ; n=20 ;
≫ for i = 1: m
    for j = 1: n
      A(i, j)=1 /(i+j) ;
    end
  end
```

通过循环语句, 生成了 10 行 20 列的矩阵 A. 循环语句以 end 结束. 循环语句中的步长是可以选择的, 如

扫描全能王 创建

```
≫ for i = 10:-1:-10
    ...
  end;
≫ for  i = 1:0.1:10
    ...
  end;
```

都是允许的.

while 循环语句

程序

```
≫ a=10 ;
≫ i=1 ;
≫ while i <= 100
     a = a + i;
     i = i + 1;
  end;
```

利用 while 循环求 1 到 100 之间所有整数的和.

if 条件语句

条件语句的一般形式为

```
≫ if x>=0
     y = 1;
  else
     y = -1;
  end;
```

其中 else 部分可以缺省, 也可以嵌套使用.

switch 条件语句

除了基本的 if else 之外, 北太天元有一个 switch 语句,switch 块从多个选择中有条件地执行一组语句, 每个选择都包含在 case 语句中.

```
≫ grade = 'B';
≫ switch(grade)
case 'A'
fprintf('Excellent \ n' );
case 'B'
fprintf('Well done\ n');
case 'C'
```


扫描全能王 创建

```
fprintf('Lucky boy\ n' );
case 'D'
fprintf('You passed\ n' );
case 'F'
fprintf('Better try again\ n' );
otherwise
fprintf('Invalid grade \ n' );
end
```

情况较多时,switch 条件语句是比 if 条件语句更好的选择.

常用数学函数

北太天元所支持的部分常用的数学函数见附表 7.

附表 7　常用数学函数表

函数名	数学运算	函数名	数学运算
sin(x)	正弦函数	asin(x)	反正弦函数
cos(x)	余弦函数	acos(x)	反余弦函数
tan(x)	正切函数	atan(x)	反正切函数
min(x)	最小值	max(x)	最大值
abs(x)	绝对值	sqrt(x)	开平方
log(x)	自然对数	log 10(x)	以 10 为底的对数
exp(x)	以 e 为底的指数	sing(x)	符号函数
real(x)	求复数的实部	imag(x)	求复数的虚部
sum(x)	求所有元素的和	round(x)	四舍五入到最接近的整数
fix(x)	向零方向取整数	floor(x)	向负方向取整数

北太天元的库函数是有限的, 使用者可以自己编写函数, 存为后缀为 .m 的文件, 或定义内联函数, 之后即可像调用库函数一样随意调用. 例如: 将程序

```
function p=p(x, y)
p=sin (x^2 +y^2 )/ exp (-x / 2) ;
end
```

存为 p.m 文件之后, 定义了二元函数

$$p(x,y)=\frac{\sin\left(x^2+y^2\right)}{\mathrm{e}^{-x/2}}$$

在命令窗口键入:

```
≫ f=inline(' 30+(1-cos(pi * x)) / 2','x') ;
```

定义了函数:

$$f(x)=30+\frac{1-\cos(\pi x)}{2}$$

扫描全能王 创建

绘 图 功 能

北太天元提供了许多可以选用的图形工具, 几乎满足了一般实际工程、科学计算中的所有图形需要. 用户可以选择直角坐标系、极坐标系等不同的坐标系, 绘制平面曲线、空间曲线、空间网面图、空间表面图、直方图及向量图等. 这里仅介绍最基本的图形操作命令.

二维曲线绘图

plot 是最基本的图形命令, 一般使用格式为

≫ plot(x, y);

≫ plot(x, y, s);

≫ plot(x, y1, s1, x, y2, s2, x, y3, s3);

其中 x, y, y1, y2, y3 是一维数组. 执行命令 plot(x, y) 时, 以 x 的元素值为横坐标, 以 y 的元素值为纵坐标绘制连线图. s 表示用户选定的线型和颜色 .plot (x, y1, s1, x, y2, s2, x, y3, s3) 表示将多条曲线画在同一坐标系下. 北太天元中可选用的线型和颜色选项见附表 8:

附表 8　绘图线型、颜色选项表

线型选项	说明	颜色选项	说明
−	实线	r	红色
−−	虚线	g	绿色
:	点线	b	蓝色
−.	点划线	y	黄色
0	小圆圈	k	黑色
*	星号	w	白色
x	叉号	c	青色
+	加号	m	洋红色
s	小方块		

北太天元还提供了一系列绘图辅助函数, 使得图形便于阅读. 常用的绘图辅助函数见附表 9:

附表 9　绘图辅助函数表

函数名	功能
title('···')	在图像的上方显示 '···' 中指定的内容
xlabel('···')	将 '···' 中指定的内容标注在 x 轴上
ylabel('···')	将 '···' 中指定的内容标注在 y 轴上
text(x,y,'···')	将 '···' 中指定的内容显示在由 x,y 定义的位置上
hold on	保留现有图像窗口, 并将随后的图形叠加在此图上
hold off	解除 hold on 命令, 随后的 plot 命令将覆盖现有的图形
grid	在图形上显示曲线网格
axis([x1 x2 y1 y2])	方括号中的四个数分别确定 x 和 y 方向显示的范围
legend('$L1$', '$L2$', ···)	图例函数, 将 $L1$, $L2$, ··· 标注到图中, 每个标注对应的图标为画图时的图标

扫描全能王 创建

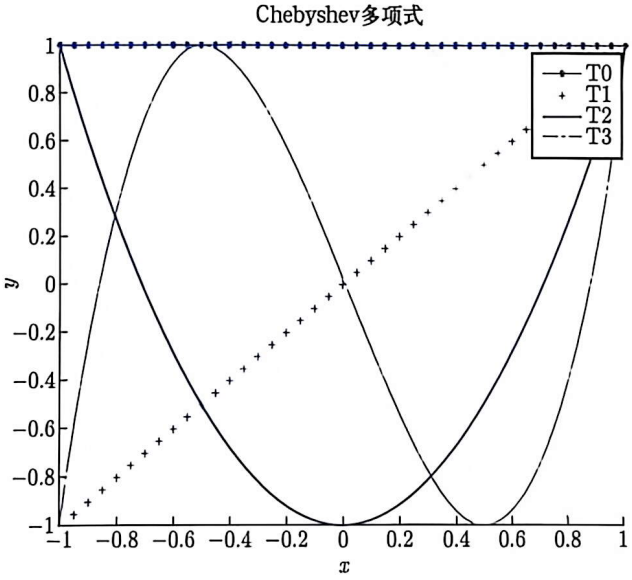
例如, 下面一段程序实现了将区间 [-1,1] 上前四个 Chebyshev 多项式

$$T_0(x) = 1$$
$$T_1(x) = x$$
$$T_2(x) = 2x^2 - 1$$
$$T_3(x) = 4x^3 - 3x$$

绘制在同一张图上的过程:

```
≫ x = -1:0.05:1;
≫t0=1.0*x;
≫t1=x ;
≫ t2=2*x.^2-1.0;
≫t3=4*x.^3-3*x;
≫plot(x,t0,'*-');
≫ title('Chebyshev 多项式')
≫ xlabel('x');ylabel('y');
≫ hold on;
≫plot(x,t1,'g+');
≫plot(x,t2,'r-',x,t3,'k—.');
≫legend('T0','T1','T2','T3');
```

得到的图像如附图 2 所示.



附图 2　Chebyshev 多项式图像

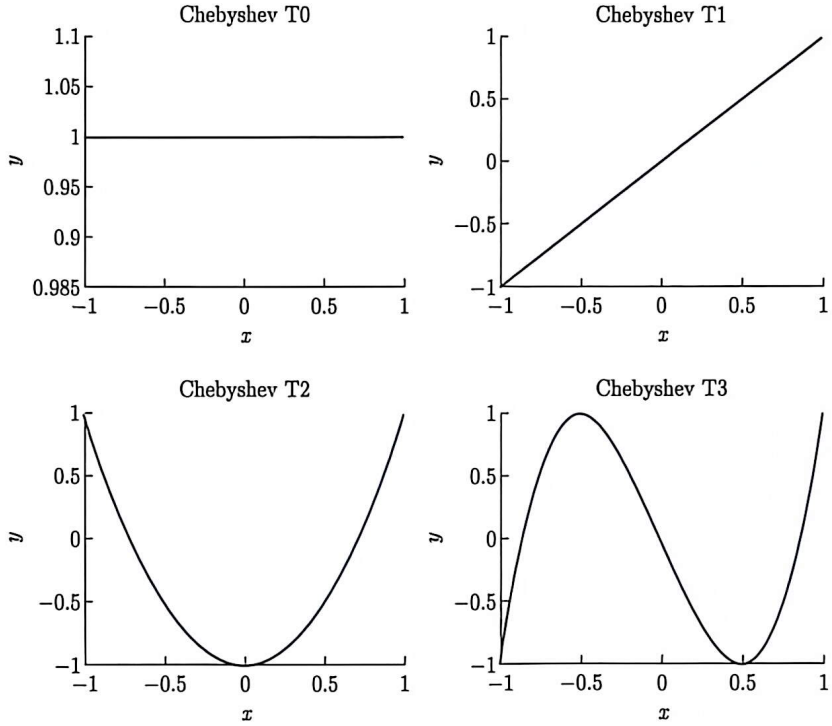
利用命令 subplot (m, n, i) 可将图形窗口分成 m 行 n 列共 m × n 个子窗口, 子窗口按行排序, 并在第 i 个子窗口上绘图. 若将上述程序中的绘图命令改作:

```
≫ subplot([2,2,1]);plot(x,t0);title('Chebyshev T0');
≫ xlabel('x');ylabel('y');
```


扫描全能王 创建

```
≫subplot([2,2,2]);plot(x,t1);title('Chebyshev T1');
≫ xlabel('x');ylabel('y');
≫ subplot([2,2,3]);plot(x,t2);title('Chebyshev T2');
≫xlabel('x');ylabel('y');
≫subplot([2,2,4]);plot(x,t3);title('Chebyshev T3');
≫ xlabel('x');ylabel('y');
```

可以将四个 Chebyshev 多项式的图形分别画在四张子图上, 见附图 3.



附图 3　Chebyshev 多项式子图像

三维图形

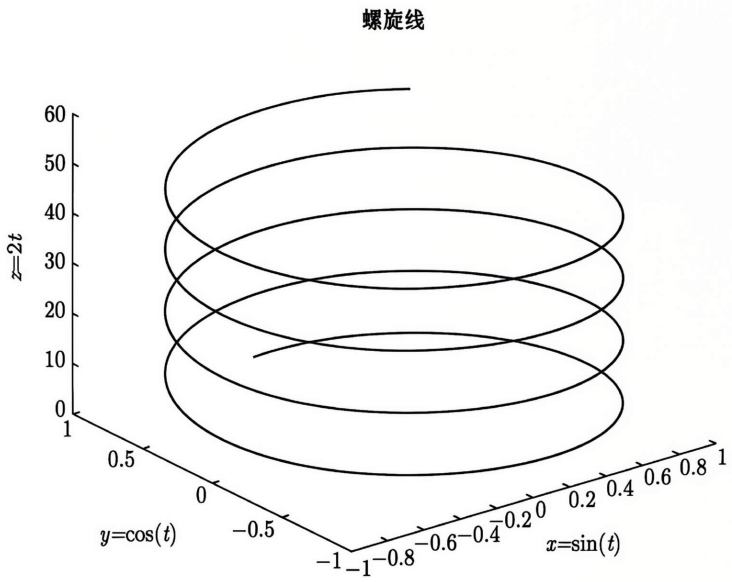
plot3 命令是绘制三维曲线的基本库函数, 它的使用格式与 plot 命令类似, 只是需增加 z 轴方向的数据. 例如, 程序

```
≫ t = 0:pi/60:8*pi ;
≫plot3(sin(t),cos(t),2*t) ;
≫ title('螺旋线') ;
≫ xlabel('x=sin(t)');ylabel('y=cos(t)');zlabel('z=2t');
```

绘制了一条三维螺旋线, 见附图 4.

如果需要画一张三维曲面, 应使用 mesh、meshgrid 和 surf 命令. 绘图时先用 meshgrid 命令产生 xOy 平面的二维网格数据, 再计算 z 轴的数据, 然后进行三维曲面的作图. 下面一段程序作出函数

扫描全能王 创建



附图 4　螺旋线

$$f(x,y)=\frac{\sin\left(\sqrt{x^2+y^2}\right)}{\sqrt{x^2+y^2}}$$

在区域 $[-15,15]\times[-15,15]$ 上的三维图形. 执行程序

≫x=−15:0.5:15; y=x;

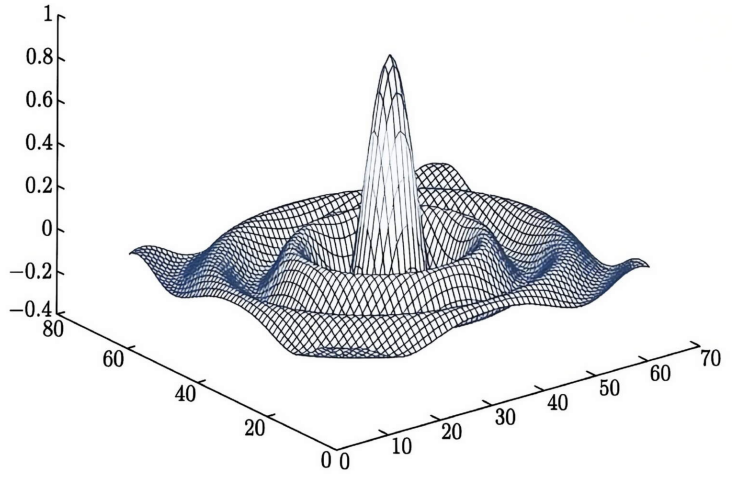
≫ [X,Y]=meshgrid(x,y);

≫r=sqrt(X.^2+Y.^2)+eps;

≫ z=sin(r)./r;

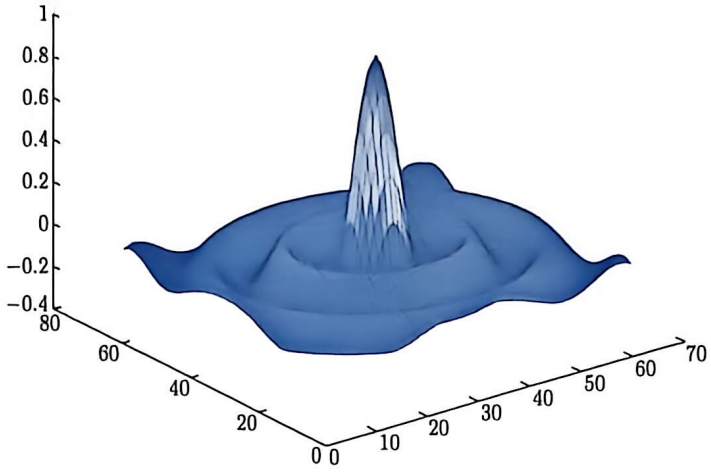
≫mesh(z);

得到如附图 5 的网格图 .Surf 命令得到的是一张曲面图, 而不是网格近似图. 如将 mesh(z) 命令改为 surf(z), 得到如附图 6 的网格图.



附图 5　mesh(z) 曲面图像

扫描全能王 创建



附图 6　surf(z) 曲面图像

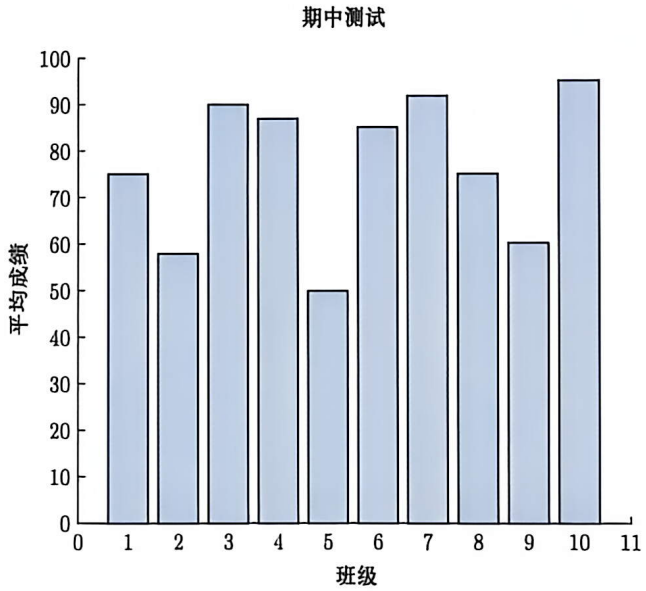
其他图形

除了上述二维、三维图形外, 北太天元可以制作很多图形, 例如条形图、等高线图等. 我们可以利用 bar 函数绘制条形图, 利用 contour 函数绘制等高线图, 利用 scatter 函数绘制散点图等.

例如

```
≫ x = [1:10];
≫ y = [75, 58, 90, 87, 50, 85, 92, 75, 60, 95];
≫ bar(x,y);xlabel('班级');ylabel('平均成绩');
≫ title('期中测试');
```

绘制条形图附图 7.



附图 7　期中测试平均成绩条形图

扫描全能王 创建

数 值 运 算

北太天元提供了丰富的具有数值运算功能的库函数, 涉及数学和科学计算中既有代表性又有实用性的计算, 几乎涵盖了本书中介绍的所有近似计算方法.

数值插值与曲面拟合

北太天元的一维插值命令是 interp1, 使用格式为 interpl(x, y, xi,′method′), 其中 x, y 是已知一维数据, xi 是插值计算点. method 参数可以有四种选择, 它们是: nearest 、linear、cubic、spline, 分别表示最近点插值、线性插值、三次分段插值和三次样条插值. method 参数可以缺省, 默认为 linear.

二维插值是对二元函数作插值近似, 库函数有 interp2. 函数 interp2 的使用格式为 interp2 (x, y, z, xi, yi, ′method′), 其中 x, y 是已知的二维数据, z 是由 x, y 确定的被插值函数的数据,xi, yi 是插值计算点, method 有四种选项: nearst、bilinear、bicubic、spline, 分别表示选用最近点插值、双线性插值、双三次插值和样条插值计算近似值 zi. 例如, 下面一段程序利用二维插值函数对 peak 函数进行插值近似:

≫ [X,Y] = meshgrid(−3 : 0.2 : 3);

≫ Z = peaks(X,Y);

≫ [xi,yi] = meshgrid(−3 : 0.1 : 3);

≫ zi = interp2(X,Y,Z,xi,yi,′bilinear′);

≫ mesh(X,Y,Z);

≫ mesh(xi,yi,zi);

除去插值之外, 我们可以使用 polyfit 函数来求最小二乘拟合多项式的系数, 用 polyval 函数按所得的多项式计算指定值.

曲线拟合

北太天元基于最小二乘原理提供了曲线拟合函数, 用 polyfit 函数来求最小二乘拟合多项式的系数, 用 polyval 函数按所得的多项式计算指定值. 调用方法如下:

p=polyfit(x,y,n). 返回次数为 n 的多项式 p(x) 的系数, 该阶数是 y 中数据的最佳拟合(在最小二乘意义下).

y=polyval(p,x). 计算多项式 p 在 x 的每个点处的值.

数值微分与数值积分

北太天元中提供 diff 函数求解数值微分, 其调用方法如下:

B = diff(A). 返回 A 沿大小不等于 1 的第一个数组维度的相邻元素的一阶差分. 如果 A 为向量, 则 diff(A) 返回元素一阶差分. 如果 A 为非空 m×n 矩阵, 则 diff(A) 将返回

扫描全能王 创建

(m−1)×n 矩阵, 其元素为 A 的行之间的差分. 如果 A 为空矩阵,diff(A) 返回空矩阵.

B = diff(A,n). 返回 A 沿大小不等于 1 的第一个数组维度的相邻元素的 n 阶差分.

B = diff(A,n,dim). 返回 A 沿大小不等于 1 的第 dim 个数组维度的元素的 n 阶差分.dim 取值需为 1 或者 2, dim=1 是行之间的差分, dim=2 是对列之间的差分.

北太天元提供了应用复化梯形法、复化 Simpson 法计算数值积分的函数. 复化梯形法的函数是 trapz, 使用格式为 trapz(x, y), 其中 x, y 是数组. 函数 quad 是用复化 Simpson 公式计算数值积分, 使用格式为 quad('f', a, b, tol), 其中 f 表示已被定义且存为 m 文件的函数, a, b 是积分限, tol 是相对误差限. quad8 是利用高阶 Newton-Cotes 求积公式计算数值积分, 使用格式与 quad 一样.

例如计算积分 $I=\int_0^{\pi}\sqrt{100\sin^2 x+25\cos^2 x}\,\mathrm{d}x$, 需先定义函数

```
function y =f(x)
  y=sqrt(100 * sin(x).^2+25*cos(x).^2);
  end
```

将其存为文件 f.m, 然后调用函数

```
≫ quad('f',0,pi,1.0e-6))
```

得到积分近似 $I=24.2206$

常微分方程数值解

北太天元提供的求解常微分方程数值解的方法是以 Runge-Kutta 法为基础的, 函数有 ode23 和 ode45. ode23 是用二阶或三阶 Runge-Kutta 法求解,ode45 是用四阶或五阶 Runge-Kutta 方法求解. ode23 的使用格式为 ode23($'f', x0, xn$, y0), 其中 f 是自定义的存为 m 文件的函数名, x0, xn 是求解区间 [x0, xn] 的两个端点, y0 是初始值. 例如求解常微分方程的初值问题

$$\begin{cases} y''+(y^2-1)\,y'=0 \\ y(0)=0, y'(0)=0.5 \end{cases} \quad (0<x<10)$$

令 $y_1'=y(x), y_2=y'(x)$, 则原初值问题化为常微分方程组

$$\begin{cases} y_1'=y_2 & (0<x<10) \\ y_2'=(1-y_1^2)\,y_2-y_1 & (0<x<10) \\ y_1(0)=0, y_2(0)=0.5 \end{cases}$$

先建立微分方程组右端函数 pol.m 文件:

```
function z =pol(x,y)
z =[y(2);(1-y(1).^2).*y(2)-y(1)];
end
```

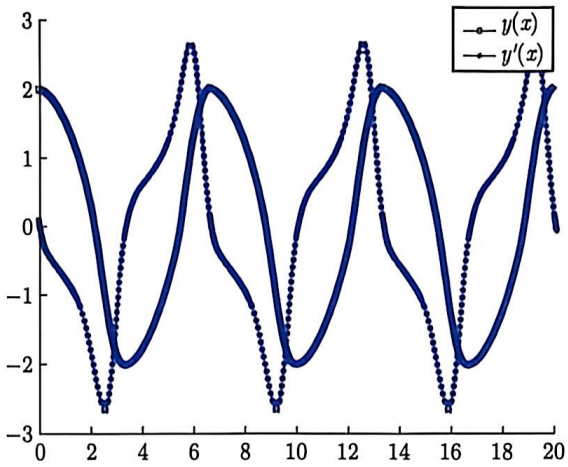
将其存为在北太天元工作目录文件夹下, 然后在命令窗口键入:

```
≫odefun = @pol;
```


扫描全能王 创建

```
≫tspan = [0 20];
≫y0 = [2; 0];
≫options = odeset('RelTol',1e-6,'AbsTol',1e-8);
≫[x,y] = ode23(odefun,tspan,y0,options);
≫plot(x,y(:,1),'-s',x,y(:,2),'-*');
≫ legend('y(x)','y'(x)');
```

计算结果如附图 8.



附图 8　常微分方程组数值解图像

北太天元中更详细的数值计算函数与命令见前面各章节中.

北太天元还在研发各种工具箱, 大致可分为两类: 功能型工具箱和领域型工具箱. 功能型工具箱主要用来扩充北太天元的符号计算功能、图形建模仿真功能、文件处理功能及与硬件实时交互功能, 而领域型工具箱具有很强的专业性. 如通信工具箱、控制系统工具箱、信号处理工具箱、图像处理工具箱、神经网络工具箱、优化工具箱、微分方程工具箱、统计工具箱、符号数学工具箱、小波工具箱等. 有兴趣的读者可查阅相关参考书, 并加入官方社区一起建设北太天元.

扫描全能王 创建

/参考文献/

[1] 豪斯霍尔德 A S. 数值分析中的矩阵论. 孙家昶, 译. 北京: 科学出版社, 1986.

[2] 奥特加 J M. 数值分析. 张丽君, 等译. 北京: 高等教育出版社, 1984.

[3] 徐树方. 矩阵计算的理论与方法. 北京: 北京大学出版社, 1995.

[4] BURDEN R L. Numerical Analysis. 7th ed. 北京: 高等教育出版社, 2001.

[5] BJORCK A. Least Squares Methods. Handbook of Numerical Analysis, Vol.1, North-Holand, Austerdam, 1990.

[6] 张平文, 李铁军. 数值分析. 北京: 北京大学出版社, 2007.

[7] 喻文健. 数值分析与算法. 北京: 清华大学出版社, 2020.

[8] MICHAEL T H. Scientific Computing, An Introductory Survey. 2nd ed. 北京: 清华大学出版社, 2001.

[9] 胡祖炽, 林源渠. 数值分析. 北京: 高等教育出版社, 1986 .

[10] 孙志忠, 袁慰平, 闻震初. 数值分析. 2 版. 南京: 东南大学出版社, 2002.

[11] 袁亚湘, 孙文瑜. 最优化理论与方法. 北京: 科学出版社, 1999.

[12] 余德浩, 汤涛. 微分方程数值解法. 北京: 科学出版社, 2003.

[13] 袁鼎兆. 刚性常微分方程初值问题的数值解法. 北京: 科学出版社, 1987.

[14] 胡祖炽, 雷功炎. 偏微分方程初值问题差分方法. 北京: 北京大学出版社, 1988.

[15] GEAR C W. 常微分方程初值问题的数值解法. 费景高, 等译. 北京: 科学出版社, 1971.

[16] EVANS D S. Sparsity and Its Applications. Cambridge : Cambridge University Press, 1985.

[17] TEWARSON R D. Sparse Matrices. New York: Academic Press, 1973.

[18] 戈卢布 G H, 范洛恩 C F. 矩阵计算. 袁亚湘, 等译. 北京: 科学出版社, 2001.

[19] WILKINSON J H. 代数特征值问题. 石钟慈, 等译. 北京: 科学出版社, 1987.

[20] 曹志浩, 张玉德, 李瑞遐. 矩阵计算与方程求根. 2 版. 北京: 高等教育出版社, 1984.

[21] SCHILLING R J, HARRIS S L. Applied Numerical Methods for Engineers Using MATLAB and C. 北京: 机械工业出版社, 2004.

[22] NUMBERGER G. Approximationly Spline Function. Berlin: Spring-Verlag, 1989.

扫描全能王 创建

[23] 王德人, 杨忠华. 函数逼近论. 北京: 高等教育出版社, 1990.

[24] 关治, 陆金甫. 数值分析基础. 3 版. 北京: 高等教育出版社, 2019.

[25] DAVIS P J, RABINOWITZ P. Methods of Numerical Integration. 2nd ed. Orlando: Academic Press, 1984.

[26] 冯康, 秦孟兆, 哈密尔顿系统的辛几何算法. 杭州: 浙江科学技术出版社, 2003.

扫描全能王 创建

郑重声明

高等教育出版社依法对本书享有专有出版权。任何未经许可的复制、销售行为均违反《中华人民共和国著作权法》，其行为人将承担相应的民事责任和行政责任；构成犯罪的，将被依法追究刑事责任。为了维护市场秩序、保护读者的合法权益，避免读者误用盗版书造成不良后果，我社将配合行政执法部门和司法机关对违法犯罪的单位和个人进行严厉打击。社会各界人士如发现上述侵权行为，希望及时举报，我社将奖励举报有功人员。

反盗版举报电话 （010）58581999 58582371

反盗版举报邮箱 dd@hep.com.cn

通信地址 北京市西城区德外大街 4 号

高等教育出版社知识产权与法律事务部

邮政编码 100120

读者意见反馈

为收集对教材的意见建议，进一步完善教材编写并做好服务工作，读者可将对本教材的意见建议通过如下渠道反馈至我社。

咨询电话 400-810-0598

反馈邮箱 hepsci@pub.hep.cn

通信地址 北京市朝阳区惠新东街 4 号富盛大厦 1 座

高等教育出版社理科事业部

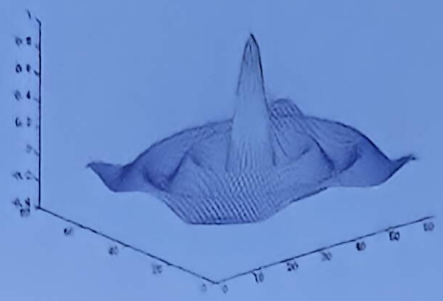
邮政编码 100029

防伪查询说明

用户购书后刮开封底防伪涂层，使用手机微信等软件扫描二维码，会跳转至防伪查询网页，获得所购图书详细信息。

防伪客服电话 （010）58582300

扫描全能王 创建



- 体现数值算法的原理和实用性
  介绍各类数值算法的新发展
- 利用数值计算软件实际模拟
  展示数值算法的计算机实现过程
- 增加数值算法历史与相关数学家简介
  系统揭示数值算法的发展

ISBN 978-7-04-062722-0
9 787040 627220 >
定价 45.90 元

扫描全能王 创建